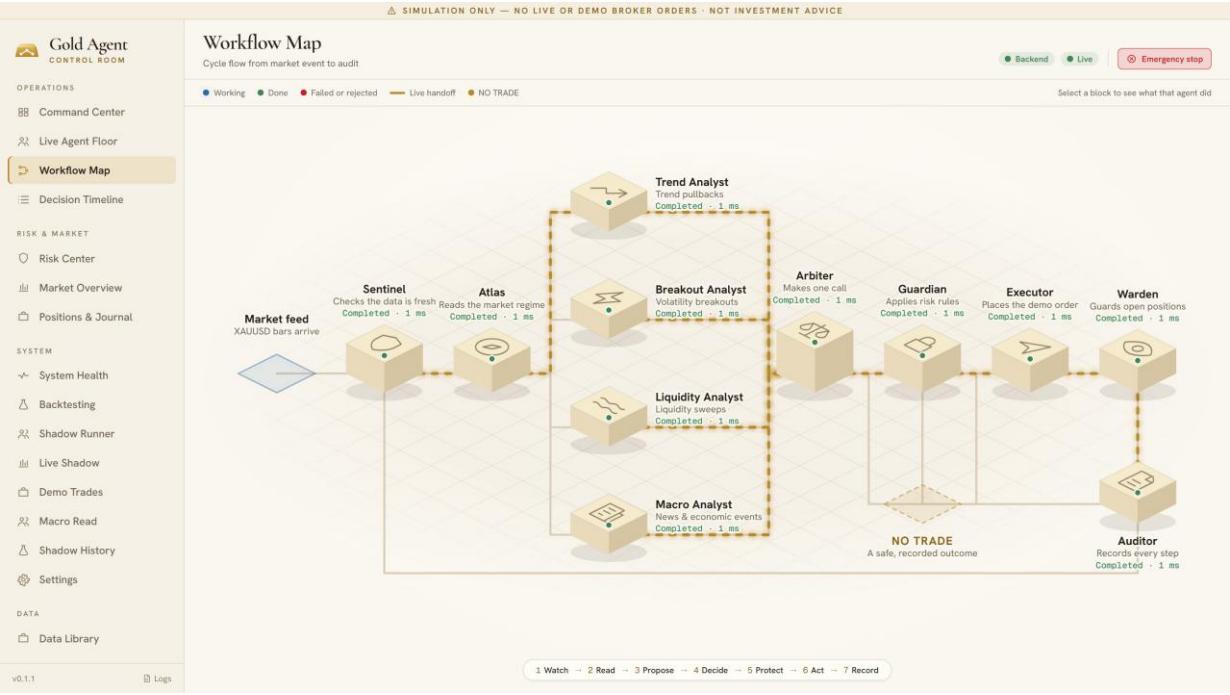
TECHNICAL REPORT

Gold Agent — Control Room

A team of twelve AI agents for researching, simulating and demo-trading gold (XAUUSD) under a deterministic risk engine — design, engineering, operations and measured results.



Ilyas Daoud El Asmi

Version 1.0 · November 2026

Simulation and demo-account results only · Not investment advice · © 2026 Ilyas Daoud El Asmi. All rights reserved.

Contents

1 Introduction 9
1.1 The problem: learning to trade is expensive twice 9
1.2 The idea: a team instead of one trader 9
1.3 Goals 9
1.4 Non-goals 10
1.5 Scope of this report 10
2 The project at a glance 11
2.1 What Gold Agent is 11
2.2 The project in numbers 11
2.3 The building blocks 12
2.4 Technology 12
2.5 Development phases 13
3 How it was built: AI-assisted engineering 14
3.1 The roles 14
3.2 Phase one: two agents, two territories 14
3.3 Phase two: one agent, adversarial self-review 15
3.4 The gates every change had to pass 15
3.5 Local-first periods 16
3.6 Memory between sessions 16
3.7 Human decisions that shaped the project 16
3.8 What worked, and what did not 17
4 Specification and safety invariants 18
4.1 The V1 specification 18
4.2 The eight phases 18
4.3 Hard safety invariants 18
4.4 Deterministic rejection taxonomy 19
5 System architecture 20
5.1 Trust boundaries 20
5.2 Components of the agent service 20
5.3 Contracts and versioning 21
5.4 The event stream 21
5.5 Backend lifecycle supervision 21
5.6 Recovery on start-up 21
5.7 Repository layout 22
6 The data platform 23
6.1 Import pipeline 23
6.2 Storage: an immutable, content-addressed data lake 23

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

6.3 Quality rules and lifecycle ... 23
6.4 Ingesting history from MetaTrader 5 ... 24
6.5 The Data Library screen ... 25
7 The feature engine ... 26
7.1 Guarantees ... 26
7.2 Indicator definitions ... 26
7.3 Gaps and segments — and a silent bug ... 27
8 The agent team ... 29
8.1 The seven steps of a cycle ... 29
8.2 Lifecycle of an agent ... 30
8.3 The agents, one by one ... 31
8.4 How the team is composed in each mode ... 33
9 Strategies ... 34
9.1 A shared contract ... 34
9.2 Why four families ... 34
9.3 Adding a strategy touches the whole chain ... 35
9.4 Research grids ... 35
10 Macro and news intelligence ... 36
10.1 The economic calendar and the blackout gate ... 36
10.2 Sources: free, key-less and unreliable ... 36
10.3 The agents that read the news ... 37
10.4 The macro read in the interface ... 39
10.5 Testing whether the news helps ... 39
11 Decision and risk ... 40
11.1 The Arbiter ... 40
11.2 Position sizing ... 40
11.3 The Guardian's twelve rules ... 41
11.4 The emergency stop ... 42
12 Execution and reconciliation ... 43
12.1 Read-only market access first ... 43
12.2 The MQL5 safety bridge ... 43
12.3 The demo executor ... 43
12.4 Reconciliation ... 44
12.5 The position monitor and trade records ... 45
13 Validation and research methodology ... 47
13.1 Measuring in R ... 47
13.2 Metrics ... 47
13.3 The event-driven backtester ... 48
13.4 Walk-forward validation ... 48
13.5 Pre-registered experiments and statistical judgement ... 49

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

13.6 Stress testing 50
13.7 Shadow mode 50
13.8 Forward shadow history 51
14 Live operations on the cloud 53
14.1 The host 53
14.2 Scheduled tasks 53
14.3 Observability 53
14.4 Reaching the server: remote commands and a private tunnel 54
14.5 The read-only viewer 54
14.6 Costs and limits 55
14.7 Disk and memory budget 56
15 Incidents and fixes 57
15.1 The MetaTrader update loop 58
15.2 The order path that never filled 58
15.3 The macro read inside the broker lock 58
15.4 Memory starvation on a 1 GB server 59
15.5 The weekend false alarm 59
16 Results 60
16.1 The fake-money demo account 60
16.2 Forward shadow history 61
16.3 Backtests 62
16.4 The edge hunt: tuned walk-forward on every strategy and timeframe 64
16.5 The shadow runner over history 65
16.6 What the results mean 65
17 Dead ends and retracted results 66
17.1 Retracted results 66
17.2 Dead ends 67
17.3 Lessons 67
18 The control room 68
18.1 Design language 68
18.2 The sixteen screens 68
18.3 Accessibility and robustness 73
19 Quality and security 74
19.1 Testing 74
19.2 Continuous integration 74
19.3 Security review 75
19.4 Dependency hygiene 76
20 Timeline 77
21 Conclusions and future work 78
21.1 What was achieved 78

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

21.2 What was learned 78
21.3 Who it is for 78
21.4 Future work 78
21.5 Contact 79
22 Appendices 80
22.1 Appendix A — Glossary 80
22.2 Appendix B — Reason-code families 81
22.3 Appendix C — The 38 data-quality rules 81
22.4 Appendix D — Architecture decision records 81
22.5 Appendix E — Formula sheet 82
22.6 Appendix F — Figures from the cloud viewer 82

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

About this report

This report documents **Gold Agent — Control Room** from its first commit to its public showcase: why it exists, how it is designed, how each of its twelve agents works, how it was engineered and operated, what went wrong along the way, and what it measured. It is written for three audiences at once: engineers and recruiters who want to judge the quality of the system, traders and prop firms who want to understand what it does and what it does not do, and general readers who want the story.

The report presents the project; it is not a manual. It describes components, rules, metrics and results in enough depth to evaluate them, but deliberately leaves out source code, installation steps and the exact entry and exit parameters of the trading strategies. Gold Agent is a private project and is not open source.

> **Reading guide.** Chapters 1–4 explain the problem, the project and how it was built. Chapters 5–13 describe the system, component by component, with the mathematics used for risk and measurement. Chapters 14–15 cover live operation on a cloud server and the incidents it produced. Chapters 16–17 report every result, including the negative ones, and the dead ends. Chapter 18 tours the interface. Chapters 19–21 close with quality, the timeline and conclusions. Appendices hold the glossary and reference tables.

Table 1. Conventions used in this report

Notation	Meaning
R	One unit of risk: the loss taken if a trade hits its stop. A result of +1.5R means a win of 1.5 times the amount risked.
XAUUSD	Spot gold priced in US dollars.
M15, H1, H4	Bars of 15 minutes, 1 hour and 4 hours.
NO TRADE	The safe default outcome of a decision cycle.
Demo account	A broker account funded with fake money. No real money was ever traded.
Shadow	Recording what the system *would* have done, without sending any order.
`monospace`	A named state, rule, reason code or configuration value.

All dates are in 2026 and all times in UTC unless stated otherwise. All numbers come from the project's own audit store, research records or the screens of the application, and are reported as measured, losses included.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Executive summary

The problem. Trading gold well takes months, often years, to learn, and most people pay for that learning twice: once in time spent watching charts and again in money lost while they learn. The work itself is repetitive and unforgiving: check the data, read the market regime, look for setups, weigh the news, size the risk, place the order, manage it and keep a record, every fifteen minutes, without fatigue and without breaking the rules after a losing streak.

The idea. Gold Agent splits that job across a team of twelve specialised agents that work on the same market at the same time, each with exactly one responsibility. A data guard, a regime reader, four analysts (trend, breakout, liquidity and macro-news), an arbiter that makes one decision, a deterministic risk engine that cannot be overridden, an executor, a position manager, an auditor, and an offline researcher. A desktop control room makes every step visible: who decided what, on which data, with what confidence and why a trade was or was not taken.

What was built. In under four months the project grew from an empty repository into a complete platform:

- a versioned data platform with 38 deterministic quality rules and an immutable, content-addressed data lake;
- a causal feature engine and four strategy families;
- a macro and news layer in which language-model agents read the economic calendar and live headlines, debate under a rule that can only make them more cautious, and produce an advisory macro read;
- an arbiter, a position-sizing bridge and a twelve-rule risk engine;
- an MQL5 safety bridge and a guarded order path to a MetaTrader 5 demo account;
- a research harness with an event-driven backtester, walk-forward validation, stress tests, pre-registered experiments with Bonferroni-corrected significance, shadow mode and forward shadow history;
- an unattended cloud deployment that ran the whole team around the clock on a fake-money demo account, with self-healing operations and daily reports;
- a sixteen-screen desktop control room, redesigned in an ivory-and-gold visual language, with a read-only viewer for the cloud system.

What it measured. The platform works as designed: it refused to trade on stale data, around high-impact news and whenever any rule was broken, and every decision is traceable. The strategies it runs do not make money on the data tested so far.

−$309.41 Demo account, 30 fills, net of costs	**5 / 25** Demo trades won / lost (17%)	**−45.00R** Shadow history, 158 counted proposals	**+41.64R** H4 backtest, 732 trades — not significant
−95.30R M15 backtest, 1,019 trades	**0** Strategies with a demonstrated edge	**12** Agents in the team	**16** Screens in the control room

> **Conclusion.** No technical strategy has shown a statistically demonstrated edge on gold across four strategy families, three timeframes and up to 21 years of history. The project's lasting value is the machinery: a transparent, auditable agent team that knows when not to trade, and a measurement loop

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

> honest enough to say so. The research continues with fundamental and macro inputs, tested offline before anything reaches the demo account again.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

1 Introduction

1.1 The problem: learning to trade is expensive twice

Retail trading attracts people with a simple promise and confronts them with a hard reality. Becoming consistently competent at trading a single instrument such as gold usually takes months of screen time and often years of practice, and the cost of that education is paid twice. The first cost is time: hours each day spent watching charts, reading news, waiting for setups and reviewing mistakes. The second cost is money: the losses taken while the trader is still learning which rules matter, which do not, and how hard it is to follow them under pressure.

What makes the job hard is not any single decision but the discipline of making all of them, every time, in the right order. Before every trade someone has to answer the same questions:

1. Is the market data I am looking at fresh and correct?
2. What kind of market is this — trending, ranging or volatile?
3. Is there a setup that my rules recognise?
4. Is there news or an economic release about to move the price?
5. If I take this trade, how much can I lose, and is that within my limits for the day and the week?
6. If I am in a trade, am I managing it by my rules rather than by my feelings?
7. Am I keeping a record that would let me learn from it?

A human can answer these questions well once. Answering them every fifteen minutes, for weeks, without fatigue, without skipping a step after a winning streak and without revenge-trading after a losing one, is where most people fail.

1.2 The idea: a team instead of one trader

Gold Agent starts from a different assumption: the job is too broad for one decision-maker, human or machine. Instead of asking one model or one person to do everything, the work is split across a team of specialised agents. Each agent has exactly one responsibility, works on the same market bar at the same time as its colleagues, and hands a structured result to the next step.

Two design choices follow from this. First, **the agents that look for trades never control risk, and the agent that controls risk never looks for trades.** The boundary between ideas and money is a hard one, enforced in code. Second, **NO TRADE is the default.** A cycle produces an order only when every step succeeds; any missing input, timeout, disagreement or broken rule ends it safely, with a record of why.

The third design choice is visibility. A trading system that cannot be inspected cannot be trusted, so the project's front end is not a chart with a buy button but a **control room**: a desktop application that shows every agent's state, inputs, outputs, confidence, reasoning and timing, live.

1.3 Goals

- Build a complete, inspectable decision pipeline for one instrument (XAUUSD) from raw data to an audited order.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

- Keep every component that touches money deterministic, testable and impossible to override from the interface.
- Use language models only where judgement over unstructured text is required (news and the economic calendar), and only in an advisory role that can stop a trade but never start or size one.
- Validate every strategy with methods that resist self-deception: out-of-sample walk-forward tests, multiple-comparison correction, stress tests, shadow mode and forward tracking.
- Run the full team unattended on real market conditions with a fake-money demo account, and measure the result honestly.
- Present all of it in an interface that a recruiter, a trader and a general reader can understand in seconds.

1.4 Non-goals

- **No live-money trading.** Live orders are permanently disabled in configuration and in code.
- **No profitability claims.** The project never claims accuracy, profit or protection from loss.
- **No model-controlled risk.** No language model or strategy chooses position size, risk or deployment.
- **No treating a short demo period or a single backtest as evidence** of an edge.

1.5 Scope of this report

The report covers the project from its first commit on 21 July 2026 to the public showcase on 8 November 2026: the method used to build it (Chapter 3), the specification (Chapter 4), each subsystem (Chapters 5–13), live operations and incidents (Chapters 14–15), every measured result and every dead end (Chapters 16–17), the interface (Chapter 18), quality and security (Chapter 19), the timeline (Chapter 20) and conclusions (Chapter 21).

> **A note on honesty.** Every number in this report is reported as measured, including the losing ones. Several results that looked promising during the project were later shown to be artefacts of small samples, data bugs or backfilled records; those stories are told in full, because the process that caught them is one of the project's real achievements.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

2 The project at a glance

2.1 What Gold Agent is

Gold Agent — Control Room is a Windows desktop platform, backed by a Python agent service, that researches, simulates and demo-trades a single instrument: spot gold against the US dollar (XAUUSD). It has two ways to run:

- **Local simulation.** The desktop application starts and supervises its own agent service on the same machine. The trading mode is locked to simulation; orders go to a deterministic simulated broker. This mode hosts the research tools: the data library, backtester, walk-forward and stress tests, and the shadow runner.
- **Cloud demo.** The same agent team runs around the clock on a Windows server in the cloud, connected to a MetaTrader 5 terminal logged into a fake-money demo account. The desktop reaches it only through a private, read-only tunnel: it can display everything and change nothing.

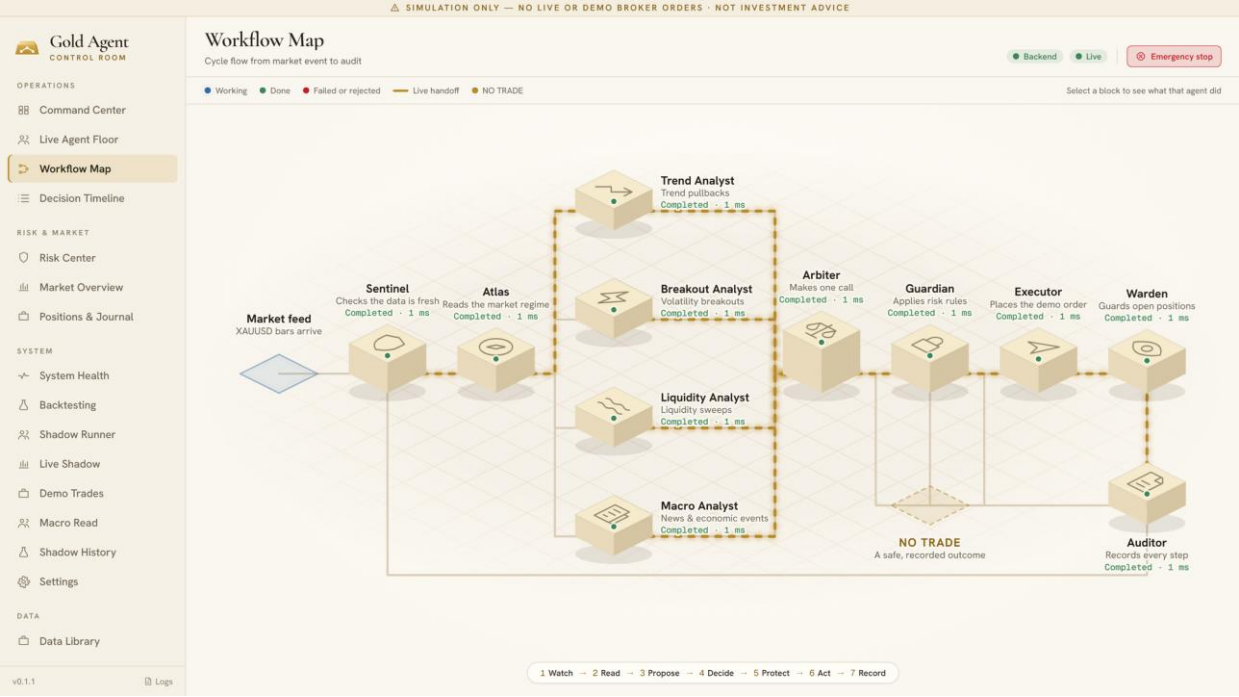


Figure 1. The Workflow Map: one market bar travelling through the agent team, from the Sentinel to the Auditor (local simulation).

2.2 The project in numbers

21,000 lines of Python in the agent service	**12,000** lines of TypeScript in the desktop app	**13,600** lines of backend tests	**860+** backend test cases
173 desktop and contract unit tests	**15** end-to-end journeys in the real app	**41** design documents	**7** architecture decision records

38 pull requests merged	**183** commits across all branches	**38** data-quality rules	**12** risk rules in the Guardian
4 strategy families tested	**3** timeframes (M15, H1, H4)	**50,000** M15 bars in the main dataset	**21 years** of H4 history (2005–2026)

2.3 The building blocks

Table 2. Main subsystems and where they are described

Subsystem	What it does	Chapter
Data platform	Imports, validates and stores historical XAUUSD bars immutably; 38 quality rules; MT5 history ingest	6
Feature engine	Computes causal indicators (EMA, ATR, RSI, volatility, bands, channels, swings, higher-timeframe trend)	7
Agent team	Twelve agents in a seven-step cycle; lifecycle states, handoffs and audit	8
Strategies	Trend pullback, volatility breakout, range reversion, liquidity sweep	9
Macro and news	Economic calendar, multi-source headlines, sentiment and macro agents, risk challenger	10
Decision and risk	Arbiter, position sizing, twelve-rule Guardian, emergency stop	11
Execution	MQL5 safety bridge, demo executor, reconciliation, position monitor	12
Validation	Backtester, walk-forward, stress tests, experiments, shadow mode, shadow history	13
Operations	Cloud host, scheduled tasks, health monitor, Telegram reports, backups, read-only viewer	14
Control room	Sixteen-screen desktop application and its design system	18

2.4 Technology

Table 3. Technology stack

Layer	Technology
Desktop shell	Electron with context isolation, a narrow typed preload bridge, no Node integration in the renderer
Interface	React 18, TypeScript, Tailwind CSS, TanStack Query, Zustand; isometric SVG workflow stage
Agent service	Python 3.12, FastAPI (REST and WebSocket), Pydantic contracts, SQLAlchemy
Storage	SQLite audit store; Zstandard-compressed Apache Parquet data lake, content-addressed by SHA-256
Market access	MetaTrader 5 terminal via its Python package; an MQL5 safety Expert Advisor
Language models	An open-weight model served through OpenAI-compatible APIs, with quota-aware failover across providers
Cloud	A Windows server on AWS; Systems Manager for remote commands and private port forwarding
Quality	pytest, Ruff, strict mypy, Vitest, Playwright end-to-end tests of the real Electron app, secret scanning, dependency audit
Packaging	electron-builder (NSIS installer and portable ZIP) with a bundled, self-contained Python backend

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

2.5 Development phases



Figure 2. Development phases from the first commit on 21 July 2026 to the public showcase on 8 November 2026.

The project moved in four distinct phases. A **foundation** built in a day established contracts, the simulated agent team, the risk engine and the desktop control room. An **intensive research-platform build** over the next three days delivered the data platform, features, strategies, every validation tool, the MetaTrader 5 integration, the live shadow loop and the fundamental layer. A long period of **unattended live operation** on the cloud demo followed from late July, punctuated by incidents, hardening, new agent capabilities and the first real demo trades. The final phase, in November, produced the **market-hours fix, the backup plan, the full visual redesign and the public showcase**. Chapter 20 gives the dated detail.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

3 How it was built: AI-assisted engineering

Gold Agent was built by one person directing AI coding agents. This chapter describes that process: how work was divided, how quality was enforced, what the human decided, and what the approach did well and badly. No tool is named; what matters is the discipline, not the brand.

3.1 The roles

The project had one human owner and, over its life, two AI engineering agents working in the same repository.

- **The owner** set the product direction, wrote the original specification with an AI assistant, chose between options at every fork, approved every change that touched safety, operated the cloud account, and personally made the one configuration change that allowed demo orders.
- **Engineering agent A** was responsible, in the first phase, for the agent service: the backend, the data platform, persistence, migrations and the shared contracts. It also acted as the reviewer and merger of pull requests.
- **Engineering agent B** was responsible, in the first phase, for the desktop application: the Electron shell, the React interface, packaging and the end-to-end tests of the real application.

From the second day onwards a single engineering agent owned the entire repository — backend, desktop, contracts, documentation, operations scripts, review and merge — under a stricter self-review discipline described below.

3.2 Phase one: two agents, two territories

The first day's work was split by ownership area so that two agents could progress in parallel without overwriting each other.

Table 4. Work in the two-agent phase

Area	Work delivered
Agent service and contracts	Repository governance, architecture documents, typed monorepo; versioned Pydantic contracts and their TypeScript mirror; FastAPI REST and WebSocket API; the seeded simulation orchestrator with twelve agents; lifecycle state machine; deterministic risk engine; SQLite audit store; fake broker; the historical XAUUSD importer with 38 quality rules and the content-addressed data lake; code review and merging
Desktop application	Electron main process with single-instance lock, backend supervision and health probing; per-launch capability-token authentication; narrow preload bridge; the original control-room screens; guarded emergency-stop recovery dialogs; packaged installer and portable build; Playwright end-to-end tests of the real app; the Data Library interface with a secure native file-picker handoff

Coordination happened through the repository itself rather than through conversation:

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

- **A coordination folder** held a handoff protocol and a decisions log. Each agent read them before starting and appended to them when it finished, so the next session — by either agent — started from a written state rather than a memory.
- **Contracts first.** The shared vocabulary (agent events, cycles, decisions, risk assessments, positions) was defined as versioned schemas with an exported fixture set; a check in CI fails if the Python and TypeScript sides drift.
- **Issues as a backlog.** Work was planned as numbered items (DEV-001 to DEV-035, plus patches and operations items), each with a scope and acceptance criteria, mapped onto the specification's eight phases.
- **Pull requests with evidence.** Each change arrived as a focused branch and pull request containing what changed, why, and the test evidence; the reviewing agent checked it before merge.

3.3 Phase two: one agent, adversarial self-review

When the second agent became unavailable, a single agent took over everything. Losing the second pair of eyes was treated as a risk, and the process was tightened rather than relaxed:

1. **A dedicated review pass after every change**, written from the point of view of an adversary looking for failure modes rather than confirmation, with up to two rounds of fixes.
2. **Extra care on safety code.** Any change touching risk, orders, reconciliation or the emergency stop required explicit failure-injection tests before merge.
3. **Hermetic tests.** A discovered defect — the test suite had been connecting to the live MetaTrader terminal on the developer's machine because it read the local environment file — led to a rule that every test constructing settings must ignore the environment file.
4. **The owner keeps the keys.** Lifting the configuration barrier that forbids demo orders was deliberately left to the owner to edit by hand; the engineering agent was not permitted to make that change.

3.4 The gates every change had to pass

Table 5. Quality gates

Gate	What it checks
Ruff	Python lint and style (line length 100)
mypy (strict)	Static types across the agent service, including unused type-ignores
pytest	More than 860 backend test cases, judged by exit code
Contract fixtures	Python and TypeScript contracts serialise identically
Secret scan	No credential-shaped strings in the repository
ESLint and TypeScript	Desktop lint with zero warnings; type checks for main and renderer
Vitest	Desktop and contract unit tests
Dependency audit	No high or critical advisory in the dependency tree
Packaging	Builds the bundled backend, installer and portable ZIP
Playwright	End-to-end journeys in the real Electron app, plus a journey of the packaged installer covering the full safety lifecycle

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Two continuous-integration workflows ran these gates: a quality workflow on every change, and a Windows release gate that also built the installer and ran the packaged journey.

3.5 Local-first periods

In late July the hosted CI storage quota filled up with packaged build artefacts, and further runs would have required a paid plan. The project switched to a **local-first** workflow: the full gate suite was run on the development machine, work was committed to the local main branch, and nothing was pushed until the owner chose to sync. Artefact retention was cut to a few days so that a later return to hosted CI would not repeat the problem. The repository was synchronised again in November, at which point every gate — including the Windows release gate — passed on GitHub.

3.6 Memory between sessions

AI sessions do not remember previous sessions on their own. The project used three layers of written memory so that each session could resume precisely:

- **The repository**: design documents, architecture decision records and commit messages that state the reasoning behind each change.
- **The coordination files**: the handoff protocol and decisions log.
- **Persistent working notes** kept outside the repository: the current state, verified facts about the environment (for example, which dataset identifiers exist, which commands are safe to run, and which local quirks are expected), open decisions for the owner, and an explicit record of retracted conclusions.

The last item mattered more than expected. Several research results were corrected after deeper analysis (Chapter 17). The working notes recorded the correction prominently next to the original claim, with an instruction never to cite the retracted number, so that a later session could not resurrect it.

3.7 Human decisions that shaped the project

Table 6. Decisions taken by the owner

Decision	Choice	Consequence
Order of work	Data, runner, shadow and validation first; real demo orders last	The full research harness existed before any order could be sent
Data source	The broker's own history through MetaTrader 5, rather than purchased CSV files	Realistic but gap-ridden data, honestly labelled QUARANTINED
Fundamental sources	Free, key-less public feeds only	Calendar and headline feeds with rate limits, handled by caching and multi-source aggregation
Failure policy for the macro layer	Blackout (no trading) when the macro read fails	Safety over activity; required careful handling of feed health
Demo orders	Enabled by the owner on the cloud host only, never live orders	The first real order path, behind every gate in Chapters 11–12
Cloud server size	Upgraded from 1 GB to 2 GB of memory after a week of paging incidents	Stable operation, at a higher credit burn
Hosting budget	Stay on the free credit plan; take a full backup before the credits end	Backup tooling built and scheduled (Chapter 14)

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Decision	Choice	Consequence
Presentation	Light, ivory-and-gold design; a public showcase without source code	The redesign, the showcase repository, the film and this report

3.8 What worked, and what did not

Worked well. Written contracts and decision records let work continue across many sessions without loss of context. Mandatory tests and gates caught regressions that a human reviewer would likely have missed, including two caused by the redesign itself. Making safety rules structural — a monotonic challenger, a constant `orders_sent = 0` in shadow mode, a promotion field that can only say `manual_review_required` — turned good intentions into properties that tests can check.

Worked badly. AI agents can be confidently wrong about the world outside the code: several operational assumptions (that an installer would upgrade MetaTrader, that a feed was alive, that a symbol bitmask was an order policy) were only disproved in production. The remedy was the same each time — measure the real system before trusting a fix — and it is now part of the method. The other weakness is optimism about early research numbers; the project's statistical discipline exists precisely because promising results kept dissolving under scrutiny.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

4 Specification and safety invariants

4.1 The V1 specification

The project began from a written V1 specification that fixed the product's shape before any code existed. Its central rules were:

- **Language models never execute trades.** They may only emit structured, schema-validated proposals or assessments.
- **Risk and execution are deterministic.** Every rule that touches money is plain code with tests.
- **Fail closed.** Any uncertainty resolves to NO TRADE.
- **One instrument.** XAUUSD only in V1, decided on 15-minute bars with 1-hour and 4-hour context.
- **Three initial strategies:** trend pullback, volatility breakout and range mean-reversion (a fourth, liquidity sweep, was added later).
- **Prohibited techniques:** martingale, grid trading, averaging down, and any position sizing by a language model or a strategy.
- **A validation ladder before money:** backtest, walk-forward, stress test, shadow mode, demo account — and only then, if ever, a restricted real-money pilot.

4.2 The eight phases

Table 7. The specification's eight phases and their status in November 2026

Phase	Content	Status
1 · Data recorder	Historical import, quality rules, immutable storage, MT5 history ingest	Complete
2 · Deterministic baseline	Feature engine, strategies, risk engine, backtester	Complete
3 · Execution simulator	Simulated broker, shadow mode, shadow runner	Complete
4 · MT5 demo execution	Read-only adapter, MQL5 safety bridge, guarded demo executor, reconciliation	Complete and running
5 · Macro and news agent	Calendar, blackout gate, headline sentiment, macro agent, risk challenger	Complete and running
6 · Multi-agent arbiter	Evidence combination, macro gate, confidence model	Complete
7 · Research agent	Pre-registered experiments, walk-forward search, analytics; no automatic deployment	Complete
8 · Restricted real-money pilot	Only after demonstrated out-of-sample edge	**Not started — no edge demonstrated**

4.3 Hard safety invariants

These properties held from the first commit and were never relaxed except where the table says so.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Table 8. Safety invariants

Invariant	How it is enforced
Live-money orders are impossible	`ALLOW_LIVE_ORDERS` is permanently rejected at configuration load; no live adapter exists
Demo orders are opt-in	Defaults are `TRADING_MODE=simulation` and `ALLOW_DEMO_ORDERS=false`; the cloud host alone was switched to demo by the owner
Only the risk engine sizes positions	Strategies and the Arbiter emit levels and confidence only; volume is computed in one module and approved by the Guardian
The Guardian cannot be bypassed	No interface path to approve, resize or force an order; a rejection always ends in NO TRADE
The emergency stop is durable	Persists across reset, restart, relaunch and upgrade; cleared only by a guarded, audited procedure
No fabricated data or results	The importer never fills, interpolates or repairs bars; synthetic data is never research-eligible
No secrets in the repository	Credentials live only in local environment files with restrictive permissions; a secret scan runs in CI
The renderer is untrusted	No shell, filesystem or secret access; only a narrow, typed preload API
Models cannot widen risk	The risk challenger can only tighten the macro read (Chapter 10)

4.4 Deterministic rejection taxonomy

When something blocks a trade, the reason is never free text alone; it is a stable code that can be counted, tested and displayed. Examples used throughout this report include `macro_blackout`, `macro_unavailable`, `no_strategy_signals`, `strategy_disagreement`, `insufficient_agreement`, `below_min_volume`, `proposal_stale`, `news_blackout`, `spread_too_wide`, `min_reward_risk`, `position_already_open` and `RECONCILE_MISMATCH`. Appendix B lists the families.

> **Why codes matter.** In September the cloud demo had attempted 126 orders without a single fill. Because every outcome carried a code, the breakdown — 92 stale proposals, 11 gate rejections, 23 broker failures — pointed straight at the real defect (Chapter 15). Free-text logs would have hidden it.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

5 System architecture

5.1 Trust boundaries

The architecture is organised around who and what can be trusted. The renderer — the part of the desktop application that draws the interface — is treated as untrusted: it receives display data and sends a small set of narrowly defined commands, and it never receives arbitrary process, filesystem, secret or shell capability. Electron runs with context isolation on and Node integration off in the renderer, and every privileged action passes through a typed preload bridge.

The agent service binds to the loopback interface only. Every REST call and the event WebSocket require a per-launch capability token, generated by the desktop's main process, stored in a protected file and presented as a header (and as a WebSocket subprotocol). A malformed token is shape-checked before a constant-time comparison so that it is rejected and audited rather than crashing the request.

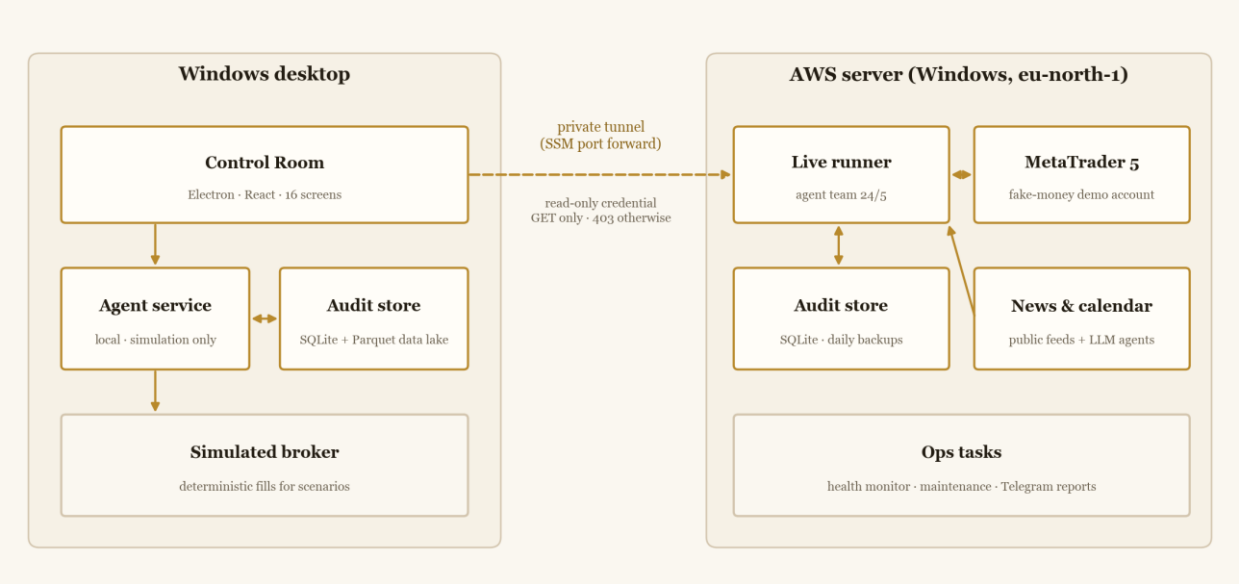


Figure 3. Deployment: the desktop runs its own simulation service locally, and reaches the cloud demo only through a private, read-only tunnel.

5.2 Components of the agent service

Table 9. Agent-service modules

Module	Responsibility
Contracts	Immutable public vocabulary with strict v1 validation; every object carries a schema version, UUIDs and a UTC timestamp
Lifecycle	The permitted agent-state transitions (Chapter 8)
Orchestrator	The seeded simulation workflow for the control room's scenarios
Data platform	Import, quality, storage, manifests, MT5 history ingest
Features	Causal indicator frames, cached by content hash
Strategies	Four deterministic strategy evaluators
Agents	News sentiment, macro-news and risk-challenger agents; tool runtime; blackboard

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Module	Responsibility
Fundamentals	Calendar and headline feeds, aggregation, the live macro provider, bias tracker
Arbiter	Combines strategy signals and the macro read into one decision
Position sizing	Converts a decision into a volume within risk limits
Risk	The deterministic Guardian and its persistent state
MT5	Read-only adapter, MQL5 bridge protocol, demo order client
Execution	Demo executor and its gates
Reconciliation	Ledger-versus-broker comparison and the execution gate
Live runner and services	The per-bar live loop, its supervisor, the demo service and position monitor
Backtesting and research	Backtester, walk-forward, stress tests, experiments, analytics
Shadow	Shadow sessions, the decision runner, shadow history and outcome scoring
Database	Append-oriented audit of events, cycles, safety records, proposals, executions and reconciliations
API routers	Narrow read endpoints per domain, plus a few guarded commands

5.3 Contracts and versioning

Every public object — an agent event, a cycle, a strategy signal, a risk assessment, an execution outcome — is a versioned schema with a `schema_version`, a UUID `id`, a UUID `cycle_id` and a timezone-aware timestamp normalised to UTC. Additive changes must be optional or carry safe defaults; a breaking change requires a new API version, a decision record and coordinated interface work. The same schemas are mirrored in TypeScript for the desktop, and an exported fixture set guarantees that both sides serialise identically.

5.4 The event stream

The interface does not poll for most of its state. The agent service publishes every lifecycle event, decision and safety action on a WebSocket. The desktop keeps a bounded in-memory window of the stream (1,500 events and 300 cycles) for display; the audit store remains the source of truth. After every reconnection the desktop re-synchronises from the REST API, so a dropped connection never leaves the interface showing a stale picture.

5.5 Backend lifecycle supervision

In local mode the desktop's main process owns the agent service. It takes a single-instance lock, probes the health endpoint before spawning (and adopts a healthy service it finds instead of starting a duplicate), launches only the packaged Python executable with fixed arguments, captures its output to rotating logs, restarts it on failure with a bounded deadline, shows a degraded screen when it cannot recover, and terminates it cleanly on exit. The packaged end-to-end test kills the service mid-session and verifies that the desktop restarts it and that the safety state survives.

5.6 Recovery on start-up

On every start the service creates or migrates the database, reads the persisted emergency-stop state, reconciles open positions and only then advertises readiness. If reconciliation is uncertain, a rule blocks new

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

orders. The emergency stop remains active across reset, restart, relaunch and upgrade while the service remains available for inspection and for the guarded recovery procedure.

5.7 Repository layout

The repository is a typed monorepo with three workspaces — the agent service, the desktop application and the shared contracts — plus documentation, operations scripts for the cloud host, the MQL5 Expert Advisor and research scripts. Generated data, local environment files and build outputs are excluded from version control by fail-closed ignore rules, with explicit re-includes for source folders that happen to be named like data folders.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

6 The data platform

Every result in this project rests on historical price data, so the data platform was built first and built strictly. Its governing rule is simple: **the platform never repairs, deletes, fills, interpolates or forward-fills a market record.** It records what it received, judges it against explicit rules, and labels the result.

6.1 Import pipeline

An import is a two-step, transactional process.

1. **Preflight.** The file is streamed into a controlled staging area under a safe name. The exact bytes are hashed, the format (CSV, JSONL or Parquet) is detected, bounded metadata is inspected and a preview is produced. Links, reparse points and oversized inputs are rejected.
2. **Commit.** The commit references the staging identifier and an idempotency key. The service re-hashes the staged bytes and the configuration; if either changed since preflight, the commit fails. This closes the race in which a file could be swapped between inspection and import.

In the desktop, the file picker runs in the privileged main process. The renderer never sees the file's bytes or path; it receives an opaque selection handle, and the main process hands the staged bytes to the service.

6.2 Storage: an immutable, content-addressed data lake

Table 10. Storage layers

Layer	Content	Identity
Raw	The original bytes, unchanged	SHA-256 of the bytes
Canonical	Normalised bar records in Zstandard-compressed Apache Parquet	SHA-256 of canonical length-prefixed JSON (not of the Parquet bytes, which can vary)
Manifest	Dataset identity, version, status, quality report and findings, lineage	Logical identity `symbol:source-symbol:timeframe` plus version

Content is immutable. A new version of the same logical dataset must explicitly supersede the previous one; the old version remains, marked `SUPERSEDED`. Re-fetching longer history from the broker uses this mechanism to grow a dataset without losing the record of what was used before.

6.3 Quality rules and lifecycle

The quality engine evaluates 38 deterministic, versioned rules. Each rule declares its inputs, logic, severity, effect on the dataset's lifecycle, the evidence it records and a suggested remediation.

Table 11. Quality rule families

Family	Rules	Examples of what they catch
File and schema	10	Unsupported format, invalid encoding, empty data, missing or duplicate columns
Time	11	Missing or ambiguous timezone, invalid or future timestamps, duplicates, out-of-order bars, unknown or unexpected gaps

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Family	Rules	Examples of what they catch
Prices	6	Non-finite or non-positive prices, a high below the open, low or close, impossible bar ranges
Spread and ticks	7	Negative spread, bid above ask, missing spread where required, unsafe spread conversion
Volume and source	4	Negative volume, unknown volume meaning, ambiguous real-versus-tick volume, synthetic test data

$$\begin{aligned} \text{status} = \text{REJECTED} &\text{ if any REJECT finding} \\ \text{else QUARANTINED} &\text{ if any QUARANTINE finding} \\ \text{else VALIDATED} & \end{aligned}$$

Synthetic fixtures are always marked and can never become research-eligible, at any status.

Gaps: expected or not

A gap in the bars is only "expected" when versioned trading sessions prove that the market was closed. Gold trades in near-continuous sessions from 22:00 to 21:00 UTC, opening Sunday to Thursday evening, with a daily maintenance break and the weekend closure. When the MetaTrader 5 history is ingested, the symbol's specification is turned into an instrument description with these sessions attached, so the daily break and the weekend are classified as `TIME.EXPECTED_SESSION_GAP` (informational). Without sessions every gap would be `TIME.UNKNOWN_SESSION_GAP` and the dataset quarantined.

> **Real broker data is almost always QUARANTINED.** Even with sessions attached, real MetaTrader 5 history contains genuine one-bar holes and a daily break that shifts with daylight-saving time. The project never fills those holes to manufacture a VALIDATED label. Instead, research tools accept QUARANTINED real data while displaying a prominent warning, and every result computed on it is labelled "indicative only".

6.4 Ingesting history from MetaTrader 5

The chosen source of historical data was the broker itself, through the read-only MetaTrader 5 adapter. A command-line tool fetches bars from the terminal, converts them into the importer's input format and runs the normal import pipeline, so broker data passes exactly the same quality rules as any file. Times are normalised to UTC using the server's configured offset.

Table 12. Datasets used for research

Dataset	Bars	Coverage	Status
XAUUSD M15	50,000	June 2024 – July 2026	QUARANTINED
XAUUSD H1	50,000	2018 – 2026	QUARANTINED
XAUUSD H4	33,000	2005 – 2026	QUARANTINED

The M15 dataset was grown in two steps: a first 20,000-bar window and then the terminal's 50,000-bar ceiling, through an automatic supersede. That growth turned out to be scientifically important: a trend-pullback result that looked promising on 20,000 bars regressed to almost nothing on 50,000 (Chapter 17).

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

6.5 The Data Library screen

The desktop's Data Library lists every dataset with its timeframe, row count, coverage, version, lifecycle status and research eligibility; it opens a detail view with the quality report and its findings, and offers a guarded import wizard in local mode. In the read-only cloud viewer, import is disabled by design.

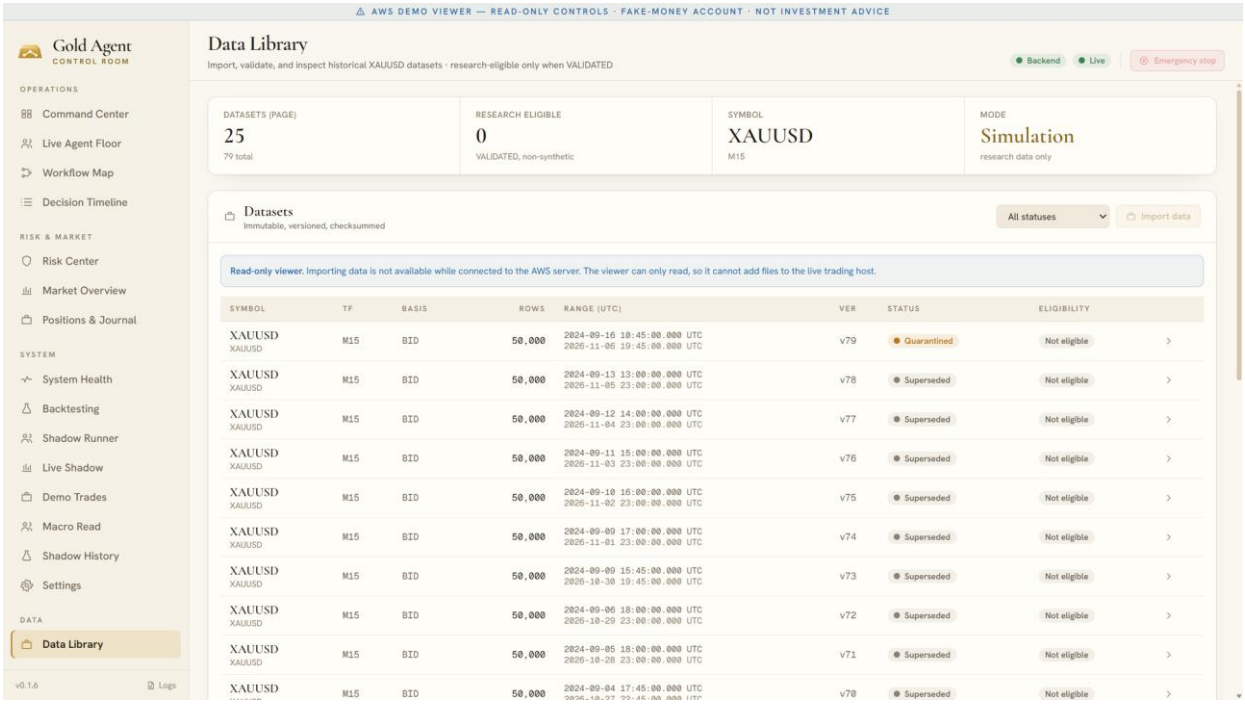


Figure 4. Data Library in the cloud viewer: dataset versions, coverage and quality status. Superseded versions remain visible.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

7 The feature engine

The feature engine turns bars into the indicators the strategies reason about. It is pure, deterministic Python with three guarantees: the same inputs always produce the same outputs, the features of a bar depend only on that bar and earlier bars, and missing data produces missing features rather than invented values.

7.1 Guarantees

- **Deterministic.** Sums use exact floating-point summation (`math.fsum`), and every output frame carries a canonical hash of the inputs, configuration and engine version.
- **Causal (no look-ahead).** Features for bar *t* depend only on bars 0 to *t*. Channels use the *prior* N bars, so a close above the channel is a genuine breakout; swing points are only reported once confirmed; higher-timeframe trend becomes visible only after the higher-timeframe bar has closed.
- **Fail-closed.** A feature whose warm-up is incomplete, or whose inputs are missing, is None, and strategies treat None as NO TRADE.
- **Cached by content.** Frames are stored in a content-addressed cache keyed by the dataset's content hash, the configuration and the versions, so a backtest never recomputes identical features.

7.2 Indicator definitions

Let C(t), H(t) and L(t) denote the close, high and low of bar *t*, and *n* the period of an indicator.

Returns

$$r(t) = C(t) \,/\, C(t-1) - 1 \qquad \ell(t) = \ln\left(C(t) \,/\, C(t-1) \right)$$

Trend

$$SMA_n(t) = (1/n) \cdot \Sigma_\{i=0..n-1\}\, C(t-i)$$

$$EMA(t) = \alpha \cdot C(t) + (1-\alpha) \cdot EMA(t-1), \quad \alpha = 2 \,/\, (n+1)$$

(seeded with the first SMA)

The EMA slope (the change of the EMA over one bar) indicates direction and strength. Default periods are 20 for both averages.

Volatility

$$TR(t) = \max\left(H(t) - L(t),\ |H(t) - C(t-1)|,\ |L(t) - C(t-1)| \right)$$

$$ATR_n(t) = (1/n) \cdot \Sigma_\{i=0..n-1\}\, TR(t-i), \quad ATR\%(t) = ATR_n(t) \,/\, C(t)$$

$$\sigma_realised(t) = \text{population standard deviation of } \ell(t-n+1) \ldots \ell(t)$$

ATR uses a 14-bar period; realised volatility a 20-bar window of log returns.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Momentum: Wilder's RSI

$$G(t) = [\ (n-1) \cdot G(t-1) + \max(\Delta C, 0)\ ] / n$$
$$D(t) = [\ (n-1) \cdot D(t-1) + \max(-\Delta C, 0)\ ] / n$$

$$RSI(t) = 100 - 100 / (1 + G(t)/D(t))$$

The averages are seeded with a simple mean over the first *n* = 14 changes. The RSI is 100 on a pure rise and 50 when the price is flat.

Bands and channels

$$\text{Bollinger: mid} = SMA_20$$
$$\text{upper / lower} = \text{mid} \pm k \cdot \sigma_price(20)$$
$$\text{width} = (\text{upper} - \text{lower}) / \text{mid}$$

$$\text{Donchian: upper}(t) = \max H(t-N \ldots t-1)$$
$$\text{lower}(t) = \min L(t-N \ldots t-1)$$
$$\text{(the current bar is excluded)}$$

Structure, candles, volume, spread and session

- **Swings.** A fractal swing high is a bar whose high exceeds the highs of the *k* bars on each side; it is reported only after the *k* following bars have closed, so it never uses the future. A close beyond the last confirmed swing marks a break of structure.
- **Candle shape.** Body, upper wick and lower wick as fractions of the bar's range (undefined for a zero-range bar).
- **Relative volume.** Tick volume divided by its 20-bar rolling mean.
- **Spread pressure.** The bar's spread divided by ATR, where both are available.
- **Session.** A UTC-hour bucket: Asian 00–06, London 07–11, New York 12–20, otherwise off-hours.
- **Higher-timeframe trend.** For each 15-minute bar, the trend of the last completed H1 and H4 bars — UP, DOWN, FLAT or NONE — from an EMA over completed, fully populated higher-timeframe buckets.

> **Indicators are inputs, not signals.** No single indicator ever triggers a trade. Indicators feed the strategy hypotheses of Chapter 9, which combine several of them with explicit gates.

7.3 Gaps and segments — and a silent bug

A series of bars is interrupted by market closures and by genuine holes. Indicators must not compute across a real hole as if the bars were contiguous, so the engine splits the series into **segments** and resets the indicators at the start of each one.

Version 1.0.0 of the engine reset at *every* gap — including gold's normal daily break and every weekend. Nothing failed and every test passed, but the effect on research was severe: the indicators restarted every day on M15 and H1 and every week on H4, so most of the history could not be evaluated at all.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

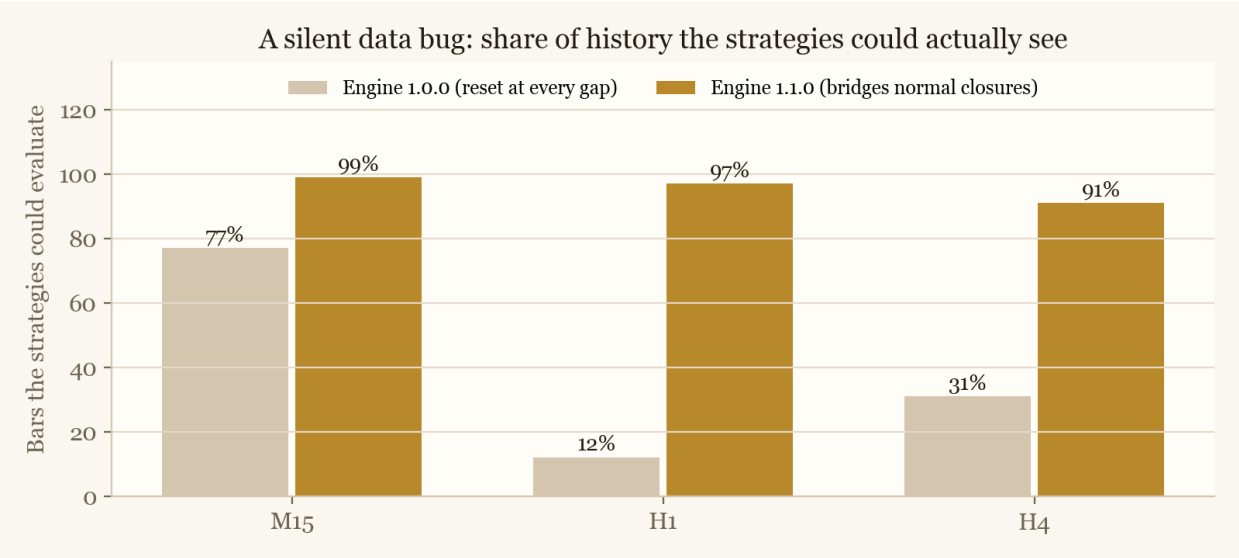


Figure 5. Share of the history on which the strategies could actually be evaluated, before and after the fix.

Version 1.1.0 **bridges** normal closures: a gap of up to 240 minutes (the daily break or a missing bar or two) and the Friday-to-Sunday weekend of up to 72 hours are treated as continuous; anything longer, or a closure on another weekday (holidays), still starts a new segment. A flag still marks the first bar after every gap, so strategies skip the reopening bar. Both limits are part of the configuration — and therefore of the cache identity and of every experiment record — and setting both to zero reproduces version 1.0.0 exactly.

Table 13. Gap treatment in engine 1.1.0

Gap	Treatment
Up to 240 minutes	Bridged — daily maintenance break, or one or two missing bars
Friday close to Sunday/Monday open, up to 72 hours	Bridged — the normal weekend
Anything longer, or a closure starting on another weekday	New segment, indicators reset

The fix mattered for more than coverage. An apparent edge in the 4-hour trend-pullback backtest (+0.24R per trade, t = 2.78) turned out to be largely an artefact of the weekly reset; with closures bridged it fell to +0.06R per trade (t = 1.06). Every research result before the fix was re-run on engine 1.1.0, and the older numbers were formally retired (Chapter 17).

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

8 The agent team

Gold Agent is organised as a team of twelve agents. The word "agent" is used in the engineering sense: a component with one responsibility, explicit inputs and outputs, an observable lifecycle and a defined failure behaviour. Only the news and macro components call a language model; every agent that touches signals, risk or orders is deterministic code. This is a deliberate choice, not a limitation: the parts of trading that can be specified precisely are specified precisely, and judgement over unstructured text is confined to an advisory role.

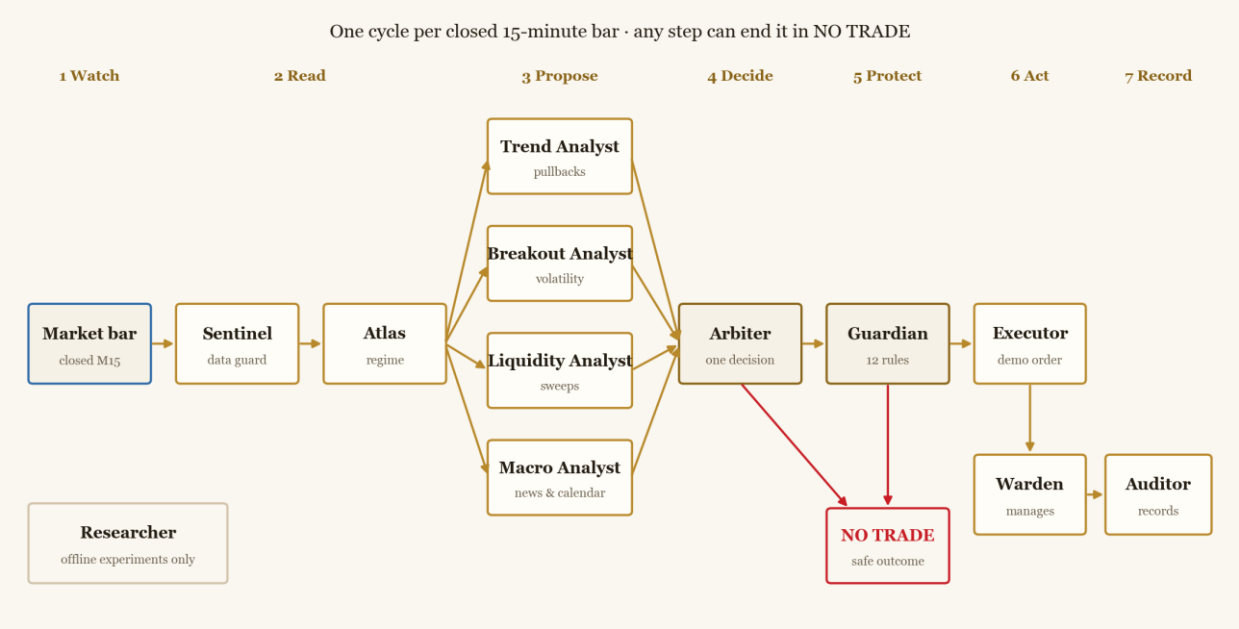


Figure 6. The seven-step cycle. One cycle runs per closed 15-minute bar, and any step can end it in NO TRADE.

8.1 The seven steps of a cycle

Table 14. Steps of a decision cycle

Step	Agents	Question answered	Typical failure outcome
1 · Watch	Sentinel	Is the data fresh, complete and well-formed?	Stale or missing data ends the cycle
2 · Read	Atlas	What kind of market is this?	No regime, no analysis
3 · Propose	Trend, Breakout, Liquidity/Range, Macro analysts	Does any specialist see a setup? Is it safe to trade the news?	No setup or a news blackout
4 · Decide	Arbiter	What is the one call, given all evidence?	Disagreement or low confidence
5 · Protect	Guardian	Does the call respect every risk rule?	A rule rejection
6 · Act	Executor, Warden	Place the order, then manage the position	Broker refusal, gate failure
7 · Record	Auditor	Write every step down	— (always runs)

A twelfth agent, the Researcher, works beside the team offline and has no path to the Guardian or the Executor.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

8.2 Lifecycle of an agent

Every agent reports its state on the event stream. The permitted transitions are enforced in code; an attempt to move an agent along a transition that is not on this list raises an error rather than silently corrupting the display.

Table 15. Agent lifecycle states

State	Meaning
`offline`	Not running (for example, before start-up)
`idle`	Ready, waiting for a cycle
`queued`	Scheduled for the current cycle
`running`	Working on the current bar
`waiting`	Waiting on an input or a dependency
`completed`	Finished successfully and handed off
`rejected`	Finished with a deliberate refusal (for example, a Guardian rejection)
`failed`	Encountered an error; the cycle fails closed
`timed_out`	Did not finish within its time budget; treated as no evidence
`blocked`	Prevented from running by a safety condition (stale feed, emergency stop)
`disabled`	Switched off by configuration (for example, the offline Researcher)

$$\text{offline} \to \text{idle} \to \text{queued} \to \text{running} \to \{\ \text{waiting}, \text{completed}, \text{rejected}, \text{failed}, \text{timed_out}, \text{blocked}\ \}$$

Each event carries the agent's task, confidence, a summary of inputs and outputs, a list of reasoning lines, the duration, an optional error code and the agent it handed off to. The Live Agent Floor and the Workflow Map render exactly this information.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

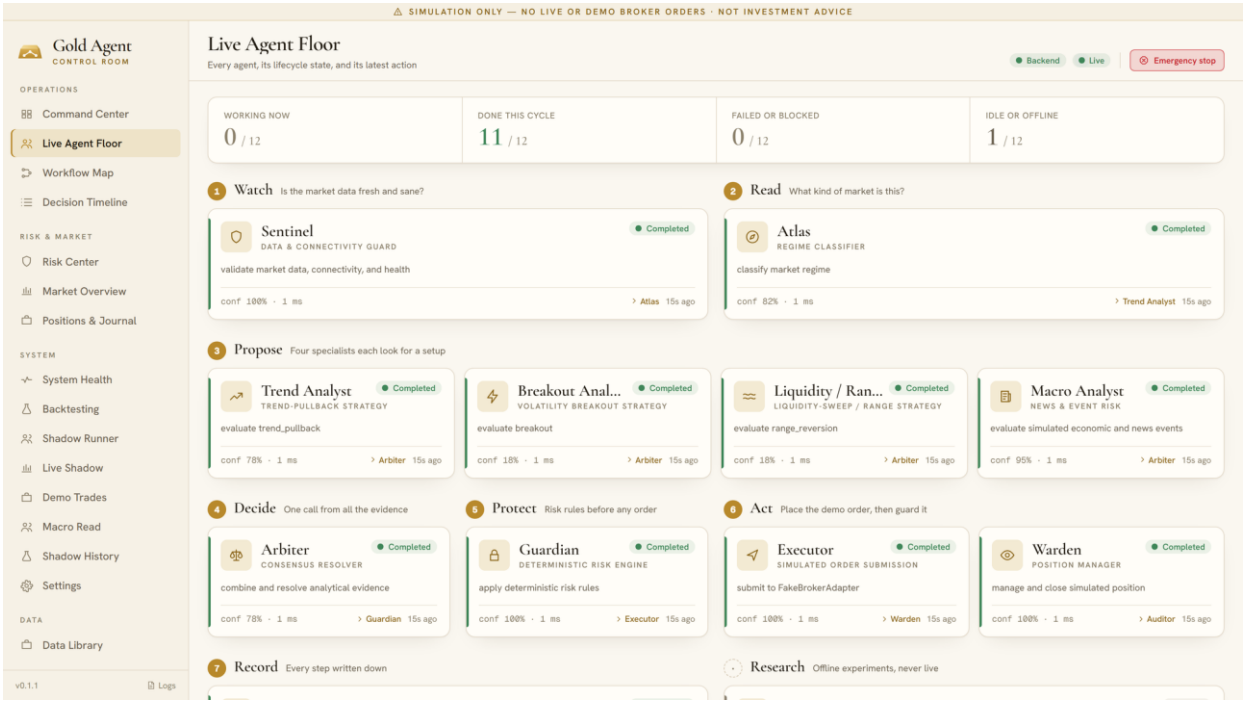


Figure 7. Live Agent Floor (local simulation): all twelve agents grouped by the seven steps, with state, confidence, timing and handoff.

8.3 The agents, one by one

1. Sentinel — data and connectivity guard

Job. Validate the incoming market data, the connection to the broker and the health of the system before anything else runs.

Inputs. The newest closed bars, the latest quote, the feed's timestamps and the system's health signals. **Output.** A verdict: data complete, fresh and well-formed — or a failure with the reason.

How it decides. The newest closed bar must be younger than the freshness bound, prices must be finite and positive, the bar's internal structure must be consistent, and the quote's bid must not exceed its ask. In live operation the loop drops the still-forming candle by timestamp (the broker's history call includes it), so decisions are taken only on closed bars.

Failure behaviour. Any failure ends the cycle. In November a market-hours model was added so that a closed market is reported as `MARKET_CLOSED` (with the reopening time) instead of a stale-feed failure; a feed that dies *before* the session close is still reported as stale. Chapter 15 tells the story.

2. Atlas — regime classifier

Job. Classify the current market regime — trending, ranging or volatile — with a confidence score, so that each strategy is judged in the context it was designed for.

Inputs. The causal feature frame: higher-timeframe trend, EMA slope, volatility and band width. **Output.** A regime label and confidence, published on the event stream and shown on the Command Center ("Trending 82%").

Failure behaviour. No regime means no analysis; strategies that require a particular regime will not fire.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

3. Trend Analyst — trend-pullback strategy

Job. Look for a pullback inside an established trend and, when the conditions align, emit directional evidence with entry, stop, target and confidence.

How it decides. It requires agreement between the 1-hour and 4-hour trend, a pullback towards the moving average, momentum in a defined band, confirmation on the 15-minute bar and acceptable spread, volatility, stop distance and reward-to-risk. If any condition fails, it records the blocking reason and returns NO TRADE.

4. Breakout Analyst — volatility-breakout strategy

Job. Detect a decisive close beyond a compressed prior range, filtered against false breakouts, weak volume and wide spreads.

Time budget. The breakout analyst has a strict budget; if it cannot complete in time it returns "no evidence" rather than delaying the cycle. The simulated scenarios include exactly this timeout, and the control room shows it as `timed_out` without blocking the rest of the team.

5. Liquidity / Range Analyst — sweep and range strategies

Job. In the live runner, detect a liquidity sweep: a false break of a prior channel extreme — the price pokes beyond it, closes back inside and leaves a rejection wick — and propose a trade against the sweep. In the simulation workflow, evaluate range mean-reversion setups that are only valid in a flat higher-timeframe regime.

6. Macro Analyst — news and economic-event risk

Job. Read the live economic calendar and recent headlines and produce a macro read: whether a blackout is in force around a high-impact event, a risk level, a directional bias and a confidence, with the evidence it cited.

How it is built. This is the only part of the team that uses language models, and it is itself a small team (Chapter 10): a sentiment agent scores headlines, a macro agent writes the read, and a risk challenger reviews it and may only make it more cautious. The read is refreshed every 15 minutes outside the order-path lock.

Authority. Advisory only. It can stop a trade (blackout) and reduce confidence (risk level). It cannot start a trade, choose its direction or size it.

7. Arbiter — consensus resolver

Job. Combine the analysts' evidence into a single decision — buy, sell or NO TRADE — with levels and a combined confidence. Its rules are in Chapter 11; in short: a macro blackout or missing macro read ends the cycle, conflicting directions end the cycle, and the confidence of the best agreeing signal is reduced by the macro risk level before being compared with a minimum.

8. Guardian — deterministic risk engine

Job. Check every proposal against twelve fixed rules (Chapter 11). A rejection forces NO TRADE and records the rule that caused it. The Guardian is ordinary deterministic code: no model chooses risk or position size, and the interface has no path to override it.

9. Executor — order submission

Job. Submit only Guardian-approved orders. In the local app it calls a simulated broker; on the cloud demo it places orders on a fake-money MetaTrader 5 account through a guarded order client (Chapter 12). Each order carries a deterministic client identifier so that a retry can never create a duplicate.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

10. Warden — position manager

Job. Manage the open position until its stop or target. Stops are never widened. On the cloud demo, a position monitor reconciles the system's ledger against the broker every cycle and reads the broker's deal history to record each trade's exit price, net result and close reason.

11. Auditor — audit and reconciliation

Job. Record the complete cycle in UTC — every event, decision, order attempt and safety action — and reconcile state after an interruption. The audit store is append-oriented and is the source of every number in this report.

12. Researcher — experiment framework

Job. Run backtests, walk-forward tests, stress tests and shadow runs over historical data, entirely offline. Its output is an experiment record whose promotion field can only ever read `manual_review_required`: the Researcher can produce evidence but cannot deploy anything.

8.4 How the team is composed in each mode

Table 16. The same team in three runtimes

Runtime	Data source	Strategies	Macro read	Orders
Simulation scenarios (control room)	Seeded synthetic scenarios	Simulated analysts with scripted outcomes	Simulated events	Simulated broker
Shadow runner (research)	An imported historical dataset	The real four strategy evaluators	Neutral or replayed	None — `orders_sent = 0`
Live runner (cloud demo)	MetaTrader 5, closed M15 bars	The real strategy evaluators	Live calendar and headlines through the agent team	Fake-money demo account, behind every gate

The simulation scenarios exist to exercise the interface and the safety lifecycle deterministically: a valid trend trade, no valid setup, a news blackout, an analyst timeout, a risk rejection, a stale feed. The live runner is the real thing.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

9 Strategies

Four strategy families were built and tested. This chapter describes what each one looks for and how it is constrained; the exact parameters are intentionally not published.

9.1 A shared contract

Every strategy is a pure function of one bar and its causal features. It returns either NO TRADE with the list of conditions that blocked it, or a signal containing a direction, entry, stop, target, confidence and the evidence behind it.

- **Levels only.** A strategy never decides volume or risk. Sizing happens later, in one place (Chapter 11).
- **NO TRADE by default.** Every gate must pass; a missing feature is a failed gate.
- **Self-gating by regime.** Each strategy declares the market conditions it is designed for and does nothing outside them.
- **Bounded parameters.** Every parameter has a documented range; research grids search only inside those ranges.
- **Common gates.** Every signal must pass spread, volatility, minimum and maximum stop distance and minimum reward-to-risk checks before it is emitted.

Table 17. The four strategy families

Strategy	Market idea	Regime	Key evidence
Trend pullback	Join an established trend after a retracement	Trending, with 1-hour and 4-hour agreement	Higher-timeframe trend, distance to the moving average, momentum band, 15-minute confirmation
Volatility breakout	Trade a decisive close beyond a compressed range	Compression followed by expansion	Prior channel, close beyond it, range compression, relative volume, false-breakout filter
Range mean-reversion	Fade an extreme back to the mean	Flat higher-timeframe regime only	Band extremes, momentum extremes, distance from the mean
Liquidity sweep	Fade a false break of a prior extreme	Any, near recent highs and lows	Poke beyond the prior channel, close back inside, rejection wick

9.2 Why four families

The project started with the three families of the V1 specification. When more data made the trend-pullback result weaker rather than stronger, re-tuning the same rules was judged low-value: a larger grid buys a better in-sample fit and a harder statistical bar at the same time. A fourth, structurally different idea — the liquidity sweep, an approximate mirror of the breakout — was added instead, so that the conclusion "technical setups have no edge here" would rest on genuinely different hypotheses rather than variations of one.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

9.3 Adding a strategy touches the whole chain

Because the strategy name is part of the typed contracts, adding a family is deliberate work: the strategy module, the shared signal and evidence types (in Python and in the TypeScript mirror), the backtester's configuration union and dispatch, the decision runner's default evaluators, the research grids and their candidate counts, the edge-hunt hypotheses, tests and documentation. The Arbiter is name-agnostic and needs no change. This friction is intentional: a strategy cannot quietly enter the live path.

9.4 Research grids

Table 18. Bounded search grids used by the experiment framework

Strategy	Axes searched	Candidates
Trend pullback	Momentum band (coupled low/high), distance to the moving average, maximum stop, take-profit reward-to-risk	108
Volatility breakout	Break decisiveness, relative volume, maximum range, take-profit reward-to-risk	144
Range mean-reversion	Momentum extremes (coupled), stop buffer, reward-to-risk	80
Liquidity sweep	Rejection-wick strength, sweep depth beyond the level, take-profit reward-to-risk, maximum body fraction	108

Axes that reject almost nothing on the real data are frozen, with the reason stated, because searching them would only raise the multiple-comparison penalty without adding information. Momentum bands are varied as coupled pairs so that asymmetric bands, which would encode a hidden directional bias, cannot be selected by accident.

> **Outcome in one sentence.** Across four families, three timeframes and up to 21 years of history, no strategy cleared the pre-registered bar. The full results are in Chapter 16.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

10 Macro and news intelligence

Gold moves on news: central-bank decisions, inflation prints, employment reports, geopolitics. A purely technical system is blind to all of it. The macro layer gives the team eyes on the calendar and the headlines while keeping the language models that read them on a short leash: their output can stop a trade, never start one.

10.1 The economic calendar and the blackout gate

The calendar module normalises raw events from a source into a typed calendar: times converted to UTC, impact mapped to LOW, MEDIUM or HIGH (holidays and "no impact" entries dropped, unknown impact values rejected), duplicates merged to their highest impact, and a stable identifier derived from a hash of each event. Every calendar records its source and when it was fetched, and carries a time-to-live.

A blackout policy then answers one question for any moment: is trading allowed?

$$\text{blackout}(t) = \exists\ \text{event } e : \text{currency}(e) \in C \ \wedge\ \text{impact}(e) \geq I_min \ \wedge \\ t \in [\ \text{time}(e) - \Delta_before\ ,\ \text{time}(e) + \Delta_after\ ]$$

where *C* is the set of relevant currencies (with an "ALL" token available), *I_min* the minimum impact and the two Δ values the windows before and after the release. The gate is **fail-closed**: a missing or expired calendar is itself a blackout. The gate can only suppress trading.

10.2 Sources: free, key-less and unreliable

The owner chose free public sources only, with no API keys. That choice made reliability the central engineering problem.

Table 19. News and calendar sources, as probed in production

Source	Kind	Status in production
Weekly calendar JSON (a public mirror of a popular forex calendar)	Calendar	Working; titles, countries, times, impact, forecast and previous values — but no actual values
A global news database's document API	Headlines	Working but strictly rate-limited (about one request per five seconds); returns plain text when exceeded
A forex news site's RSS feed	Headlines	Working — the best single source
Two RSS feeds of a financial portal (commodities, indicators)	Headlines	Working
Two RSS feeds of a business broadcaster (economy, finance)	Headlines	Working
Several gold-specific feeds	Headlines	Dead (404 or permanent redirect) — removed
A large newswire's markets feed	Headlines	Reachable but about 18 months stale — removed, and a staleness guard added

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

> **A zombie feed is worse than a dead one.** A dead feed fails loudly; a stale feed returns valid-looking headlines from a year and a half ago. The aggregator therefore rejects sources whose newest item is too old.

Aggregation with asymmetric health

Headlines from all sources are merged by a composite feed with an asymmetric health rule: **any one healthy source makes the feed usable**; only when every source fails is the feed marked as failed. Zero headlines from a healthy source means a quiet news period, not a failure. Without this rule, the rate-limited news API alone would have produced a near-permanent blackout. The rule proved itself on its first day in production: the news API returned "too many requests" while the RSS sources delivered 16 headlines, and the cycle proceeded normally.

Hardening the input path

- **Network safety.** Redirects are followed only within the same origin and only over HTTPS; every response body is capped; record parsing is bounded.
- **XML safety.** Feeds are parsed with the standard library, but any document containing a DOCTYPE or entity declaration is refused before parsing, which removes the external-entity and "billion laughs" classes of attack.
- **Prompt-injection resistance.** Headlines are treated as untrusted data: both system prompts say so, each headline is collapsed to a single line so that it cannot forge list entries, and every model output is validated against a strict schema.
- **Secret safety.** A language-model endpoint must use HTTPS (loopback excepted), so an API key can never be sent in clear text.

10.3 The agents that read the news

Table 20. Language-model components

Agent	Input	Output
News sentiment	A batch of headlines	A score for each headline in [−1, +1] for its likely effect on gold, in one batched call
Macro analyst	Calendar events and scored headlines	Blackout, risk level, directional bias, confidence, uncertainty, cited input identifiers
Risk challenger	The macro read plus tools	A challenge that may only tighten the read

The monotonic challenger

A second model reviews the macro analyst's read and may challenge it. Debate between models is useful but dangerous near an order path: a broken, hallucinating or prompt-injected challenger must not be able to make the system *more* aggressive. The challenge is therefore applied through a function that is **monotonic towards caution**:

$$\text{blackout}' = \text{blackout} \vee \text{challenge.blackout}$$

$$\text{risk}' = \max(\text{risk}, \text{challenge.risk}) \quad \text{with} \;\; \text{LOW} < \text{MEDIUM} < \text{HIGH}$$

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

$$\text{confidence}' = \min(\text{confidence}, \text{challenge.confidence})$$

There is no field in the challenge that can express the reverse. An exhaustive, parametrised test covers every combination of state and verdict. This single property is what made it acceptable to put a debating pair of models upstream of real orders.

In production the challenger behaved as intended. On its first deployment it took four turns and three tool calls, found an upcoming US inflation print and a central-bank meeting through the calendar tool, and tightened the risk level from low to medium.

Tools, blackboard and memory

The agents can use a bounded tool loop. Their tools are read-only and operate only on data already fetched for the current cycle — they make no network calls and never raise:

- `get_calendar_events` — query the cycle's calendar;
- `search_headlines` — search the cycle's headlines;
- `get_recent_macro_reads` — look back at the agents' own recent reads, a short-term memory across cycles.

A bounded, append-only **blackboard** gives the agents a shared workspace with a known cost ceiling, rather than free-form chat; control characters are scrubbed so that one agent cannot forge another's entry. Every loop is limited by a maximum number of turns, tool calls and seconds.

Cost governance and failover

Free inference tiers are rate-limited, and a failed macro read is a blackout — so more deliberation can mean *less* trading. Three mechanisms keep reasoning from stopping the system:

1. **A governor** tracks deliberation cost and failures, with a circuit breaker that falls back to the simple single-shot read when the tool loop misbehaves. Every failure path of the richer loop degrades to the earlier behaviour, never to a new blackout.
2. **Quota-aware failover** across several OpenAI-compatible providers, respecting each provider's rolling quota.
3. **Caching.** The live macro provider computes at most one read per refresh interval and is refreshed *before* each poll, outside the lock that protects the broker connection, so the trading loop only ever reads the cache.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

10.4 The macro read in the interface

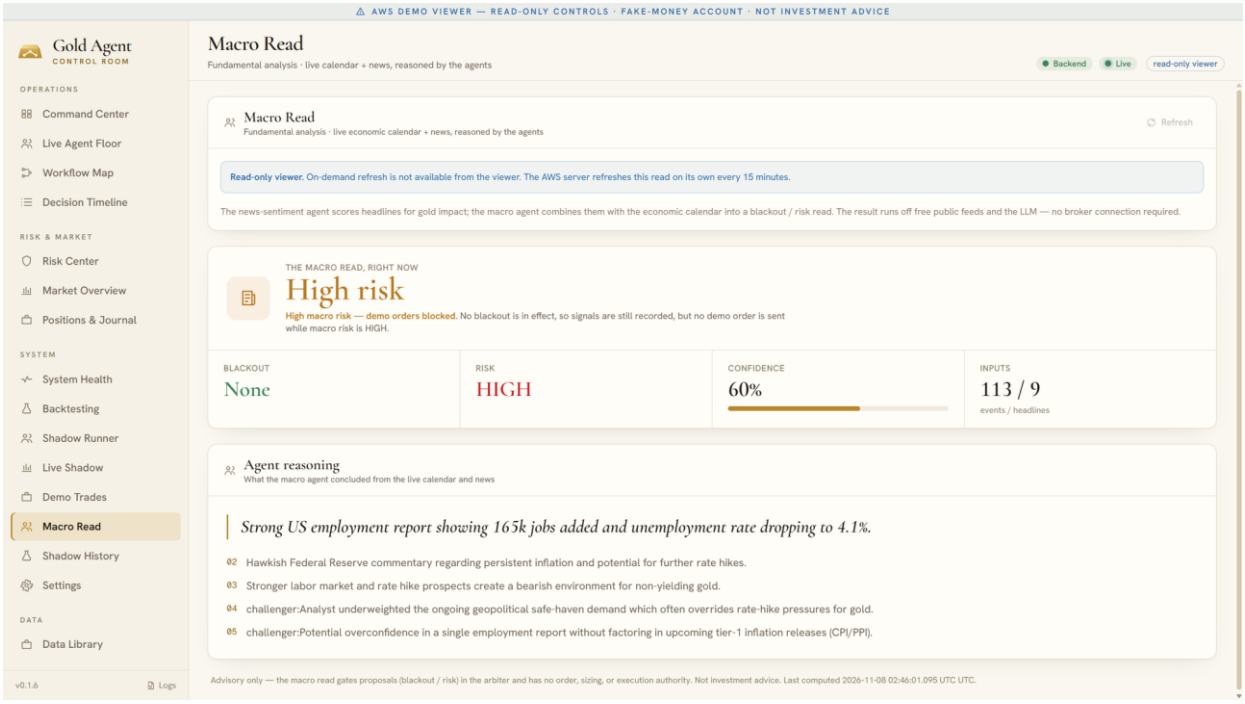


Figure 8. Macro Read in the cloud viewer: verdict, blackout, risk, confidence, inputs, and the agent's reasoning.

10.5 Testing whether the news helps

Two experiments asked whether the news layer adds information, not just safety.

The bias tracker. Since October every macro read's directional bias is recorded with its anchor price and later scored against the actual move over 1, 4 and 24 hours (reads taken while the market is closed are unscorable). An LLM's directional call cannot be honestly backtested — the model may have seen the period in training — so this evidence can only be collected forward.

The post-release drift study. If the market under-reacts to surprises in major US releases (CPI, NFP, PPI), a system acting after the first 15 minutes might profit from continuation. The study measured the direction of the first 15 minutes after each release and the move over the following hour and four hours, net of a $0.60 cost, against a control of the same time on days without a release.

Table 21. Post-release drift results (continuation strategy, four-hour horizon)

Sample	Releases	Result	t-statistic
M15, 2024–2026	74	−9.88 (price units, net)	−1.93
H1, 2018–2026	299	−1.67	−1.79

Continuation lost in both samples; a hint of a fade effect in 2024–2026 disappeared over eight years. The surprise is priced before a 15-minute system can act, so a live actual-versus-forecast recorder was not built. The calendar remains a risk gate, not a signal.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

11 Decision and risk

This chapter follows a would-be trade from the analysts' evidence to an approved order: the Arbiter's decision, the position-sizing formula and the Guardian's twelve rules.

11.1 The Arbiter

The Arbiter combines the strategies' signals and the macro read into one decision. It is pure and deterministic and fails closed at every step.

1. **Macro gate.** If the macro read reports a blackout — or if there is no macro read and missing macro is treated as a blackout — the answer is NO TRADE (`macro_blackout` or `macro_unavailable`).
2. **Signals present.** No directional signal → NO TRADE (`no_strategy_signals`).
3. **Agreement.** Signals in both directions → NO TRADE (`strategy_disagreement`). Fewer agreeing signals than the configured minimum → NO TRADE (`insufficient_agreement`).
4. **Primary signal and confidence.** Among the agreeing signals, the one with the highest confidence provides the entry, stop and target (ties broken by strategy name, for determinism). Its confidence is reduced by the macro risk level:

$$\text{confidence} = \max_\{s \in \text{agreeing}\}\ \text{conf}(s) \times \text{m}(\text{risk})$$
$$\text{m}(\text{LOW}) = 1.00,\ \ \text{m}(\text{MEDIUM}) = 0.85,\ \ \text{m}(\text{HIGH}) = 0.60$$

5. **Threshold.** Below the minimum confidence → NO TRADE. Otherwise the decision is `PROPOSE_TRADE` with direction, levels, combined confidence, the contributing strategies and reason codes.

The decision carries **no volume and no risk percentage**. Those are decided in exactly one place.

11.2 Position sizing

Sizing turns a decision into a volume. It is the only place in the system where a size is computed, and its formula is deliberately simple and conservative.

$$\text{allowed_loss} = \text{equity} \times \text{risk\%} \,/\, 100$$

$$\text{loss_per_lot} = |\ \text{entry} - \text{stop}\ | \times \text{contract_size}$$

$$\text{volume} = \text{clamp}(\ \text{floor_to_step}(\ \text{allowed_loss} \,/\, \text{loss_per_lot}\ )\ \text{within}\ [\ \text{min_volume}\ ,\ \text{max_volume}\ ]$$

If the floored volume is below the broker's minimum, the trade is **not taken** (`below_min_volume`) rather than rounded up. Because the volume is floored, the realised risk is always at or below the target, and it is recomputed from the final volume. Margin is estimated as:

$$\text{margin} \approx \text{entry} \times \text{contract_size} \times \text{volume} \,/\, \text{leverage}$$

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Table 22. Worked example (illustrative values)

Quantity	Value
Account equity	10,000
Target risk per trade	0.25 % → allowed loss 25.00
Entry / stop	4,000.00 / 3,994.00 → distance 6.00
Contract size (1 lot = 100 oz)	loss per lot = 6.00 × 100 = 600.00
Raw volume	25.00 / 600.00 = 0.0417 lots
Volume after floor to 0.01 step	**0.04 lots**
Realised risk	0.04 × 600 = 24.00 → **0.24 %** of equity
Margin at 1:100 leverage	4,000 × 100 × 0.04 / 100 = **160.00**

The sized proposal then goes to the Guardian, which applies every rule independently of how the size was derived.

11.3 The Guardian's twelve rules

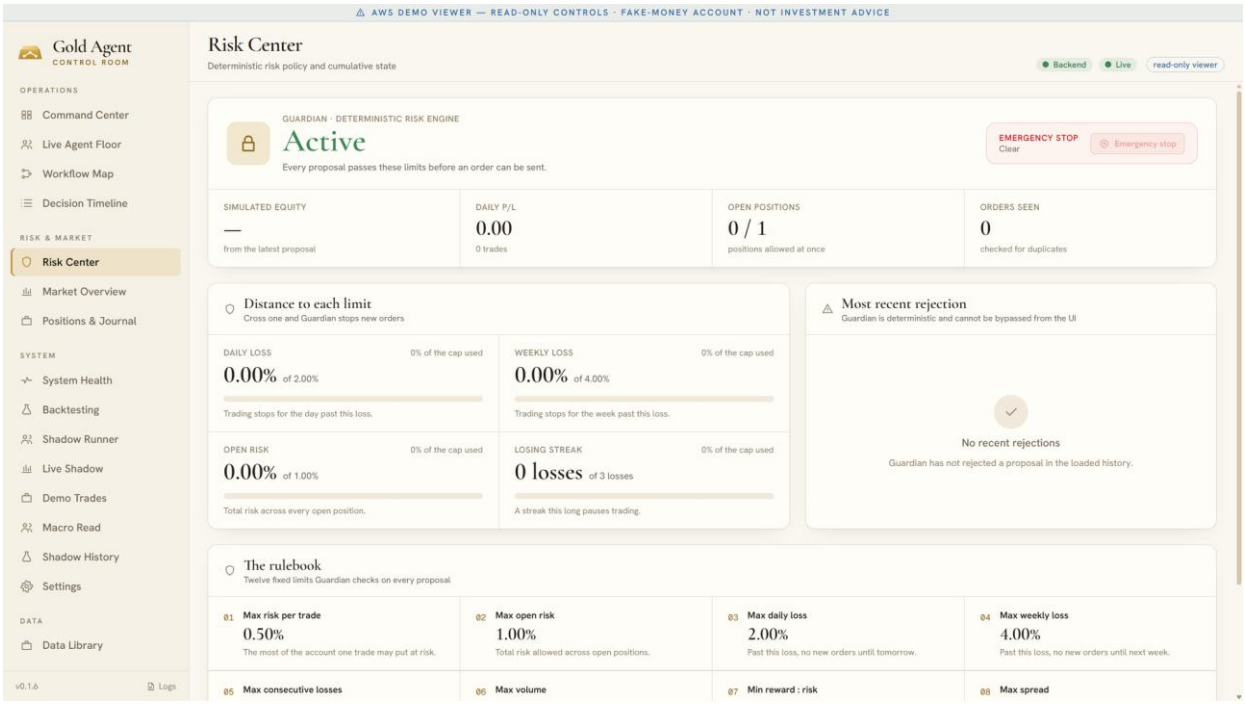


Figure 9. Risk Center in the cloud viewer: the Guardian's status, the distance to each limit, the most recent rejection and the rulebook.

Table 23. The Guardian's rulebook (values used on the cloud demo)

#	Rule	Limit	What it prevents
1	Max risk per trade	0.50 % of the account	One trade doing outsized damage
2	Max open risk	1.00 % across open positions	Stacked exposure
3	Max daily loss	2.00 % — no new orders until the next day	A bad day becoming a disaster
4	Max weekly loss	4.00 % — no new orders until the next week	A bad week compounding

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

#	Rule	Limit	What it prevents
5	Max consecutive losses	3 — trading pauses	Tilt after a losing streak
6	Max volume	0.1 lots per order	Size errors
7	Min reward : risk	1.5	Trades that need an unrealistic win rate
8	Max spread	45 points	Entering in illiquid moments
9	Max slippage	10 points	Fills far from the intended price
10	Max data age	90 seconds	Trading on stale prices
11	Max margin fraction	20 % of free margin	Margin stress
12	Position limit	1 open position	Over-trading

Rule 7 connects directly to the mathematics of Chapter 13: with a reward-to-risk of b, a strategy needs a win rate above $1 / (1 + b)$ just to break even; at 1.5 that is 40 %.

Persistent risk state

The Guardian keeps a durable state: the realised result of the day and the week, the current losing streak, open risk, recently seen order identifiers (for duplicate protection) and the emergency-stop flag. After a restart it rebuilds that state from the audit store and from reconciliation with the broker before it approves anything. An uncertain reconciliation blocks new orders.

11.4 The emergency stop

The emergency stop is the last line of defence. One control in the interface (with a confirmation dialog) halts the runner and blocks every new cycle and order.

- It **persists** across a simulation reset, a restart of the service, a relaunch of the desktop and an upgrade of the application. A reset never clears it.
- It can be lifted only through a **guarded reactivation**: a backend-owned transition that checks the configuration, the risk state, reconciliation, readiness and concurrency, requires an exact typed confirmation phrase and a reason, and atomically appends a durable safety record. Reactivation leaves the runner stopped; resuming is a separate, deliberate action.
- The interface makes it hard to trigger by accident and impossible to hide: the confirmation dialog takes the whole window, focuses the safe action, closes on Escape, and the Command Center shows the state as `ENGAGED` in red while it is active.

> **Tested end to end.** The packaged-installer test engages the stop, kills and restarts the service, verifies that the stop survived, checks that the event history continued correctly, performs the guarded reactivation and verifies the durable audit record — on the real installed application.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

12 Execution and reconciliation

Sending an order is the highest-risk thing the system does, so the order path was built last, behind more gates than any other part, and only for a fake-money demo account.

12.1 Read-only market access first

Before any order path existed, a read-only MetaTrader 5 adapter was built over an injected client interface. It exposes observations only — the symbol's specification, bars, the latest quote, the account and open positions — and has no method that can place an order. It is disabled by default, imports the broker's package lazily, normalises every time to UTC using the server's configured offset, reads credentials from the environment (never storing or logging the password) and fails closed: any adapter error becomes NO TRADE.

12.2 The MQL5 safety bridge

A demo-only safety Expert Advisor, written in MQL5, defines how the platform may talk to the terminal. Its Python counterpart validates every command against a strict protocol before it can be acted on.

Table 24. Checks applied to every bridge command

Check	Rejects
Account type	Anything that is not a demo account, including unknown types
Credential	A token whose SHA-256 hash does not match (the raw token is never transmitted)
Allow-list	Command types not explicitly permitted
Idempotency	A command identifier already seen (replay protection)
Time	Expired or future-dated commands
Kill switch	Any risk-adding command while the kill switch is on (closing is still allowed)
Symbol	Any symbol other than the configured gold symbol
Opening	Opening new positions while demo opening is disabled (the default)
Hard limits	Volume, stop distance and number of positions above fixed ceilings

A heartbeat lets each side detect that the other is alive. Order opening through the bridge is off by default.

12.3 The demo executor

The demo executor is the first and only component that can place a real order on a broker, and it does so only on a demo account. It runs through a chain of gates; every one must pass.

1. **Configuration.** Demo orders must be explicitly enabled. Live orders are permanently rejected at configuration load.
2. **Account.** The connected account must be verified as a demo account; live or unknown accounts are rejected.
3. **Freshness.** The proposal's bar must be at most 20 minutes old. (This gate was added after an outage, see Chapter 15: the risk engine's tick-age check is always fresh and cannot catch an old *signal*.)

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

4. **Reconciliation.** The latest reconciliation must be clean. A system that has never reconciled is treated as unsafe.
5. **Guardian.** The sized proposal must be approved by the risk engine.
6. **Emergency stop and kill switch.** Either one blocks.
7. **Idempotency.** The order carries a deterministic client identifier derived from the bar, so a retry cannot duplicate it.

After submission, the outcome is classified: a full fill at an acceptable price is `FILLED`; a partial fill or a fill beyond the slippage threshold is `RECONCILE_MISMATCH`; a broker error or exception is `FAILED`. Failures are logged with the broker's error, raise an alert on the operator's phone (at most one per hour) and appear in the daily summary.

Three defects that only production could reveal

Table 25. Order-path defects found and fixed in September

Defect	Effect	Fix
The symbol's filling-mode bitmask was passed as the order's filling policy	Every order rejected with "unsupported filling mode"	Map the symbol's capabilities to a policy in order of preference: fill-or-kill, then immediate-or-cancel, then return
The terminal's algorithmic-trading switch was off on the server	Orders refused by the terminal	The start-up script enables it in the terminal's configuration before every launch
The order comment exceeded the broker API's 29-character limit	Every order rejected as an invalid argument	A short comment (`gold-agent` plus the bar's epoch, 21 characters); the client caps comments at 29

Until these fixes the cloud demo had attempted 126 orders without a single fill. After them, the first real demo trade was placed on 24 September at 17:00 UTC: a sell of 0.01 lots, filled and protected by its stop and target on the broker's server.

12.4 Reconciliation

The system keeps its own ledger of what it believes is open; the broker keeps the truth. Reconciliation compares them by ticket on every cycle.

Table 26. Reconciliation findings

Finding	Meaning
`MISSING_ON_BROKER`	The ledger has a position the broker does not
`UNEXPECTED_ON_BROKER`	The broker has a position the ledger does not
`DIVERGENT_VOLUME`	The same ticket with a different volume
`DIVERGENT_DIRECTION`	The same ticket with a different direction
`DUPLICATE_TICKET`	One ticket recorded more than once

$$\text{status} = \text{MISMATCH} \;\text{ if }\; \text{findings} \neq \emptyset$$
$$\text{execution_allowed} = (\,\text{status} = \text{RECONCILED}\,)$$

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Any finding blocks execution; the block heals itself when a later reconciliation is clean. The first real trade exposed one more subtlety: its fill was flagged `RECONCILE_MISMATCH` because the distance from the *signal* price exceeded the slippage threshold, and the ledger only counted `FILLED` positions as open — so the broker's position looked unexpected, and the monitor restarted the service in a loop. The fix treats both `FILLED` and `RECONCILE_MISMATCH` as open positions, and the monitor now alerts on a reconciliation problem instead of restarting.

12.5 The position monitor and trade records

A position monitor runs on the demo host. Every cycle it reads the broker's open positions, feeds reconciliation, and, when a position disappears from the broker, reads the deal history for that ticket to record the exit price, the net result after costs and the close reason (stop, target or program). These close records are reporting-only: a failure to read them is retried and never affects reconciliation or trading.

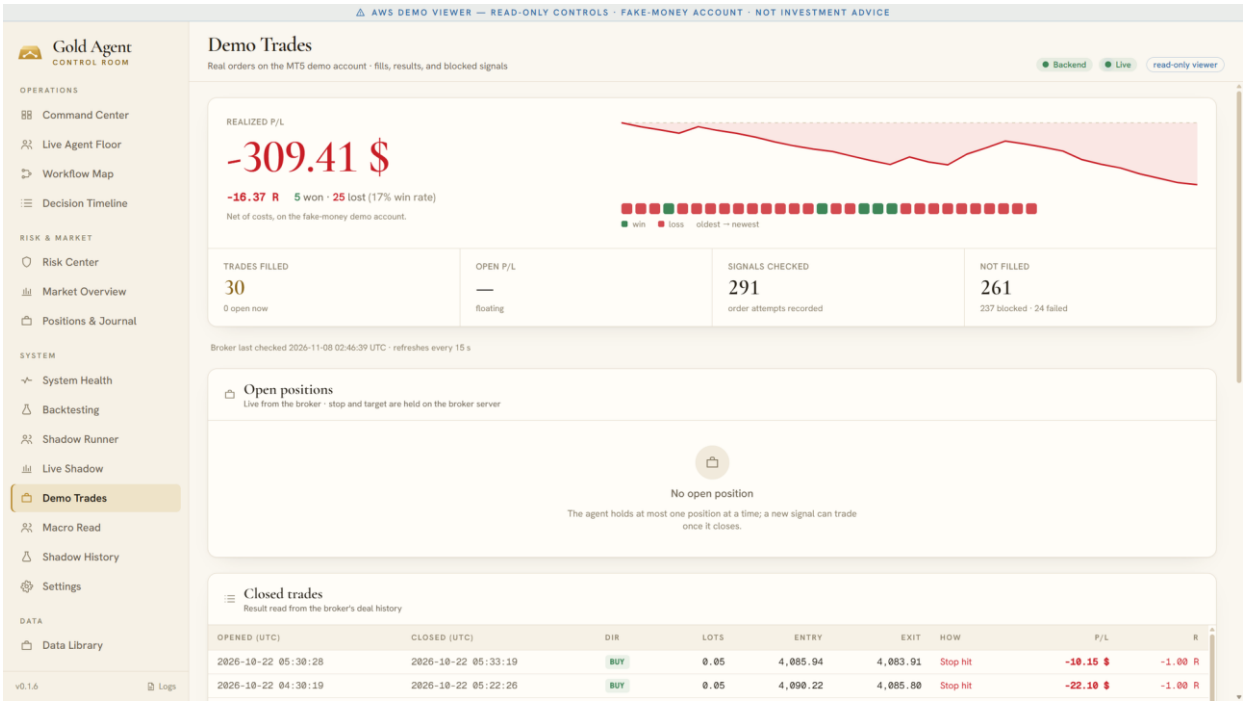


Figure 10. Demo Trades in the cloud viewer: realised result, the win/loss strip, fills, signals checked and signals not filled.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

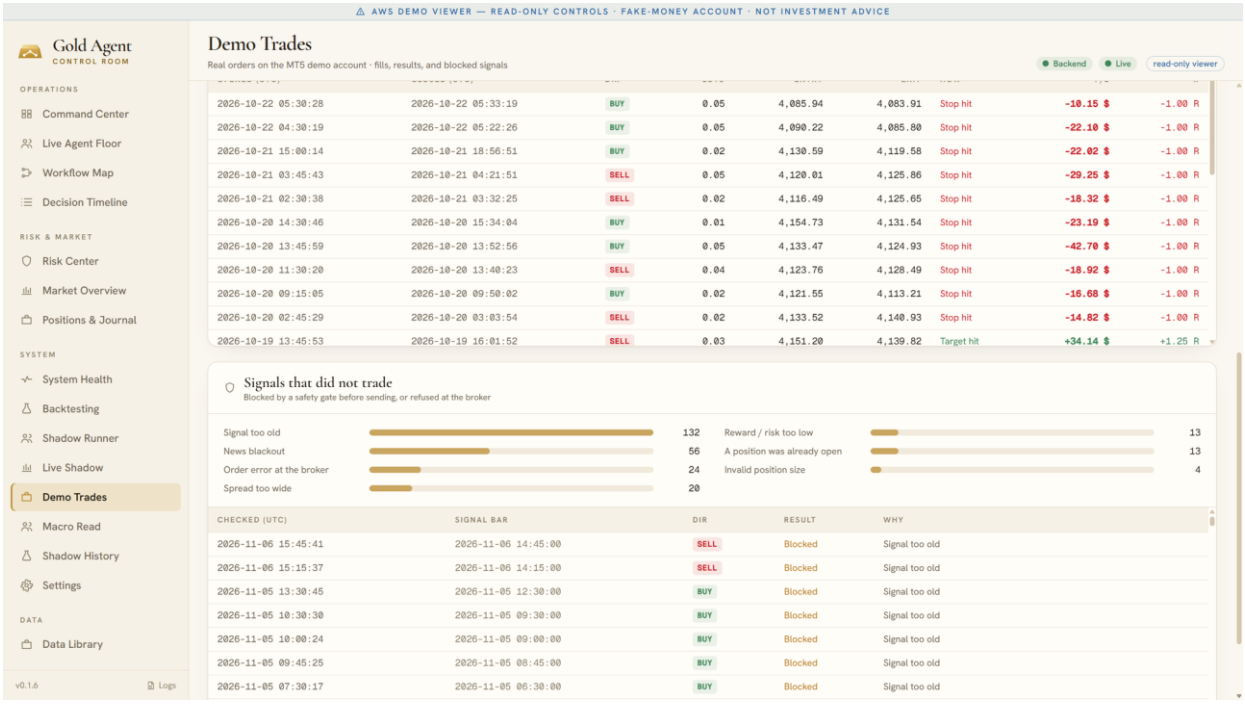


Figure 11. The closed trades table and the breakdown of signals that did not trade, by reason.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

13 Validation and research methodology

A trading strategy is easy to make look good and hard to prove good. This chapter describes the tools the project built to tell the difference, in the order a strategy meets them: backtest, walk-forward, statistical judgement, stress test, shadow mode and forward tracking.

13.1 Measuring in R

Results are measured in **R**, multiples of the risk taken on each trade, rather than in currency. A trade's result is its price gain or loss divided by its initial risk:

$$R_i = (\text{exit}_i - \text{entry}_i) \times d_i \;/\; |\text{entry}_i - \text{stop}_i|$$
$$d_i = +1 \text{ for a buy}, -1 \text{ for a sell}$$

R is independent of account size and position size, so it measures the rule set itself. Costs are included in the entry and exit prices, so every R is net.

13.2 Metrics

For N closed trades with results $R_1 \dots R_N$, of which W are wins:

$$\text{win rate } p = W / N \qquad \text{expectancy } E = (1/N) \cdot \Sigma R_i$$

$$\text{profit factor } PF = \Sigma_{R_i > 0} R_i \;/\; |\Sigma_{R_i < 0} R_i|$$

$$\text{equity } Q_k = \Sigma_{i \le k} R_i$$
$$\text{max drawdown } DD = \max_k (\max_{j \le k} Q_j - Q_k)$$

Also reported: average win and average loss in R, maximum consecutive losses and average bars held. A strategy that wins a fraction p of trades with average win b R and average loss 1 R has

$$E = p \cdot b - (1 - p) \qquad \text{and breaks even when} \quad p^* = 1 / (1 + b)$$

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

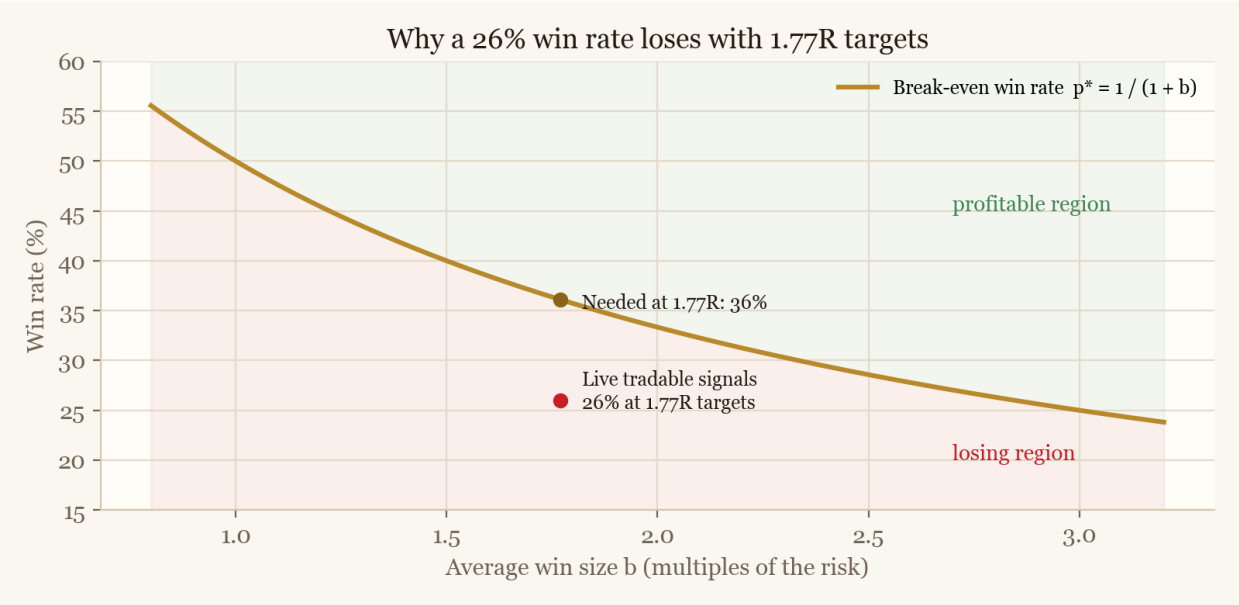


Figure 12. The break-even win rate as a function of the average win. The live tradable signals sat well inside the losing region.

13.3 The event-driven backtester

The backtester replays history bar by bar through the same feature engine and the same strategy code used live.

1. On each bar the strategy is evaluated on that bar's causal features.
2. A signal is filled at the **next bar's open** — never on the signal bar — so no information from the fill bar is used to decide the trade.
3. At most one position is open at a time.
4. While a position is open, each bar's high and low are checked for the stop and the target. When one bar's range spans **both**, the stop is assumed to have been hit first (pessimistic intrabar ordering).
5. A position still open at the end of the data is closed at the last close.
6. An entry whose fill lands on the wrong side of its stop or target (for example after a gap) is cancelled, not forced.

Costs are applied adversely to every fill: a slippage amount per fill, half the spread per side (the bar's own spread when available), and a round-trip commission. The default model uses a spread of 0.20 and slippage of 0.02 per side. Every run is reproducible from the dataset's content hash, the configurations and the code version, and produces an ordered audit trail of events.

13.4 Walk-forward validation

A backtest on the same data used to choose parameters is optimistic by construction. Walk-forward validation separates the two.

1. Split the history into consecutive folds, each with an **in-sample** window followed by an **out-of-sample** window.
2. In each fold, choose the best configuration from the candidate grid using the in-sample window **only**, ranked by in-sample expectancy; candidates with too few in-sample trades are distrusted and rank last; ties keep the earlier candidate.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

3. Evaluate the chosen configuration on the out-of-sample window, warmed up with the end of the in-sample window (strictly past data); any trade that would have entered before the out-of-sample window opens is discarded.
4. Advance by one out-of-sample window (rolling or anchored) and repeat; pool the out-of-sample trades.

Leakage is prevented by construction and verified by tests: the windows never overlap, selection sees only in-sample bars, warm-up trades are excluded, and the series is never shuffled. The output includes the dispersion of out-of-sample expectancy across folds: its mean, standard deviation, standard error and the count of positive folds.

13.5 Pre-registered experiments and statistical judgement

The experiment framework turns a research question into a record that cannot be quietly edited after the fact.

1. **Pre-register.** The hypothesis, dataset and its status, exact candidate grid, walk-forward and cost settings, code commit and — critically — the **decision rule** are fixed before the search runs. The experiment's identifier is a hash of all of it, so a changed grid or rule is a new experiment.
2. **Run** the search through the walk-forward engine.
3. **Judge** mechanically against the pre-registered rule, producing one of three verdicts.

Table 27. Experiment verdicts

Verdict	Meaning
`INSUFFICIENT_EVIDENCE`	Too few out-of-sample trades or folds to say anything. Never read as "no edge".
`NO_EDGE_DEMONSTRATED`	Ran cleanly but failed the bar; each failing condition is named.
`SURVIVES_OUT_OF_SAMPLE`	Cleared every condition. Still not a profitability claim; still requires manual review.

Table 28. Default decision rule

Condition	Default	Purpose
Minimum out-of-sample trades	100	Below this, insufficient evidence
Minimum folds	4	One lucky window must not carry a verdict
Minimum share of positive folds	0.5	Consistency across time
Minimum expectancy	0.0R	An economic floor before statistics
Family-wise α	0.05	Error rate across the whole search
Bonferroni correction	On	Widen the bar for every candidate tried

Why the size of the search matters

Testing 108 configurations and reporting the best one is a multiple comparison: under the hypothesis of no edge, the best of 108 noisy results is positive almost by construction. The bar is therefore widened with the number of candidates *k*:

$$\alpha_test = \alpha \,/\, k$$
$$|z| \geq \Phi^{-1}(\,1 - \alpha \,/\, (2k)\,)$$
$$z = \mathrm{mean}(E_fold) \,/\, (\,\mathrm{sd}(E_fold) \,/\, \sqrt{F}\,)$$

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

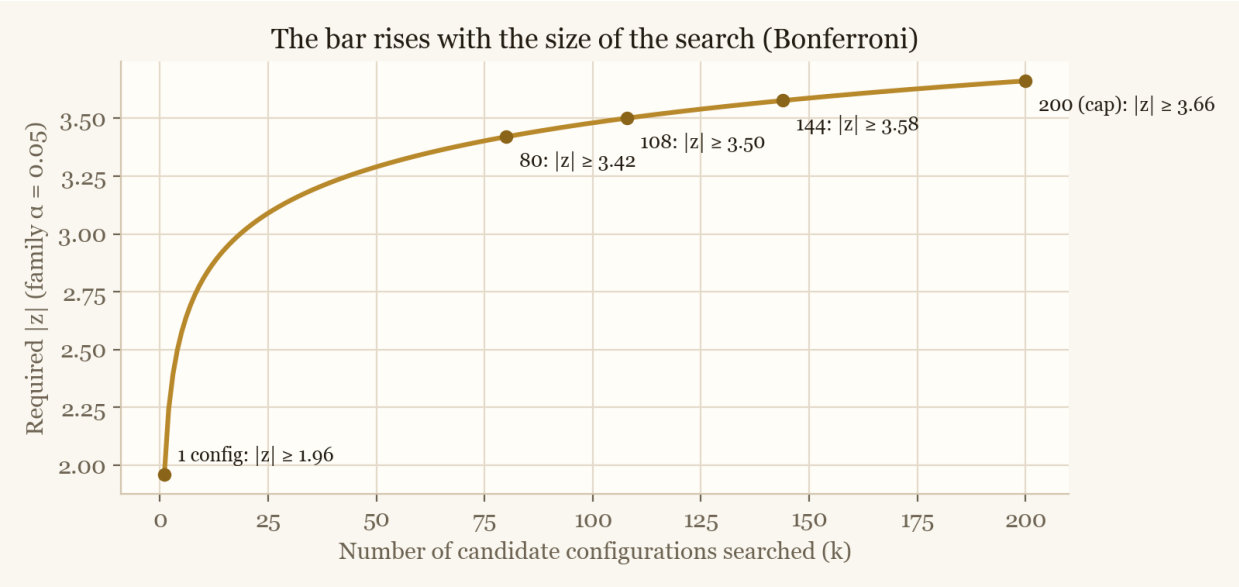


Figure 13. The required |z| rises with the size of the search: 1.96 for one pre-specified configuration, about 3.50 for 108 and 3.66 at the cap of 200.

A larger grid buys a better in-sample fit and a harder bar at the same time; the framework forces that trade and caps grids at 200 candidates. Every record also states its own weakness: folds share one price history, so they are not independent draws, and the test is a weak upper bound on significance rather than a proper hypothesis test.

Baselines

A strategy that makes money in a rising market may simply be long a rising asset. Results were therefore compared with Monte-Carlo baselines that trade at random times using the strategy's own stop and target geometry: random direction, the strategy's own buy/sell mix, and always-buy. Beating random direction is necessary; beating always-buy is the real test in a bull market.

13.6 Stress testing

Table 29. Stress scenarios applied to every candidate

Scenario	Perturbation
Spread ×1.5 and ×2	Every bar's modelled spread widened
Extra slippage	+0.2 adverse slippage per fill
Injected gaps	Every 20th bar removed
Stale feed	Price frozen for 3 bars in every 40 while timestamps advance
Data outage	A contiguous 10 % block removed mid-series
Remove best 1 / best 3	The top trades by R removed from the baseline

A result that survives only with its best trade, or collapses with a slightly wider spread, is not robust.

13.7 Shadow mode

Shadow mode runs the full decision chain on real or historical bars and records every would-be trade with its context — the reference price, the intended-versus-reference difference, the spread and the latency — while

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

sending nothing. The guarantee is structural: the session's mode is `SHADOW`, its `orders_sent` field is a constant zero, and each decision carries `order_sent = false`.

The **shadow runner** in the desktop runs the whole team over an imported dataset and shows the decision funnel: bars evaluated, bars gated to NO TRADE, and would-be trades, with optional advisory sizing for a hypothetical equity.

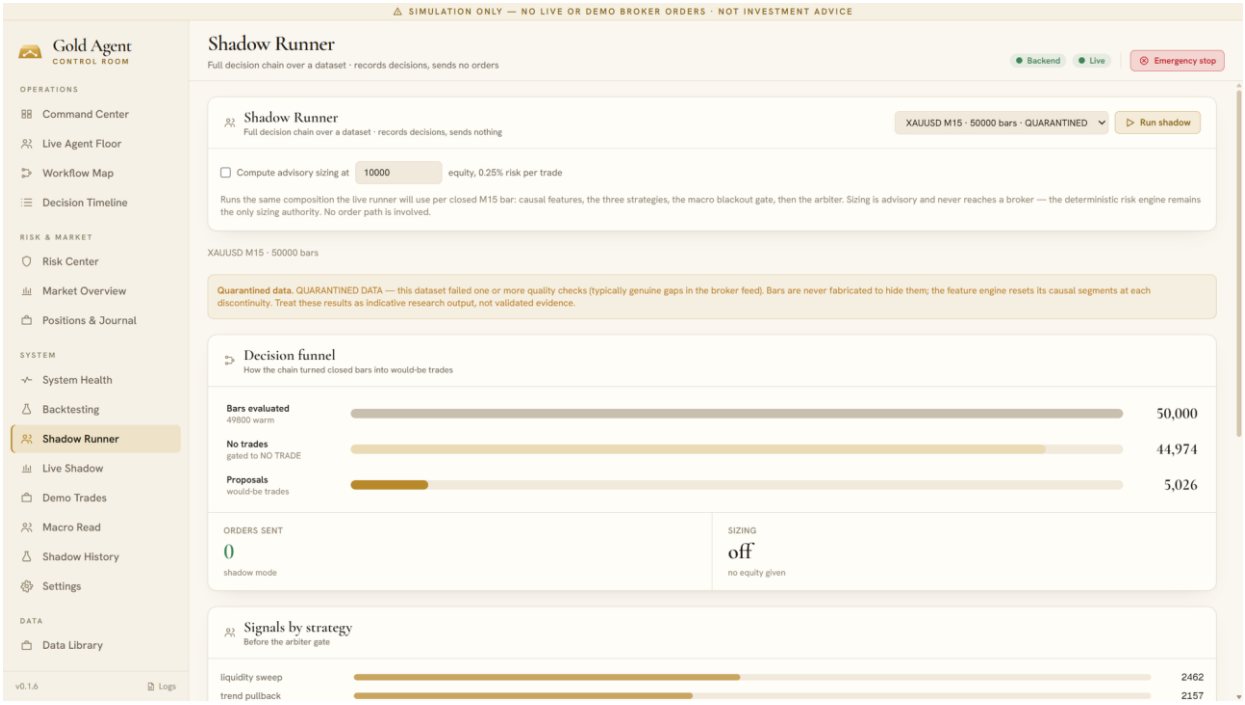


Figure 14. Shadow Runner on the M15 dataset: 50,000 bars, 44,974 gated to NO TRADE, 5,026 would-be trades, 0 orders.

13.8 Forward shadow history

The live loop records every proposal it makes to the database and later scores it on the bars that close after it, as if it had been traded — without sending an order. This is the honest way to measure a forward hypothesis such as "the macro read improves results", which cannot be backtested.

Table 30. Rules for scoring a recorded proposal

Rule	Purpose
Follow the proposal only from the moment it was recorded	No hindsight
A stop and a target touched in the same bar count as a stop	Pessimism, as in the backtester
Expire after 96 M15 bars and mark to the close	Bounded holding period
Exclude proposals recorded more than 20 minutes after their bar	Backfilled records cannot inflate statistics

The last rule was added after a diagnosis showed that proposals backfilled after outages had been scored against prices that occurred *after* they were recorded, creating tens of R of fictional profit (Chapter 17).

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

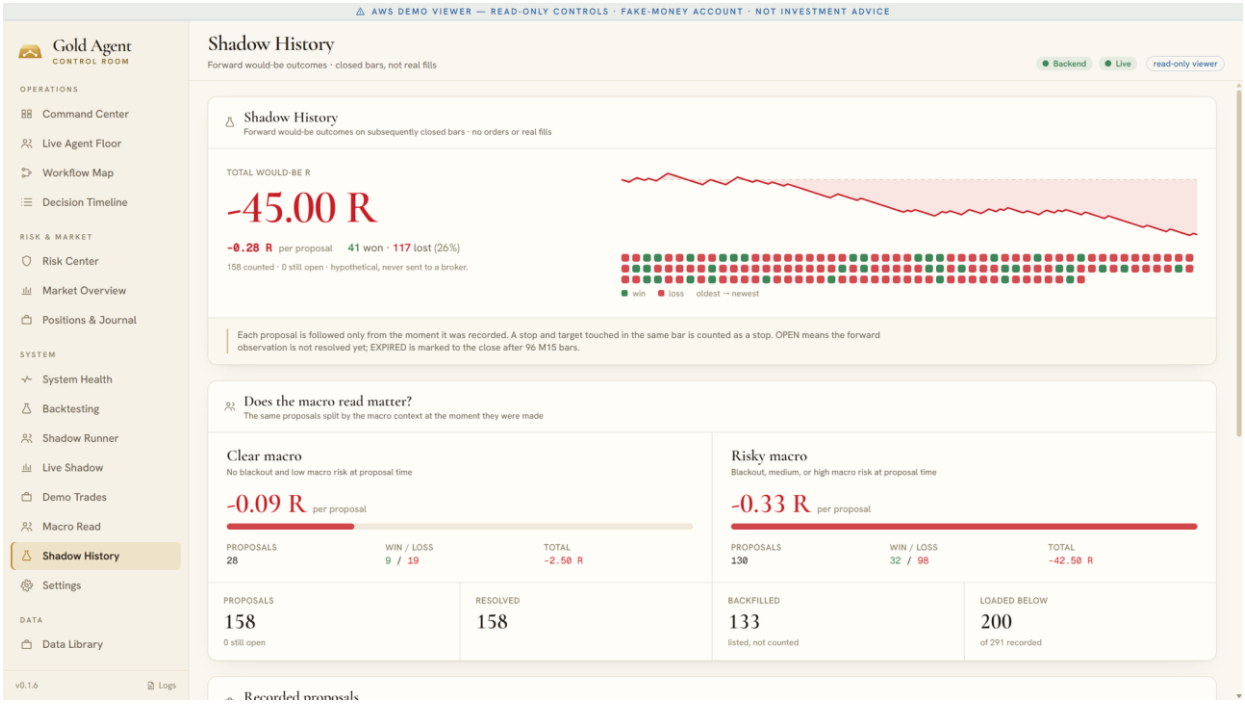


Figure 15. Shadow History in the cloud viewer: total would-be result, per-proposal mean, win/loss strip and the macro split.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

14 Live operations on the cloud

A strategy can only be judged forward, on bars that did not exist when it was designed. From late July the full team ran unattended on a cloud server, trading a fake-money demo account, 24 hours a day, five days a week. This chapter describes how that system was operated; Chapter 15 describes what went wrong.

14.1 The host

Table 31. Cloud host

Item	Value
Provider and region	AWS, Europe (Stockholm)
Operating system	Windows Server 2025
Size	1 GB of memory until 12 October; 2 GB afterwards
Disk	30 GB, of which only 1–3 GB remained free in October–November
Exposure	The agent service binds to loopback only; no inbound port is open for it
Broker	A MetaTrader 5 terminal logged into a fake-money demo account (hedging, 1:100)

The installation lives in one directory tree: the application, an embedded Python, the operations scripts, logs, data and downloads. The operations folder is writable by administrators only; the runtime account can only read and execute it.

14.2 Scheduled tasks

Table 32. Operations tasks on the host

Task	Runs as	Schedule	Job
Backend	A dedicated non-administrator service account	At start-up, kept alive	Applies any staged MetaTrader update without elevation, enables algorithmic trading, launches the terminal, starts the agent service and the live loop
Health monitor	SYSTEM	Every 5 minutes	Probes health, the live loop, reconciliation and the position monitor; restarts what failed; detects and repairs MetaTrader update loops; sends alerts
Maintenance	SYSTEM	Daily at 02:30	Backups of the database and data, retention, log rotation

Running the terminal and the agent service under the same non-administrator account was a deliberate fix: the broker's Python package can only attach to a terminal owned by the same user.

14.3 Observability

- **Health endpoint.** Readiness of the service, the live loop, the broker connection, reconciliation and the position monitor, with the age of each last check.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

- **Structured, UTC-stamped logs** for the service and for start-up, including dedicated lines for MetaTrader updates and algorithmic-trading state, and a `DEGRADED` line that names the failing condition.
- **Telegram.** A daily report of activity and results, an alert when the monitor cannot recover the service (and a "healthy again" message when it does), and an alert, at most once an hour, whenever an order fails at the broker. Telegram delivery is isolated from the process's proxy settings so that it cannot interfere with the broker or the language-model calls.
- **The read-only desktop viewer** described below.

14.4 Reaching the server: remote commands and a private tunnel

The server is administered without remote desktop or open ports, through the cloud provider's systems-manager agent:

- **Remote commands** run scripts on the host and return their output. Because the command channel caps output size, large results are compressed and returned in chunks, and host output is scrubbed to plain ASCII (the language model's typographic characters otherwise break the local command-line client's decoder).
- **Deployments** compress each changed file, write it on the host with a backup of the previous version, run an import check of the whole package *before* restarting, purge cached bytecode and restart the backend task — with the health monitor disabled for the duration of the restart and re-enabled afterwards.
- **Port forwarding** opens a private tunnel from the desktop to the agent service's loopback port.

14.5 The read-only viewer

The desktop includes a launcher for an **AWS demo viewer** mode. It checks the operator's cloud credentials, opens the private tunnel, starts the Control Room pointed at it and closes the tunnel when the application exits.

The viewer authenticates with a separate credential that the service accepts for read-only requests and the event stream; every other request is refused with `403 VIEWER_READ_ONLY`. In viewer mode the desktop accepts only a service that reports the demo trading mode, never starts, stops or restarts it, and hides or disables every action it cannot perform — research runs, imports, refreshes and the simulation controls.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

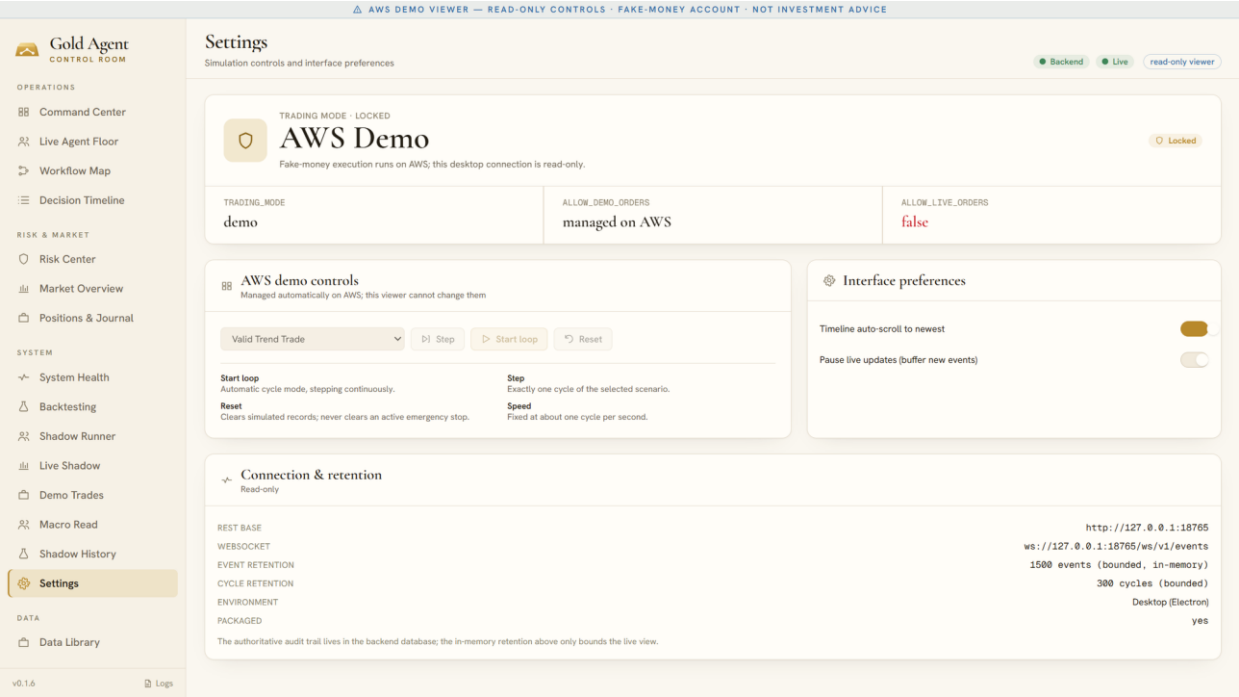


Figure 16. Settings in the cloud viewer: the locked trading mode with its three safety flags. Live orders: false.

14.6 Costs and limits

The project ran on the cloud provider's free-credit plan. The server's burn rate was measured from two readings of the remaining balance.

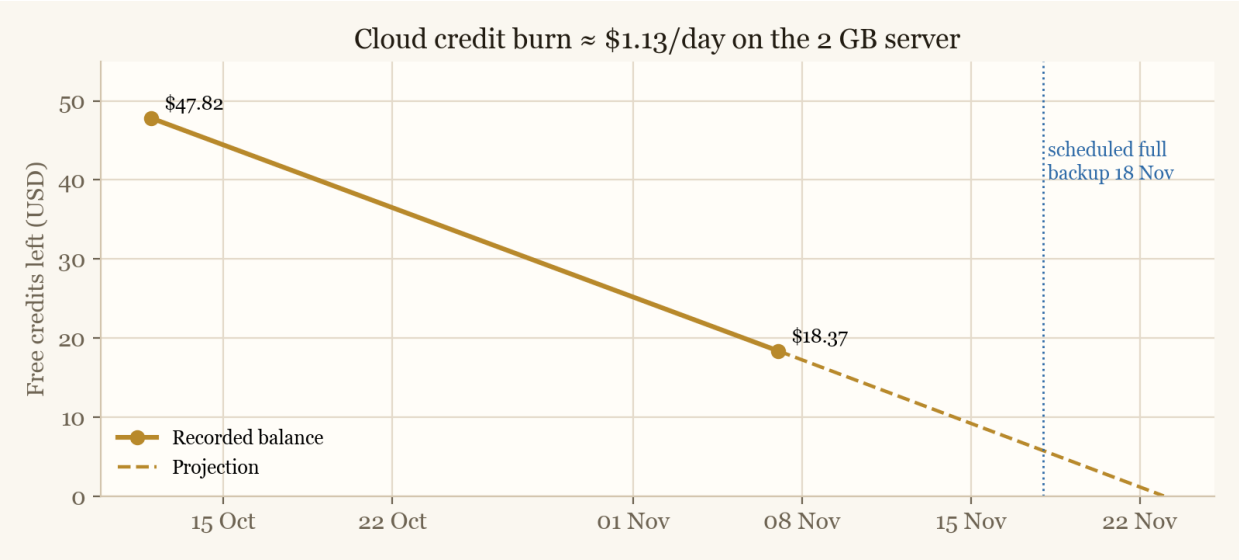


Figure 17. Remaining free credits and their projection. At about $1.13 per day the credits run out around 22–23 November, long before the plan's end date in January.

On the free plan, exhausted credits stop the resources. The owner chose not to upgrade, so a **backup plan** was built before the end:

1. On the host, a consistent snapshot of the database is taken with SQLite's online-backup API, and the historical data and logs are compressed into a staging folder.
2. A one-time file server bound to the host's loopback interface, protected by a random token and an automatic timeout, serves the archive exactly once.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

3. The desktop pulls it through a private tunnel, verifies its SHA-256 checksum and the archive's integrity, and the staged copy is deleted from the host.

The first full backup (130 MB) was taken on 7 November, and a scheduled task runs the same procedure on 18 November, before the credits run out.

14.7 Disk and memory budget

The small server forced continuous attention to resources. Daily backups grew by about 90 MB per day and, with 14 days of retention, left less than 1 GB free; retention was cut to 7 days. With 1 GB of memory, the operating system's antivirus alone held about 300 MB, the agent service and the terminal were paged out to a few megabytes of working set, and restarts took four to seven minutes. After the upgrade to 2 GB the agent service kept a working set of about 214 MB with roughly 500 MB free.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

15 Incidents and fixes

Running unattended exposed problems no test had predicted. Each incident below is reported with its symptom, its root cause and the fix. Together they explain most of what the project learned about operating an automated trading system.

Table 33. Incident log

Date	Incident	Root cause	Fix
29–30 Jul	Unattended start-up failures after reboot; alerts not delivered	Terminal and service under different accounts; proxy settings leaking into alert delivery; language-model quota exhaustion	One service account; isolated alert delivery; rolling-quota awareness and provider failover
30 Jul	Security review: three privilege-escalation paths and a bypassed safety gate	Writable folders executed by SYSTEM tasks; a hard-coded "reconciled" flag	Locked-down permissions; reconciliation consulted before every order
3 Aug – 19 Sep	Service down for weeks; repeated "could not recover" alerts	MetaTrader update helper launched with an elevation prompt nobody could answer	Apply updates directly, unprivileged, before launch and in the market break
19 Sep	First fix made it worse	The vendor installer installed an older public build, downgrading the terminal	Installer removed from automatic recovery
19 Sep	Latent risk of trading weeks-old signals at market open	The live watermark was 47 days old; nothing measured a signal's age	Reject proposals whose bar is older than 20 minutes
19 Sep	Demo account deleted	The broker deletes demo accounts idle for six weeks	New demo account; outage cost made visible
23 Sep	Health failures and a second terminal downgrade	The macro read ran inside the broker lock; loop detection counted the whole day	Macro refreshed outside the lock; detection window of 30 minutes
23 Sep	Most of the history invisible to the strategies	The feature engine reset at every normal market closure	Engine 1.1.0 bridges normal closures; all research re-run
24 Sep	126 order attempts, zero fills, for two months	Wrong filling policy, algorithmic trading off, comment too long — all swallowed	Three fixes, failure alerts, first real trade
24 Sep	Restart loop after the first fill	Flagged fill not counted as an open position	Both statuses count as open; alert instead of restart
1 Oct	Update loop with nothing to apply	An update downloaded without its helper	Move incomplete downloads aside automatically
6–12 Oct	Twice-daily outage alerts	1 GB of memory: constant paging; monitor waited 3 minutes for 4–7-minute restarts	Wait 8 minutes; upgrade to 2 GB; backup retention cut to 7 days
13 Oct	Shadow results looked strongly positive	Backfilled proposals scored on prices after they were recorded	Late-recorded proposals excluded from statistics
7 Nov	Every agent shown as failed or blocked at weekends	The stale-feed guard knew nothing about market hours	Market-hours model and a `MARKET_CLOSED` status

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

15.1 The MetaTrader update loop

The broker pushes new terminal builds frequently. The terminal downloads a build into a local folder together with a small helper program and, at a moment it chooses — sometimes a day later, mid-session — launches the helper with an elevation request. The terminal runs under a non-administrator service account, so the elevation prompt waits for an answer that never comes, times out, and the terminal relaunches on the old build — and tries again, forever. Each relaunch looked like a crash to the health monitor.

The key observation was that **elevation is not actually needed**: the helper is manifested to run with the caller's rights, and the service account already has write access to the terminal's installation folder. Running the helper directly, without the elevation verb, applies the update in one step. The fix is layered:

1. The start-up script applies any staged build, unprivileged, *before* launching the terminal.
2. The health monitor applies a staged build proactively while the service is healthy, inside the daily market break or on Saturday, and reports the result on Telegram.
3. Only if the service is still unhealthy after a restart does the monitor run the helper itself.

The first attempted fix had used the vendor's web installer, which installs the latest *public* release. That release was older than the broker's build, so the "fix" downgraded the terminal and deepened the loop. The installer was removed from automatic recovery entirely. A later variant — a downloaded build arriving without its helper — is handled by moving the incomplete download aside and restarting.

15.2 The order path that never filled

From late July to late September the dashboard showed a healthy, running system. In fact it had **placed zero trades**: 126 attempts, of which 92 were stale proposals left over from an outage, 11 were correctly rejected by gates (seven news blackouts, three reward-to-risk, one spread) and **23 had failed at the broker — every single order that passed the gates**. The exception was being caught and recorded, but health checks and alerts watched only uptime, so the failure was invisible.

The causes were found by running the broker's order-check function as the service account, which reproduced the rejection exactly. Two causes were fixed (the filling policy and the algorithmic-trading switch) and verified with a tiny test order; the next real order then failed for a third reason — an over-long comment — that the test order had not exercised because its own comment happened to be short. The lesson was recorded in the project's method: **always test with the exact production request**. Order failures now raise an alert within the hour and appear in the daily report.

15.3 The macro read inside the broker lock

The macro read — fetching feeds and running the language-model agents — could take minutes. It ran inside the live loop's poll, which holds the lock that protects the broker connection; the position monitor shares that lock, so its last-check timestamp went stale, health failed and the monitor restarted everything. The monitor's update-loop detection then counted the whole day's update attempts, including a harmless one from the early morning, declared a loop and ran the installer — downgrading the terminal again. The fixes: refresh the macro read *before* the poll and outside the lock, so the poll only reads a cache; count update attempts in the last 30 minutes only; and remove the installer from automatic recovery.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

15.4 Memory starvation on a 1 GB server

For a week, the operator received "could not recover" and "healthy again" messages twice a day. The service log simply went silent, with no crash and no error. The server had about 138 MB of free memory; the agent service and the terminal had been paged out to working sets of 4–8 MB; restarts took four to seven minutes while the monitor waited only three, so every restart was reported as a failure followed, minutes later, by a recovery. The monitor's wait was extended to eight minutes, and the owner upgraded the server to 2 GB, which ended the incidents.

15.5 The weekend false alarm

The live loop guards against a stale feed: if the newest closed bar is older than the freshness bound, every agent is blocked and the cycle fails. That is correct on a Tuesday afternoon and wrong on a Sunday, when the market is closed. The guard had no notion of trading hours, so every weekend and every daily break the control room showed the whole team as failed or blocked.

The fix adds a market clock built from the instrument's trading sessions. A poll now reports `MARKET_CLOSED`, with the next opening time, **only if the feed kept up until the last session close**; a feed that died before the close is still a stale feed. After the market reopens, a grace period of one freshness bound lets the first bars arrive. The desktop shows the team as idle with "Market closed · reopens Sun 22:00 UTC". The fix is part of the code base; at the time of writing it had not yet been deployed to the cloud host, which is why the cloud screenshots taken on a Sunday in this report still show the stale-feed state.

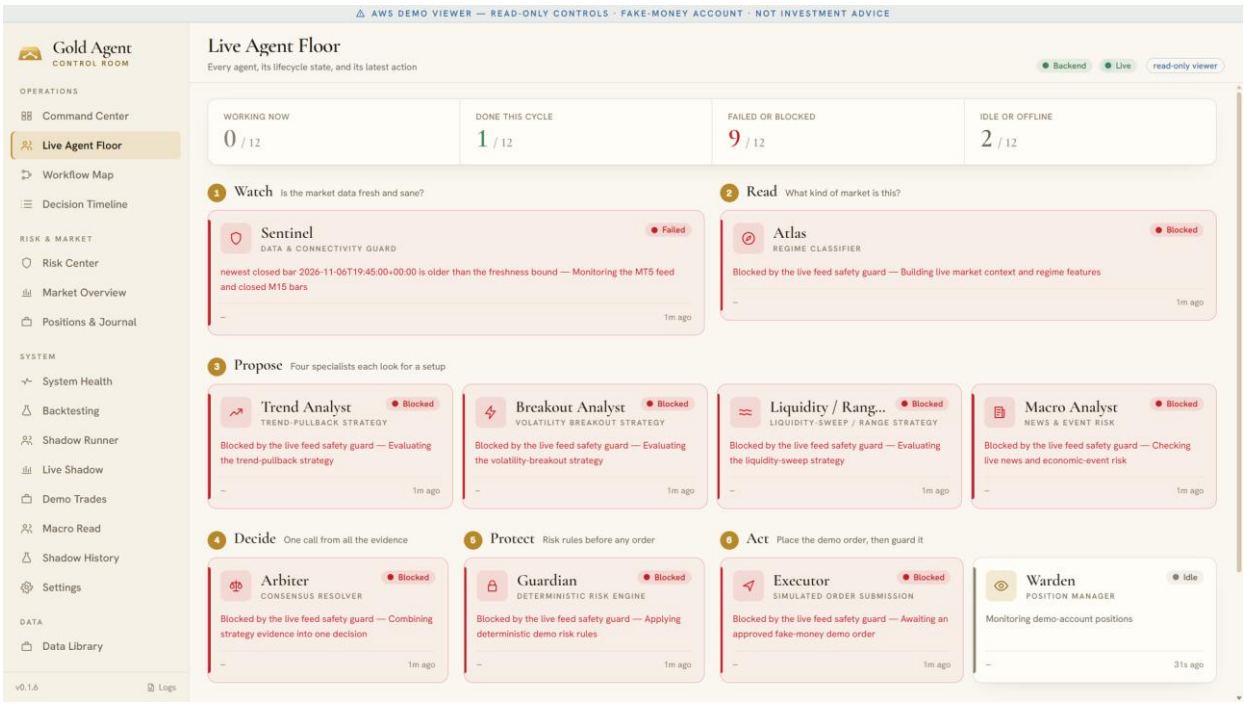


Figure 18. The cloud viewer on a Sunday, before the market-hours fix was deployed: the Sentinel fails on the closed market's last bar and every downstream agent is blocked — the safety guard doing its job, for the wrong reason.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

16 Results

This chapter reports every measured result: the fake-money demo account, the forward shadow history, the backtests, the walk-forward edge hunt and the shadow runner. All historical datasets are QUARANTINED broker data, so research results are indicative rather than validated. All numbers are reported as measured.

> **Summary.** The platform behaved as designed. The strategies it runs lost money on the demo account, in forward shadow tracking and in two of three backtests. The one positive backtest is not statistically significant, depends on gold's recent bull market and does not survive tuning. **No strategy has demonstrated an edge.**

16.1 The fake-money demo account

The full team traded a MetaTrader 5 demo account on the cloud server. The first fill occurred on 24 September; by early November the account had recorded 30 fills.

30 trades filled	**5 / 25** won / lost (17 %)	**−$309.41** realised result, net of costs	**−16.37R** realised result in R
291 signals checked	**261** signals not filled	**237** blocked by a safety gate	**24** refused or failed at the broker

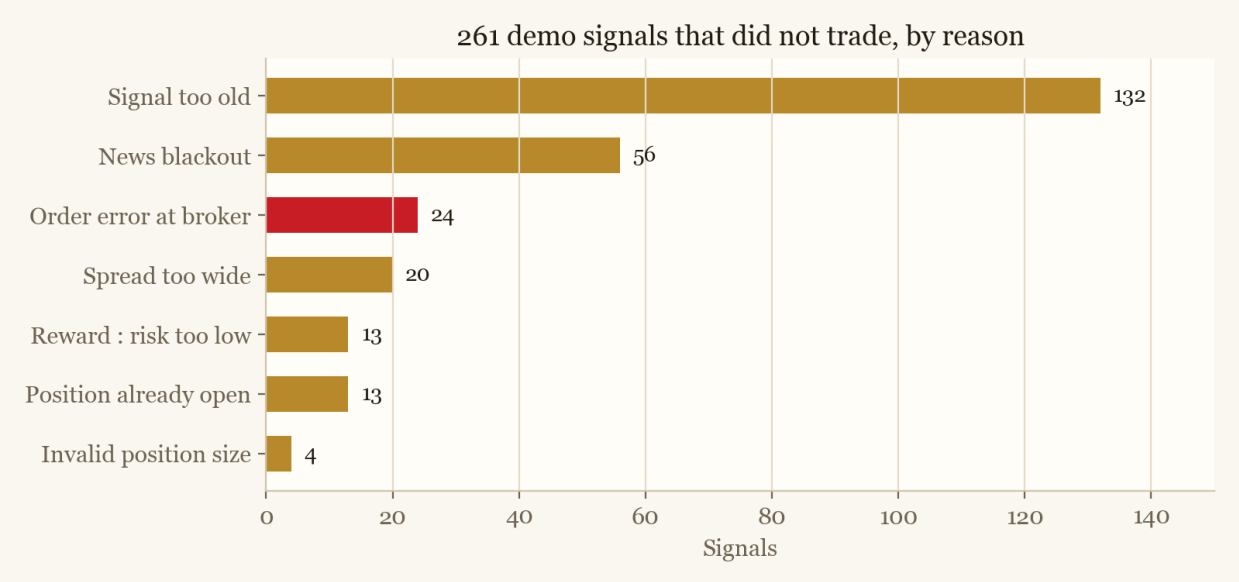


Figure 19. Why the 261 signals that reached the demo order path did not trade.

The largest category, "signal too old", is mostly the backlog of proposals generated during outages and correctly refused by the 20-minute freshness gate. News blackouts account for 56 refusals: the macro layer working as a brake.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Is the loss an execution problem?

A diagnosis in October compared every closed live trade with the shadow proposal for the same bar. All fifteen trades closed at that point had exactly the same outcome as their shadow proposal, with the same stop and target, and slippage relative to the signal averaged about zero. **The losses come from the signals, not from execution.**

Measured over all genuinely tradable signals since late July (excluding backfilled records), the picture was consistent:

Table 34. Live tradable signals, late July to mid-October

Measure	Value
Signals	98
Win rate	26 %
Mean result	−0.31R per signal (standard error 0.12)
Total	about −30R
Average target	1.77R → break-even win rate about 36 %
By direction	Both buys and sells negative
Trend	Worsening from August to October

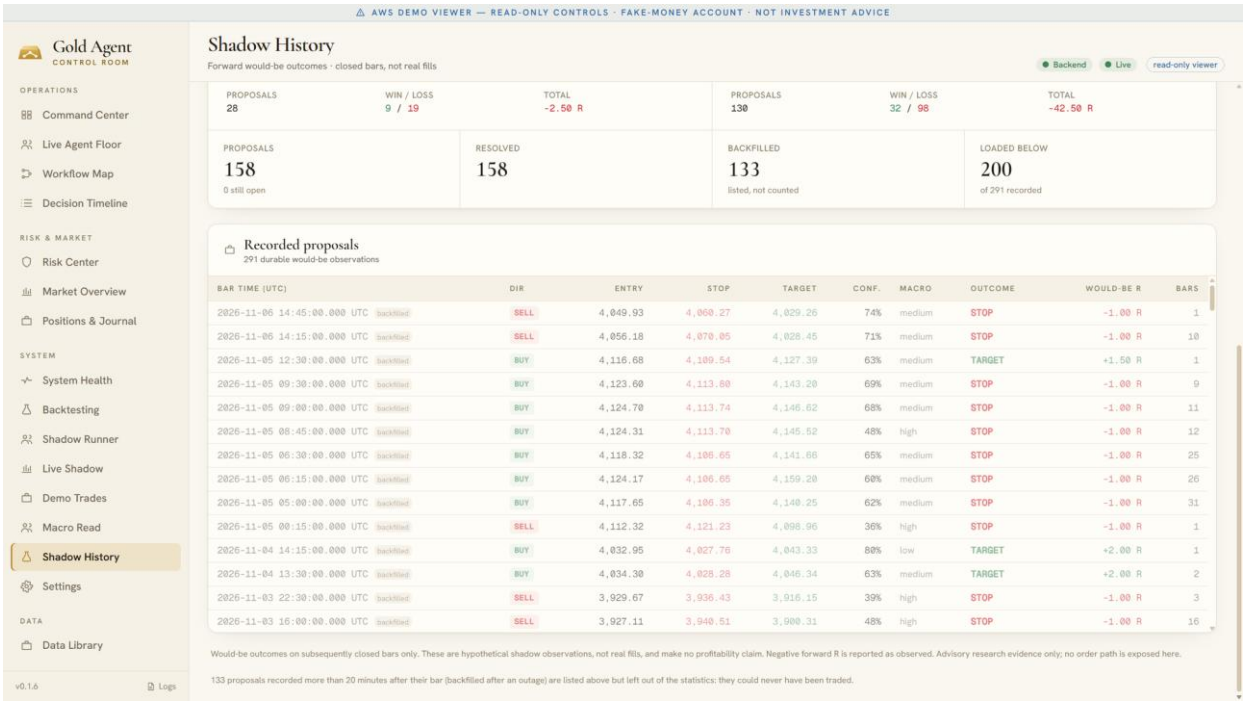


Figure 20. Shadow History in the cloud viewer, including the split by macro read.

16.2 Forward shadow history

The live loop recorded every proposal and scored it forward without an order, under the rules of Chapter 13 (pessimistic same-bar ordering, a 96-bar expiry, exclusion of late-recorded proposals).

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Table 35. Shadow history in November

Measure	Value
Proposals recorded	291
Counted (recorded on time and resolved)	158
Listed but not counted (backfilled after outages)	133
Won / lost	41 / 117 (26 %)
Total	**−45.00R**
Mean per proposal	−0.28R

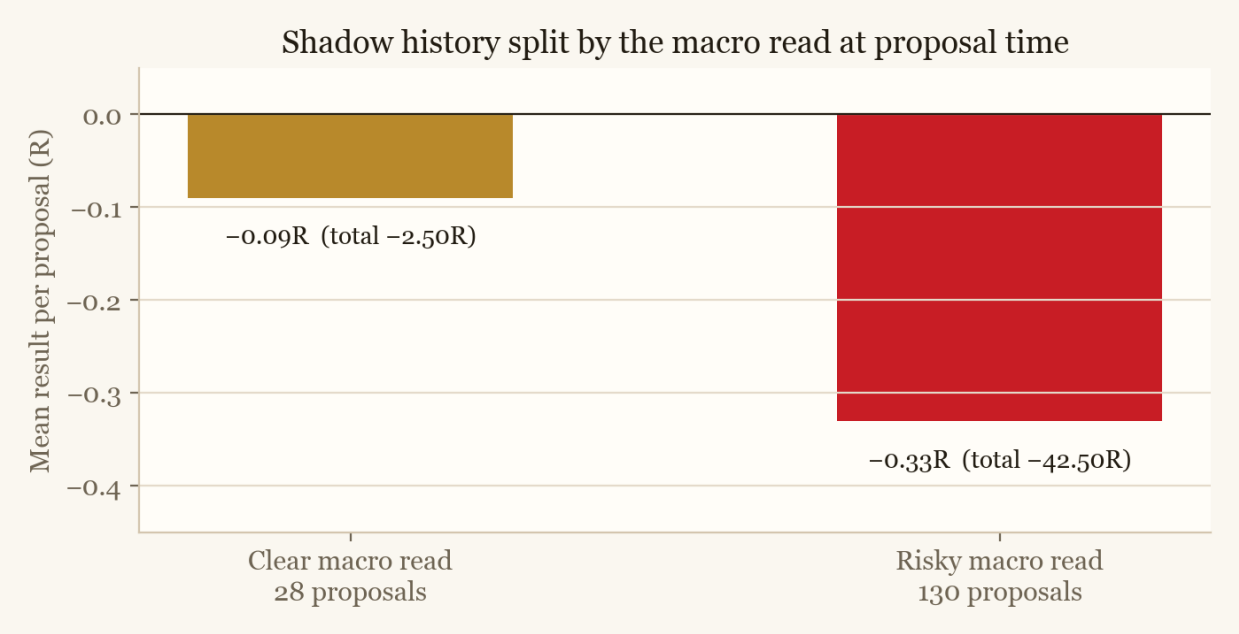


Figure 21. Mean result per proposal by the macro read at the time of the proposal.

Proposals made while the macro read was risky lost almost four times as much per trade as those made in clear conditions (−0.33R against −0.09R). That is consistent with the macro layer adding protective information, but the clear-macro sample is small (28 proposals) and still negative: the macro read reduces losses; it does not create an edge.

16.3 Backtests

The default trend-pullback strategy was backtested on each dataset with feature engine 1.1.0 and the default cost model.

Table 36. Backtests of the default trend-pullback strategy (engine 1.1.0)

Dataset	Trades	Win rate	Expectancy	Profit factor	Max drawdown	Max losing streak	Total	t-stat
M15 · 50,000 bars	1,019	32.5 %	−0.09R	0.87	108.67R	17	**−95.30R**	−2.16
H1 · 50,000 bars	1,064	33.6 %	−0.07R	0.90	105.30R	18	**−73.08R**	−1.62
H4 · 33,000 bars	732	36.7 %	+0.06R	1.09	28.19R	13	**+41.64R**	1.06

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

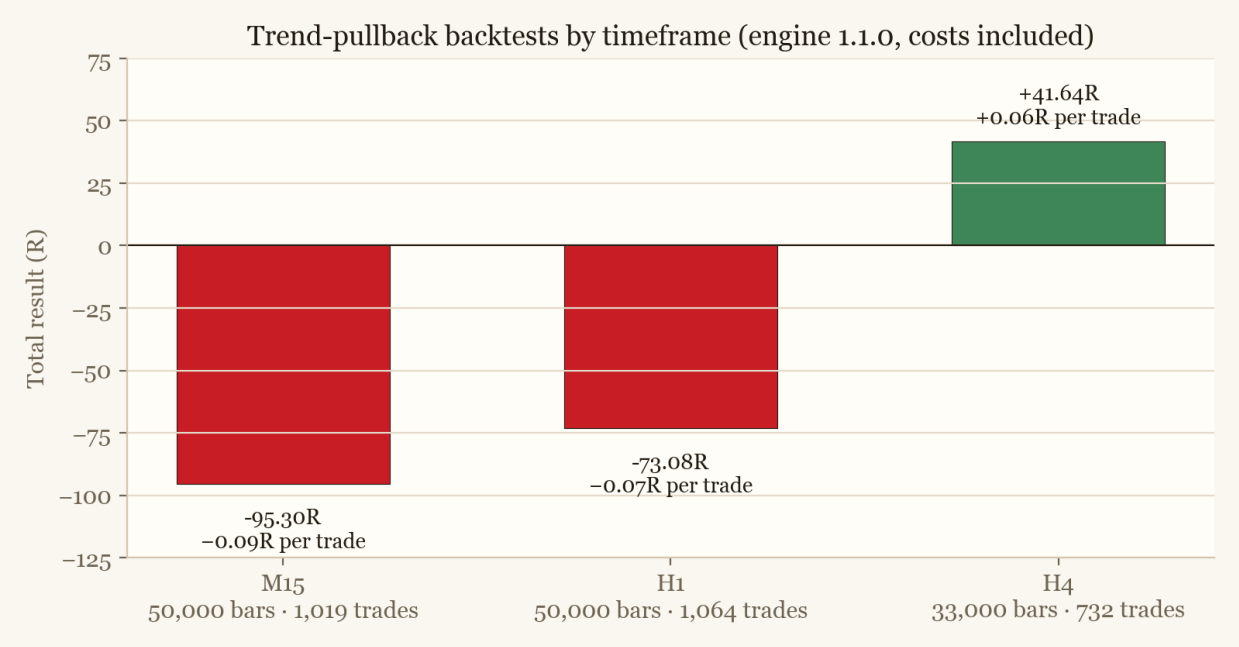


Figure 22. Backtest totals by timeframe.

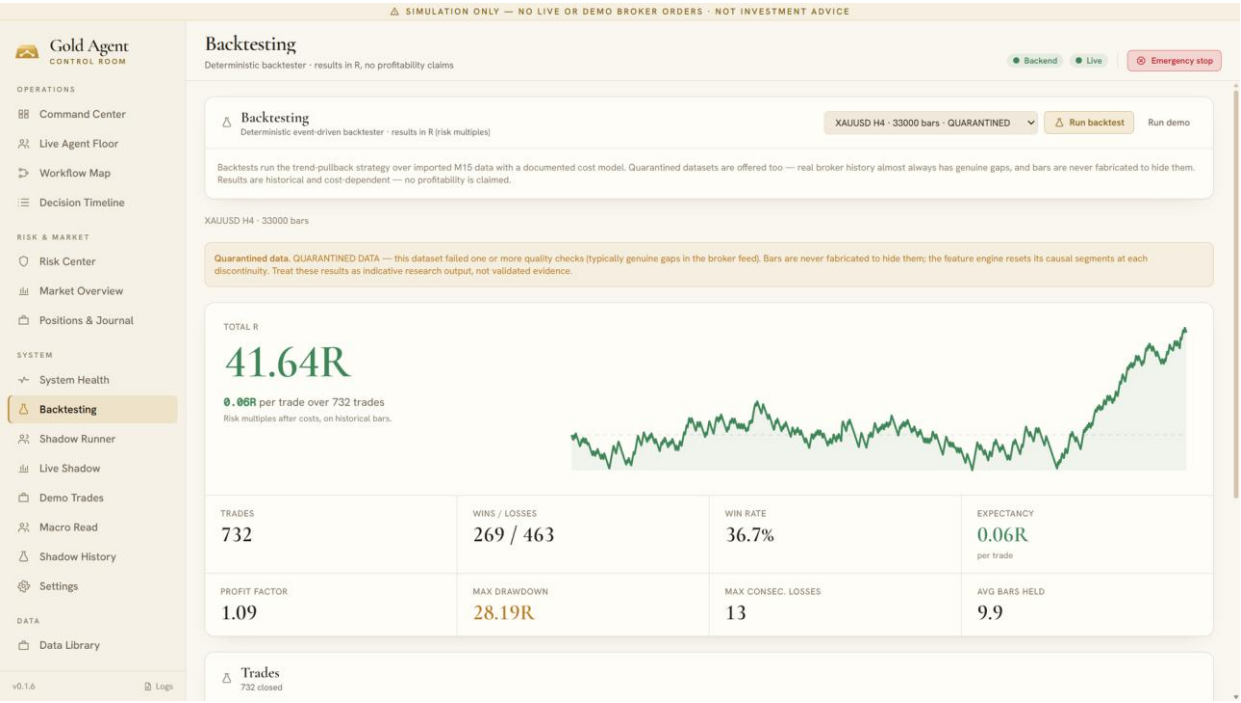


Figure 23. The H4 backtest in the application: total R, equity curve and the full metric ledger.

Costs are not the explanation

With every cost set to zero, the M15 strategy still loses (−0.024R per trade) and H1 is roughly flat (+0.007R); H4 rises to +0.102R. The losses on the shorter timeframes are not a cost artefact.

The H4 result under the microscope

Because H4 was the only positive result, it was examined further.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Table 37. The H4 default trend-pullback against baselines

Test	Result
Walk-forward with the default configuration	9 of 14 folds positive; out-of-sample +0.050R per trade; fold t = 1.29
Beats random-direction trades with the same geometry	98.8 % of 1,000 runs (p ≈ 0.012)
Beats random trades with the strategy's own buy/sell mix	97.8 % (p ≈ 0.022)
Beats always-buy	90.7 % (p ≈ 0.093 — not significant)
Where the profit comes from	Entirely 2022–2026: about −10.7R at the end of 2021, +41.6R by 2026
By direction	Buys +52.9R; sells −11.2R
Tuned walk-forward (108-configuration grid)	Out-of-sample −0.027R — fails

The result does not beat simply buying gold, its profit is concentrated in gold's 2022–2026 bull market, sells lose, and tuning destroys it. The honest reading is **long exposure to a rising asset, not skill**.

16.4 The edge hunt: tuned walk-forward on every strategy and timeframe

Each strategy's bounded grid was searched with walk-forward validation on each timeframe, and judged against the pre-registered rule with Bonferroni correction. The full suite was re-run after the feature-engine fix.

Table 38. Edge-hunt verdicts (engine 1.1.0)

Strategy	M15	H1	H4
Trend pullback	NO EDGE (−0.006R)	NO EDGE (−0.086R)	NO EDGE (−0.027R)
Volatility breakout	NO EDGE	NO EDGE (+0.119R, t 0.83 vs 3.58 required)	NO EDGE
Liquidity sweep	NO EDGE (−0.129R, t −4.99)	NO EDGE	NO EDGE
Range mean-reversion	INSUFFICIENT EVIDENCE	INSUFFICIENT EVIDENCE	INSUFFICIENT EVIDENCE

Twelve verdicts: nine `NO_EDGE_DEMONSTRATED`, three `INSUFFICIENT_EVIDENCE` (range reversion fires too rarely — 9 to 50 out-of-sample trades — to judge), and **zero passes**. The closest candidate, the H1 breakout (+0.119R out of sample, 15 of 22 folds positive, 298 trades), reached a test statistic of 0.83 against a required 3.58. The liquidity sweep on M15 is reliably bad (t = −4.99): it fires often and loses consistently.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

16.5 The shadow runner over history

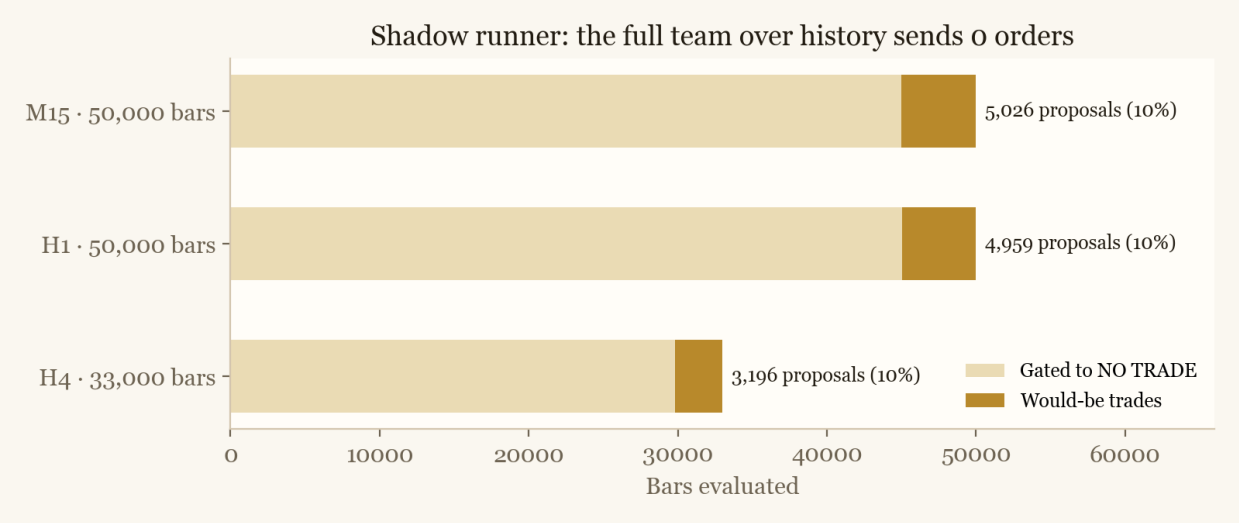


Figure 24. The full team over each dataset: about nine bars in ten end in NO TRADE, and no order is ever sent.

Table 39. Shadow runner results

Dataset	Bars evaluated	Gated to NO TRADE	Would-be trades	Orders sent
M15	50,000	44,974	5,026	0
H1	50,000	45,041	4,959	0
H4	33,000	29,804	3,196	0

The team is selective by construction: roughly 90 % of bars end in NO TRADE before any risk rule is even consulted.

16.6 What the results mean

- **The machinery is proven.** Data guards, freshness gates, news blackouts, the Guardian, reconciliation and the emergency stop all behaved as designed under real broker conditions, and every decision can be traced.
- **The technical strategies are not.** Four strategy families, three timeframes, up to 21 years of history, walk-forward with multiple-comparison correction, a live demo account and forward shadow tracking agree: none of the technical setups has a demonstrated edge on gold.
- **The macro layer helps defensively.** Risky-macro proposals lost more; blackouts prevented 56 trades. It does not, by itself, turn a losing signal into a winning one.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

17 Dead ends and retracted results

Research is mostly a record of ideas that did not work. This chapter collects the project's dead ends and, more importantly, the results that looked positive at first and were later retracted. Each retraction is an example of the validation process doing its job.

17.1 Retracted results

Table 40. Results that were later retracted

Original claim	Why it looked real	What retracted it	Corrected value
Tuned M15 trend pullback: +0.101R per trade, 5 of 7 folds positive, +23R	Positive in a walk-forward on 20,000 bars	2.5 times more data	+0.025R, 4 of 10 folds, t 0.32 — regressed to zero
Tuned H4 trend pullback: +0.34R, t 3.04, 8 of 10 folds	Closest anything came to significance	The adjacent timeframe: H1 gave −0.05R, t −1.19	Not replicated — a selection artefact
Default H4 trend pullback: +0.24R, t 2.78, beats random 99.8 %, 13 of 14 folds	Strong on every check	The feature engine reset its indicators every weekend	+0.06R, t 1.06, does not beat always-buy
Shadow history: +40R; "clear macro is far better"	Forward-tracked, live data	Backfilled proposals scored on prices after they were recorded	−45R over counted proposals

More data killed the first "edge"

On the first 20,000-bar M15 dataset, the tuned trend pullback looked encouraging: positive expectancy, five positive folds out of seven, +23R in total. Its test statistic (1.15) already failed the uncorrected bar, let alone the Bonferroni bar of 3.50 for 108 candidates — and on 50,000 bars the expectancy collapsed to +0.025R with a statistic of 0.32. The signal got *weaker* with more data, the signature of noise selected from a grid.

The adjacent timeframe test

The tuned H4 result (t = 3.04) was flagged at the time as promising but unproven, pending a cross-check on H1. If a trend-following rule works because it matches gold's trend horizon, a neighbouring timeframe should show at least a weaker version of it. H1 instead flipped sign (−0.05R, t = −1.19). A result that reverses between adjacent timeframes is a small-sample selection artefact.

The silent data bug

The default H4 result survived every check available at the time — Monte-Carlo baselines at p ≈ 0.002, 13 of 14 walk-forward folds — until a deeper look at the feature engine showed that it reset all indicators at every gap, including each weekend. On H4 that meant the strategy could only evaluate the last bars of each week with freshly reset indicators: an accidental "trade late in the week" rule that happened to fit history. Fixing the engine reduced the result to +0.06R per trade, not significant, and every earlier research number was formally retired and re-run (Chapter 7).

The backfilled-shadow artefact

In October the shadow history showed about +40R, and proposals made under a clear macro read looked dramatically better than the rest. A diagnosis found that 95 proposals had been recorded between 3 and 592

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

hours (median 293 hours) after their bar, during the catch-up after outages, and then scored against prices from *after* they were recorded, with their original stop and target. Those records alone contributed +46.7R of fiction, and nearly all of the clear-macro proposals were among them. Late-recorded proposals are now listed but excluded from every statistic.

17.2 Dead ends

- **Technical strategies on gold.** Four families, three timeframes, 21 years of H4 history, walk-forward with correction: no edge (Chapter 16).
- **A reinforcement-learning or meta-labelling filter.** The idea was to learn which signals to skip. With tradable signals averaging −0.31R and no positive subgroup of meaningful size, there is nothing positive for a filter to select. The idea was dropped before any model was trained.
- **Post-release drift.** Trading continuation after major US releases lost on both samples (Chapter 10).
- **Range mean-reversion.** Too selective on gold to produce enough trades for any verdict.
- **The "horizon" hypothesis.** If H4 worked because it matched gold's trend horizon, H1 with four-times-longer lookbacks should behave like H4. It did not (+0.031R and +0.005R in two variants, out of sample negative).

17.3 Lessons

1. **Pre-register the bar before looking.** Every retraction above would have been a published "edge" without a decision rule fixed in advance.
2. **Correct for the size of the search.** The best of 108 noisy configurations is positive by construction.
3. **More data is the cheapest falsification test.** If an effect weakens as the sample grows, it was noise.
4. **Check the neighbours.** A real effect should not reverse on an adjacent timeframe.
5. **Beat the naive baseline that matters.** In a bull market, beating random entries is easy; beating always-buy is the test.
6. **Audit the data pipeline, not just the strategy.** The most convincing false edge came from a feature-engine bug that broke no test.
7. **Make the timing of a record part of its validity.** A forward record scored with hindsight is not forward evidence.
8. **Write retractions down where the next session will see them**, next to the original claim, with an instruction never to cite the old number.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

18 The control room

The control room is how people meet Gold Agent. Its purpose is not to sell trades but to make a team of agents legible: what each one is doing, what it decided and why, how close the system is to every limit, and what the results really are.

18.1 Design language

In November the interface was redesigned from a dark dashboard into a light, editorial **ivory-and-gold** language, without changing any logic. The direction combined two ideas: an isometric "vault floor" stage for the Workflow Map, on which each agent is a gold-leaf block on an ivory floor, and a "bullion ledger" style for every other screen — serif headlines and numbers, ruled ledger tables, generous whitespace and a single gold accent.

Table 41. Design tokens

Token	Colour	Use
Base	Ivory #FAF7F0	Page background
Surface	#FFFDF8	Cards and panels
Vellum	#F6F1E6	Section headers, ledger heads
Ink	#221C12	Text
Gold	#B8892B	The one accent: handoff lines, highlights, focus
Deep gold	#8A6418	Gold text on ivory
Gold leaf	#EADBB4	Washes and tracks
Success	#3E8658	Completed, wins
Information	#2F6BA8	Working
Warning	#B7791F	Caution
Danger	#C81D25	Losses and the emergency stop only

Typography pairs a serif display face for headlines and large numbers (with lining numerals, so "11" never reads as "II"), a clean grotesque for text and a geometric monospace for prices and times; all three are bundled with the application because its security policy forbids loading fonts from the network. Motion is "alive but calm": handoff lines flow gently between active agents, panels fade in with a short stagger, and everything respects the operating system's reduced-motion preference. A new application icon — a gold ingot with three linked points for the team — completes the identity.

18.2 The sixteen screens

Table 42. Screens of the control room

Group	Screen	What it answers
Operations	Command Center	What is the price, what was the last call, how safe are we, what is the team doing?

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Group	Screen	What it answers
Operations	Live Agent Floor	What is each agent doing right now, step by step?
Operations	Workflow Map	How does a decision flow through the team, and where is it now?
Operations	Decision Timeline	What exactly happened in each decision, event by event?
Risk and market	Risk Center	How close are we to each limit, and what did the Guardian last reject?
Risk and market	Market Overview	What do the market and each analyst see?
Risk and market	Positions & Journal	What is open, and what closed, with the full decision chain?
System	System Health	Is every part of the system healthy?
System	Backtesting	How would a strategy have done on history?
System	Shadow Runner	What would the whole team have done on history?
System	Live Shadow	What is the live loop doing on the real feed?
System	Demo Trades	What really happened on the demo account?
System	Macro Read	What do the news and calendar agents think, and why?
System	Shadow History	How did the live proposals turn out, forward?
System	Settings	Which trading mode is locked in, and with which flags?
Data	Data Library	Which datasets exist, and how good are they?

Command Center

The landing screen leads with the price, a sparkline of recent bars, the session and regime; then "the last call" — the decision of the latest cycle in large serif type; a safety ledger (risk status, emergency stop, backend, consecutive losses, open risk, open positions); the simulation controls; and the team at a glance.

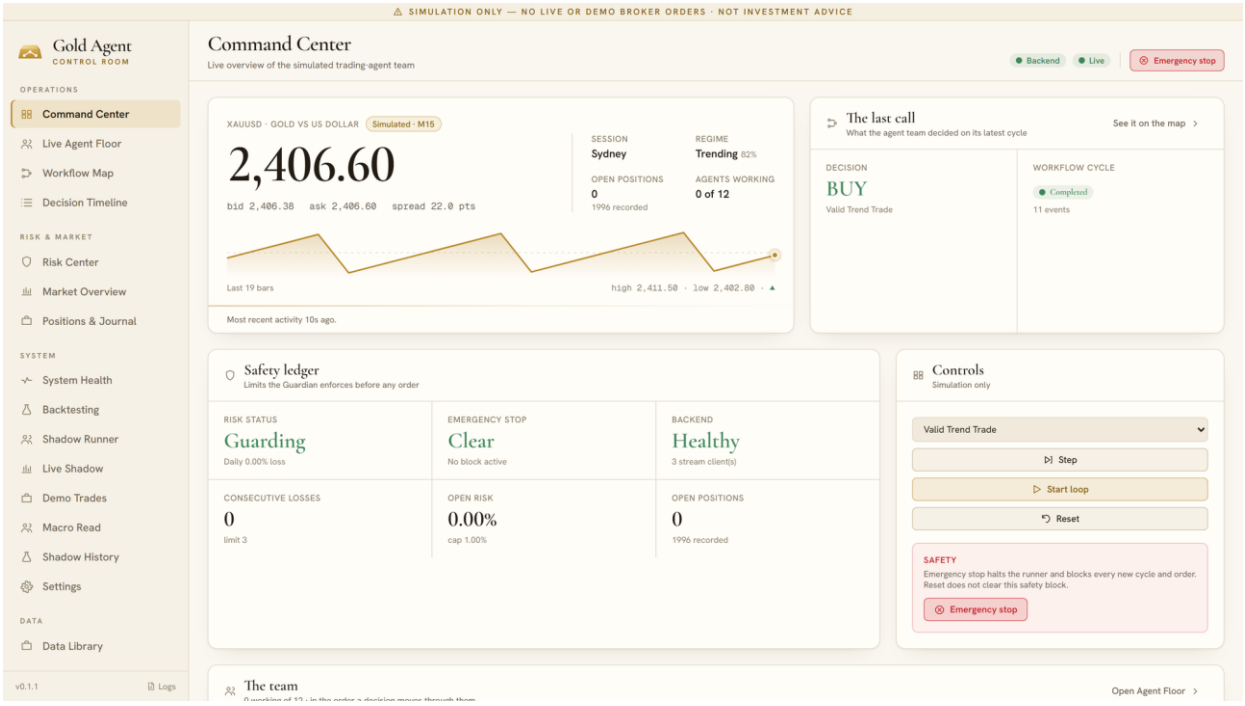


Figure 25. Command Center (local simulation) after a valid trend-trade cycle: the last call is BUY, every safety cell is green.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Decision Timeline

A rail on the left lists recent decisions with their outcome; the main list shows every event of the selected cycle along a gold spine — agent, event type, task, confidence and handoff — searchable, filterable and exportable.

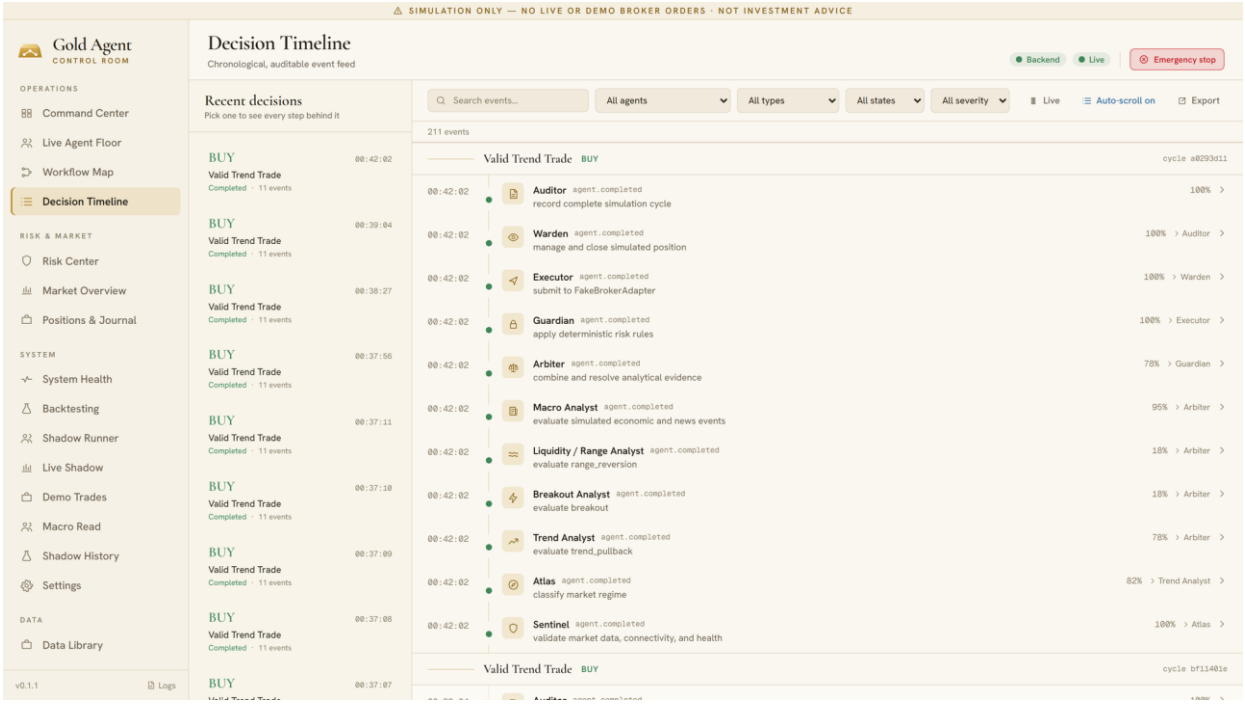


Figure 26. Decision Timeline (local simulation): the decision rail and the event chain of one cycle.

Market Overview

The price chart, a ledger of the market state (price, bid, ask, spread, regime, session, volatility, data age), the latest candle, what each analyst sees with its confidence, and the latest proposal on a price ladder with its target and stop.

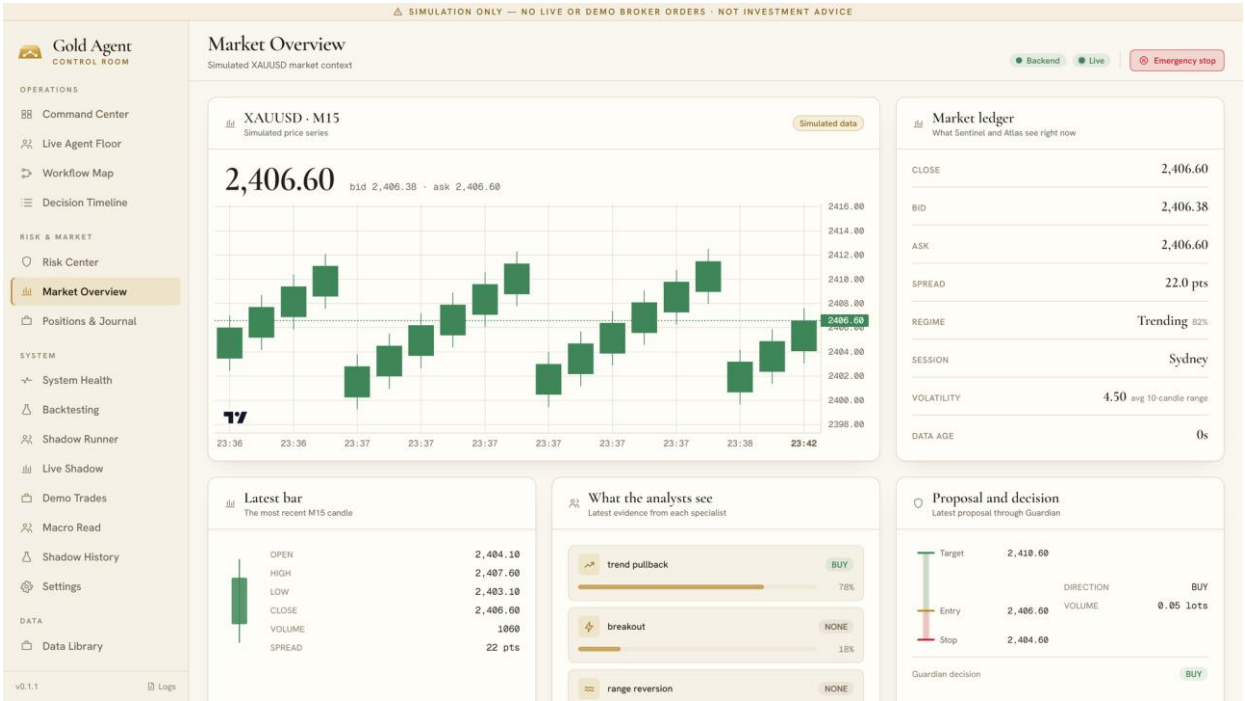


Figure 27. Market Overview (local simulation).

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

System Health

One word sums up the system — Healthy, Attention, Halted or Down — above a row of status lamps for the backend process, API readiness, the event stream, the safety state, the simulation runner and its capability; then the backend process, stream recovery and the health of every agent.

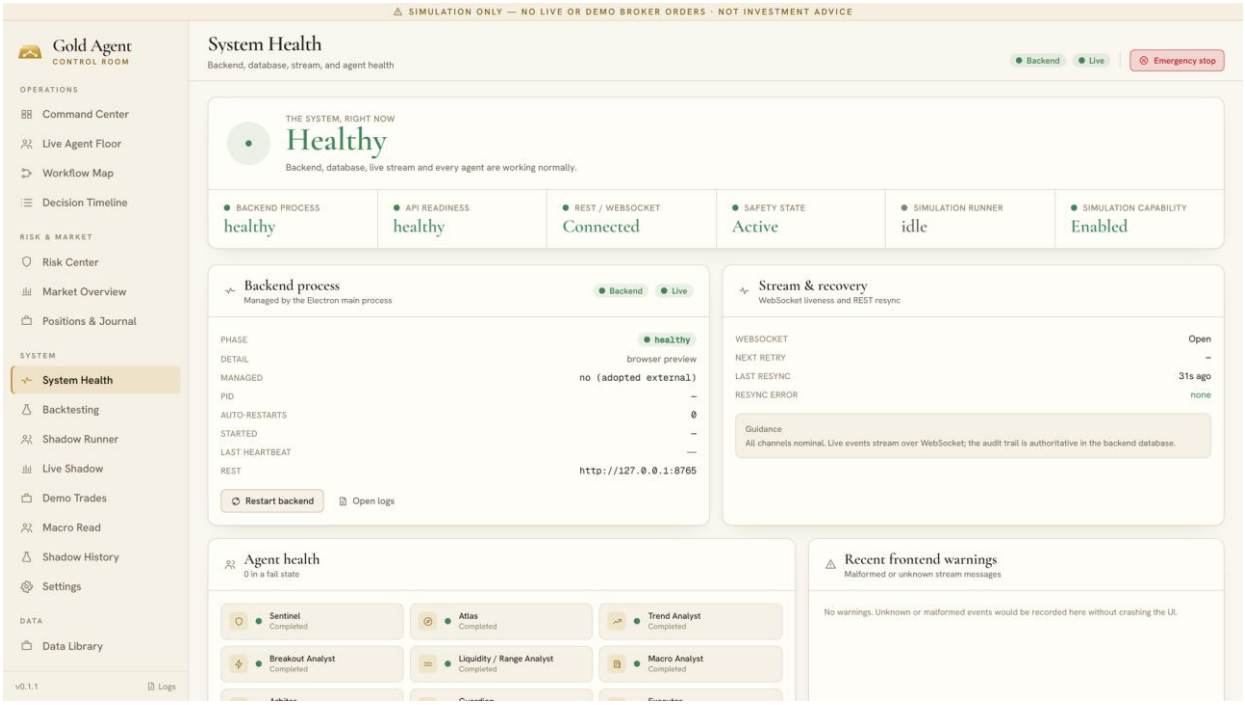


Figure 28. System Health (local simulation).

Live Shadow

The live loop's state in large type, the feed's freshness, orders, proposals and polls, the macro overlay strip and the table of recent proposals.

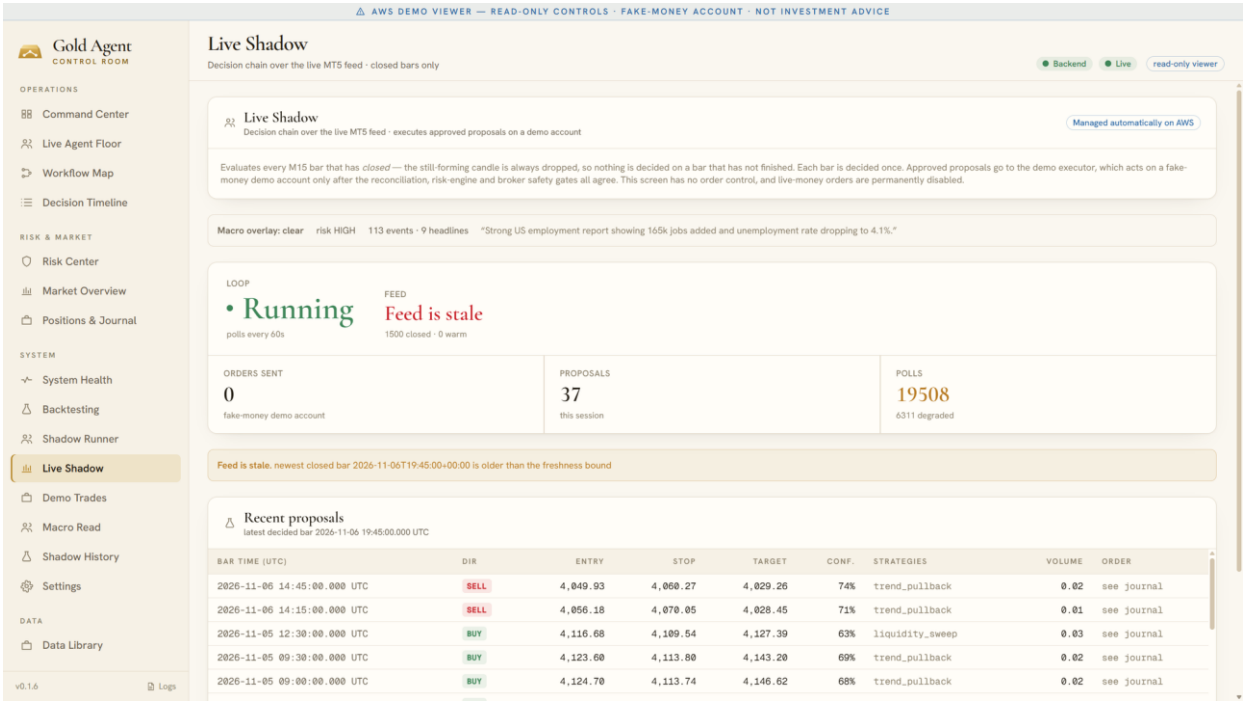


Figure 29. Live Shadow in the cloud viewer, on a Sunday: the loop is running but the feed is stale because the market is closed.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Backtesting

A dataset picker and run button; then the result: Total R as the headline beside the equity curve, and a ledger of eight metrics; the full trade list follows.

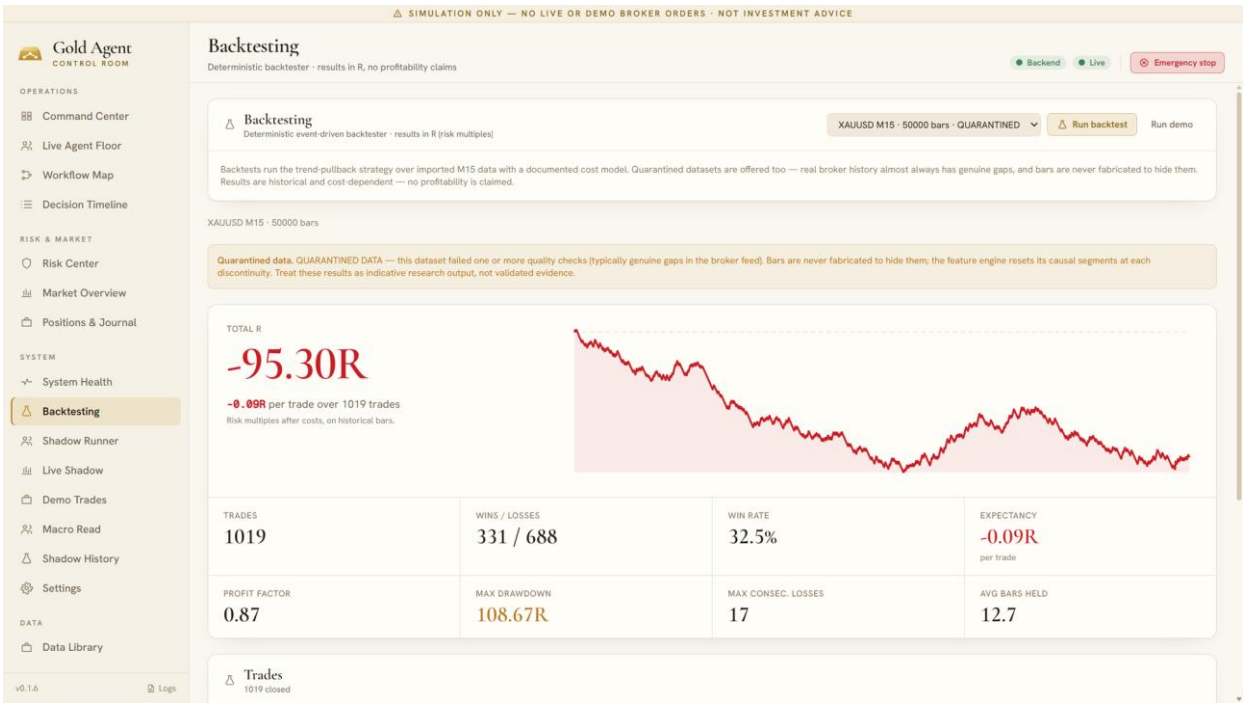


Figure 30. Backtesting on the M15 dataset.

Positions & Journal

Net simulated result as the headline with a running total and a win/loss strip; open simulated positions; and a journal of completed trades, each expandable into its full decision chain.

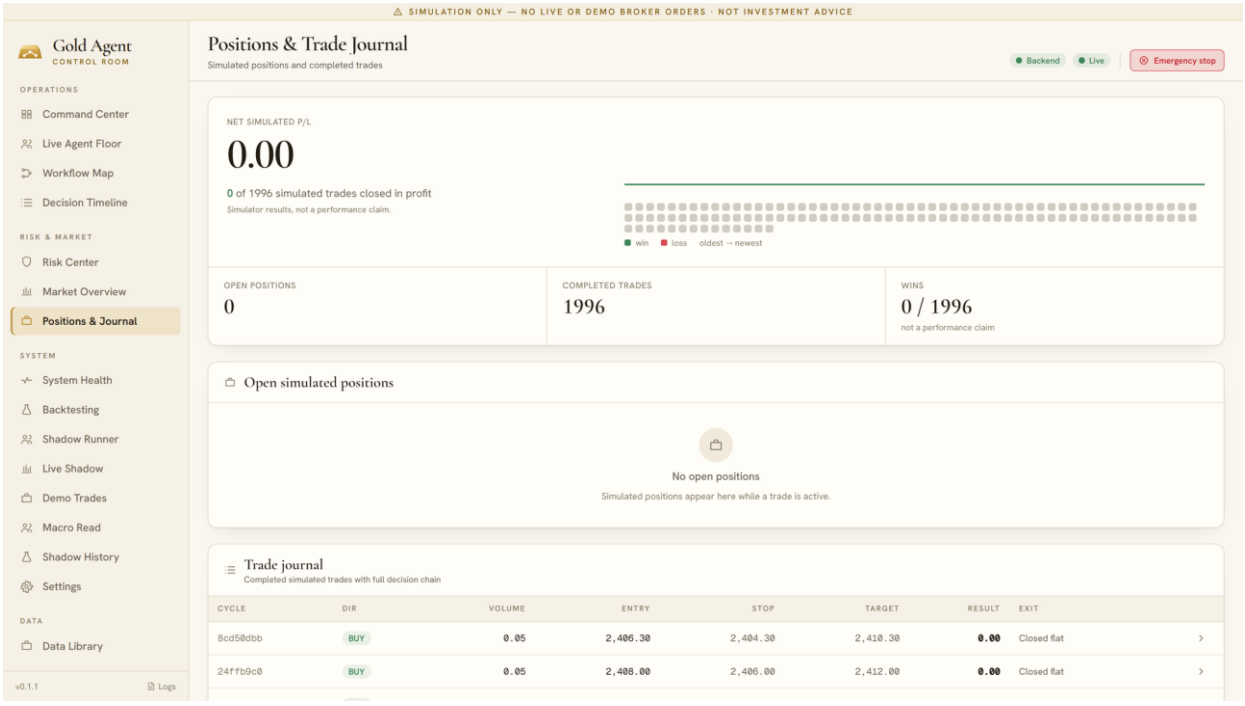


Figure 31. Positions & Journal (local simulation).

18.3 Accessibility and robustness

- Every screen has real headings, landmark regions and keyboard-reachable controls; the agent blocks on the Workflow Map are focusable buttons with descriptive labels.
- Dialogs trap focus, focus the safe action first and close on Escape.
- Loading, empty and error states exist for every data view; an unavailable feature explains why (for example, research runs in the read-only viewer).
- The renderer never trusts its own data for safety: the emergency stop and every command are enforced by the agent service, not by the interface.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

19 Quality and security

19.1 Testing

Table 43. Test suites

Suite	Scope	Size
Backend unit and integration	Contracts, lifecycle, risk, data import and quality, features, strategies, backtester, walk-forward, stress, experiments, arbiter, sizing, MT5 adapter and bridge, executor, reconciliation, live runner and services, agents, feeds, API routers, security hardening	More than 860 test cases in 60 files
Contract fixtures	Python and TypeScript serialisation parity	Exported fixture set, checked in CI
Desktop unit	Components, screens, stores, formatting, the workflow stage, charts	165 tests in 35 files
Shared contracts	TypeScript contract validation	8 tests
End-to-end, development build	Smoke, safety lifecycle, backtesting, shadow runner, live shadow, macro read, shadow history	14 journeys in the real Electron app
End-to-end, packaged installer	Self-contained package: control room, emergency stop, forced backend restart, history continuity, guarded reactivation	1 long journey

Several test practices deserve mention:

- **Failure injection.** The demo executor, the bridge and reconciliation have tests that inject broker errors, partial fills, slippage, duplicates, expired commands and mismatched ledgers.
- **Exhaustive safety properties.** The monotonic challenger is tested on every combination of state and verdict; the live router's allowed paths are pinned so that an order route cannot be added silently.
- **Hermeticity.** Tests construct settings without reading the developer's environment file, after a defect in which the suite had been connecting to a real terminal.
- **Isolation of end-to-end runs.** The journeys run with the broker and the language models disabled, on a throwaway database, on their own port and with their own user-data folder, so they can run beside a developer's open application without touching its data.

19.2 Continuous integration

Two workflows guard the main branch. The quality workflow runs lint, types and tests for the service and the desktop on every change. The Windows release gate additionally checks the contract fixtures, scans for secrets, audits dependencies, builds the packaged application with its bundled backend, runs the end-to-end journeys and the packaged-installer journey, and keeps diagnostic artefacts for only a few days.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

19.3 Security review

At the end of July, before the cloud demo was left unattended with a real order path, a full security review — graded from critical to low — covered the service, the desktop, the operations scripts and the dependencies. The main findings, grouped by area:

Table 44. Security review findings and fixes

Area	Finding	Fix
Host privileges	Three paths by which a writable folder or temporary file could be executed by a SYSTEM task	Protected, non-inheriting permissions on the operations folder and the bundled interpreter; no execution from temporary files
Order safety	The reconciliation gate was bypassed: the demo service hard-coded "reconciled"	Reconciliation consulted before every order; evidence produced by the position monitor; never-reconciled blocks
Secrets	The private environment file was readable more broadly than necessary	Restricted to SYSTEM and administrators, read-only for the runtime account
Availability	Blocking I/O on the event loop could delay the emergency stop by about 90 seconds	Feed, model and large history reads moved off the event loop
Network input	Redirect-based server-side request forgery and unbounded response bodies in feeds	Same-origin, HTTPS-only redirects; byte caps; bounded parsing
Secrets in transit	Unvalidated language-model endpoint URLs	HTTPS required (loopback excepted)
Model input	Prompt injection through headline text	Untrusted-data prompts, one-line headlines, schema-validated outputs
API	A malformed credential header caused a server error instead of a rejection	Shape check before a constant-time comparison; audited 401

Table 45. Threat model (selected rows)

Threat	Control
Compromised renderer	Context isolation, no Node integration, narrow typed preload, inert background behind dialogs
Shell injection	No shell bridge; fixed executables and arguments in the main process
Credential exposure	Credentials only in protected local files; secret scan in CI
Model overreach	Models cannot size, authorise, execute or deploy
Duplicate or replayed orders	Deterministic client identifiers; idempotency in the risk engine, the bridge and the broker client
Stale or corrupt data	The Sentinel, the Guardian's data-age rule and the 20-minute proposal-age gate
Backend crash	Durable audit trail; reconciliation gate on start-up
API exposure	Loopback bind; per-launch capability token; separate read-only viewer credential
Hostile historical file	Controlled staging, safe names, link rejection, exact hashes, strict parsers and limits
Hostile feed	Same-origin redirects, byte caps, entity-free XML parsing, staleness guard

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

19.4 Dependency hygiene

Dependencies are pinned to exact versions with a lockfile, and the release gate fails on any high or critical advisory. In November, newly published advisories — 56 high and 2 critical in libraries used to build and test the application — had been failing the gate on every branch for weeks. The application was upgraded to a newer major version of its desktop runtime (which fixed eight runtime advisories and removed a vulnerable archive library) and of its test runner (which removed the dependency carrying the two critical advisories), and eight indirect libraries were pinned to patched versions within their own major lines so that their behaviour could not change. The audit went to zero high and zero critical, a previously documented audit exception became unnecessary and was removed, and every unit test, end-to-end journey and the packaged-installer journey passed on the new versions.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

20 Timeline

Table 46. Dated development timeline (pull-request numbers in the private repository)

Date	Milestone
21 Jul	Repository initialised; governance, architecture and typed monorepo established
21 Jul	Foundation: deterministic simulation architecture, twelve simulated agents, risk engine, audit store, fake broker (#36)
21 Jul	Desktop control room: supervision, capability-token auth, screens, packaging (#41); v0.1.0-simulation milestone (#42)
21 Jul	Guarded emergency-stop recovery and health-state correction — v0.1.1-simulation (#44)
21 Jul	Historical XAUUSD importer with 38 quality rules and the content-addressed data lake (#45)
22 Jul	Data Library interface (#48); feature engine (#49); trend-pullback strategy (#50)
22 Jul	Event-driven backtester (#51); walk-forward (#52); backtest visualisation (#53); stress testing (#54)
22 Jul	Read-only MT5 adapter (#55); shadow mode (#56); economic calendar and blackout gate (#57)
22 Jul	Breakout and range strategies (#58); macro-news LLM agent (#59); Arbiter (#60); position sizing (#61)
22 Jul	MQL5 safety bridge (#62); MT5 demo execution (#63); MT5 history ingest (#64)
23 Jul	Shadow decision runner wiring the full chain (#65); session-aware gap classification (#66)
23 Jul	Research on QUARANTINED data (#67); Shadow Runner screen (#68); live shadow loop (#69) and its supervision (#70)
23 Jul	Pre-registered experiments and analytics (#71); auto-supersede (#73); liquidity-sweep strategy (#74)
23 Jul	H1 and H4 timeframes (#75); free, key-less calendar and news sources (#76)
24 Jul	Live macro provider with the news-sentiment agent (#77); Macro Read screen; trade reconciliation engine
27 Jul	Persistent shadow-proposal history with forward outcome scoring
28 Jul	Guarded MT5 demo runner with daily Telegram report; secure AWS demo viewer
29–30 Jul	Unattended-operations hardening; quota-aware LLM failover; full security review and fixes
3 Aug	MetaTrader update loop takes the cloud demo down
19 Sep	Multi-source news, tool-using agent deliberation, cost governor; stale-proposal gate; new demo account
23 Sep	Unprivileged MetaTrader updates; feature engine 1.1.0; macro read moved out of the broker lock; research re-run
24 Sep	Order path fixed (filling policy, algorithmic trading, comment length); **first real demo trade**; Demo Trades screen
1 Oct	Recovery from updates downloaded without their helper
12 Oct	Longer restart wait; backup retention cut; server upgraded to 2 GB
13 Oct	Live diagnosis; late-recorded proposals excluded; macro bias tracker; post-release drift study
7 Nov	Market-hours awareness (`MARKET_CLOSED`); backup tooling; ivory-and-gold redesign of every screen
8 Nov	New icon, screenshots and hero animation; merge of the AWS runner and redesign (#92)
8 Nov	Security upgrade to zero high and critical advisories (#93); showcase film (#94); public showcase repository

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

21 Conclusions and future work

21.1 What was achieved

In under four months Gold Agent went from an empty repository to a complete, operating system: a team of twelve agents with a hard boundary between ideas and money; a data platform that refuses to fabricate; a research harness that resists self-deception; a guarded order path to a broker; an unattended cloud deployment with self-healing operations; and a control room that makes all of it legible. Every safety property that matters — no live money, no model-controlled risk, a durable emergency stop, a challenger that can only make the system more cautious, a shadow mode that cannot send an order — is enforced structurally and checked by tests.

21.2 What was learned

The central research question — does any of these technical strategies have an edge on gold? — has a clear answer for now: **no**. Four strategy families on three timeframes, walk-forward validation with correction for multiple comparisons, Monte-Carlo baselines, a fake-money demo account and forward shadow tracking all point the same way. Several results that looked like edges along the way were caught and retracted by the same process, which is the strongest evidence that the process works.

The second lesson is about operations. Most of what went wrong in production was not in the trading logic but around it: a terminal update that needed an answer nobody could give, an order field longer than an undocumented limit, a lock held too long, a server too small, a guard that did not know when markets close. Each was found by measuring the real system, and each fix made the system more honest about its own state.

21.3 Who it is for

- **For traders and prop firms**, Gold Agent is a disciplined execution and research framework: a place to express a strategy as an agent, validate it without fooling oneself, run it under hard risk rules and watch every decision. The framework, not any particular strategy, is the asset.
- **For engineers and recruiters**, it is an example of building a safety-critical system with AI-assisted engineering: contracts first, gates on every change, failure-injection tests, structural safety properties and an honest record of what went wrong.

21.4 Future work

1. **Fundamental and macro inputs.** The only hypothesis not yet exhausted. The bias tracker is collecting forward evidence on the macro agents' directional calls; any signal built on it will be tested offline first, then in shadow, before it reaches the demo account.
2. **Deploy the market-hours fix** to the cloud host, so that weekends and daily breaks are reported as closed markets rather than stale feeds.
3. **Preserve the cloud history.** Complete the scheduled backup before the free credits end, and keep the audit store as the permanent record of the live period.
4. **Forward-track the closest candidates**, such as the H1 breakout, alongside the live M15 loop, without trading them.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

5. **Broaden the data**: a second broker's history and other trending instruments, to test whether any effect is specific to one feed.
6. **Keep the bar where it is.** No real-money pilot until a strategy survives a pre-registered, out-of-sample test — and then only a restricted one.

> Gold Agent was built to answer a question honestly. For now the answer is that the team knows how to watch, decide, protect and record — and that the strategies it was given are not yet good enough to trade. The research continues.

21.5 Contact

Gold Agent is a private project and is not open source. Traders, prop firms or teams interested in a private demonstration, licensing or custom work can get in touch through the author's GitHub profile: github.com/ilyasdaoudrma.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

22 Appendices

22.1 Appendix A — Glossary

Table 47. Glossary

Term	Definition
Agent	A component with one responsibility, explicit inputs and outputs, an observable lifecycle and a defined failure behaviour
Arbiter	The agent that combines strategy signals and the macro read into one decision
ATR	Average True Range, a measure of volatility
Backfilled proposal	A proposal recorded long after its bar, typically while catching up after an outage
Bonferroni correction	Dividing the significance level by the number of candidates tested, to control false discoveries across a search
Blackout	A period around a high-impact economic event in which trading is suppressed
Capability token	A per-launch secret that the desktop presents to the local agent service on every request
Demo account	A broker account funded with fake money
Drawdown	The largest fall of cumulative results from a previous peak
Edge	A statistically demonstrated positive expectancy after costs, out of sample
Emergency stop	A durable halt of all cycles and orders, cleared only by a guarded procedure
Expectancy	The average result per trade, in R
Fail closed	Resolve any uncertainty to the safe outcome — NO TRADE
Guardian	The deterministic risk engine with twelve rules
In-sample / out-of-sample	Data used to choose parameters / data used only to evaluate them
Live runner	The loop that evaluates each closed M15 bar from the broker feed
MetaTrader 5 (MT5)	The broker's trading terminal and its Python interface
MQL5	MetaTrader's programming language, used for the safety Expert Advisor
NO TRADE	The default, safe outcome of a decision cycle
Profit factor	Gross wins divided by gross losses
QUARANTINED	A dataset with findings that require caution; usable for research only with a warning
R	One unit of risk: the loss if a trade hits its stop
Reconciliation	Comparing the system's ledger of positions with the broker's
Shadow	Recording what the system would have done, without sending an order
Walk-forward	Repeatedly choosing parameters on one window and testing them on the next
XAUUSD	Spot gold priced in US dollars

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

22.2 Appendix B — Reason-code families

Table 48. Families of deterministic reason codes

Family	Examples	Raised by
Data	stale feed, `MARKET_CLOSED`, feed error, no closed bars, warming up	Sentinel, live runner
Macro	`macro_blackout`, `macro_unavailable`, `news_blackout`	Macro layer, Arbiter, demo gate
Decision	`no_strategy_signals`, `strategy_disagreement`, `insufficient_agreement`, below minimum confidence	Arbiter
Sizing	`below_min_volume`	Position sizing
Risk	per-trade, open-risk, daily, weekly, streak, volume, reward-to-risk, spread, slippage, data-age, margin and position-limit rejections	Guardian
Execution	`proposal_stale`, order error at the broker, `FAILED`, `RECONCILE_MISMATCH`	Demo executor
Reconciliation	`MISSING_ON_BROKER`, `UNEXPECTED_ON_BROKER`, `DIVERGENT_VOLUME`, `DIVERGENT_DIRECTION`, `DUPLICATE_TICKET`	Reconciliation
Safety	emergency stop engaged, kill switch, reactivation refused	Guardian, safety service
Viewer	`VIEWER_READ_ONLY`	API

22.3 Appendix C — The 38 data-quality rules

Table 49. Data-quality rule identifiers

Family	Rule identifiers
File and schema (10)	`FILE.UNSUPPORTED_FORMAT`, `FILE.INVALID_ENCODING`, `FILE.EMPTY_DATA`, `SCHEMA.MISSING_COLUMNS`, `SCHEMA.DUPLICATE_COLUMNS`, `SCHEMA.AMBIGUOUS_MAPPING`, `SCHEMA.TYPE_MISMATCH`, `SCHEMA.MIXED_SYMBOLS`, `SCHEMA.MIXED_TIMEFRAMES`, `SCHEMA.DRIFT`
Time (11)	`TIME.MISSING_TIMEZONE`, `TIME.AMBIGUOUS_TIMEZONE`, `TIME.INVALID_TIMESTAMP`, `TIME.DUPLICATE_TIMESTAMP`, `TIME.CONFLICTING_DUPLICATE`, `TIME.OUT_OF_ORDER_ROW`, `TIME.M15_MISALIGNMENT`, `TIME.UNEXPECTED_GAP`, `TIME.UNKNOWN_SESSION_GAP`, `TIME.FUTURE_TIMESTAMP`, `TIME.EXPECTED_SESSION_GAP`
Prices (6)	`PRICE.NON_FINITE`, `PRICE.NON_POSITIVE`, `PRICE.HIGH_BELOW_OHLC`, `PRICE.LOW_ABOVE_OHLC`, `PRICE.EXTREME_DISCONTINUITY`, `PRICE.UNKNOWN_BASIS`
Spread and ticks (7)	`SPREAD.NEGATIVE`, `TICK.BID_ABOVE_ASK`, `SPREAD.MISSING`, `SPREAD.CONVERSION_IMPOSSIBLE`, `TICK.DUPLICATE`, `TICK.CONFLICTING_DUPLICATE`, `TICK.NON_MONOTONIC_TIME`
Volume and source (4)	`VOLUME.NEGATIVE`, `VOLUME.UNSUPPORTED_MEANING`, `VOLUME.AMBIGUOUS_REAL_TICK`, `SOURCE.SYNTHETIC_TEST_ONLY`

22.4 Appendix D — Architecture decision records

Table 50. Architecture decision records

Record	Decision
0001	A simulation-only foundation: no broker path until the research harness exists
0002	Guarded reactivation of the emergency stop as a backend-owned, audited transition

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

Record	Decision
0003	Content-addressed Parquet layers for historical data, identity by canonical hash
0004	Binding hostile-file preflight to commit, so a staged file cannot change in between
0005	A demo-only MQL5 safety bridge with a strict command protocol
0006	MT5 demo-account execution behind configuration, account, Guardian and reconciliation gates
0007	Unattended AWS demo operations: service account, scheduled tasks, monitor, alerts and backups

22.5 Appendix E — Formula sheet

$$R_i = (exit_i - entry_i) \times d_i \ / \ | entry_i - stop_i |$$

$$E = (1/N) \cdot \Sigma R_i \qquad PF = \Sigma R^{+} / | \Sigma R^{-} | \qquad p^{*} = 1 / (1 + b)$$

$$DD = max_k (max_{j \le k} Q_j - Q_k), \quad Q_k = \Sigma_{i \le k} R_i$$

$$volume = floor_to_step(equity \times risk\% / (|entry - stop| \times contract_size))$$
clamped to [min_volume , max_volume]

$$confidence = \max conf(s) \times m(risk), \quad m = 1.00 / 0.85 / 0.60$$

$$|z| \ge \Phi^{-1}(1 - \alpha / (2k)), \quad \alpha = 0.05$$

$$EMA(t) = \alpha \cdot C(t) + (1-\alpha) \cdot EMA(t-1), \quad \alpha = 2/(n+1)$$

$$RSI = 100 - 100 / (1 + G/D) \quad \text{(Wilder smoothing, } n = 14)$$

22.6 Appendix F — Figures from the cloud viewer

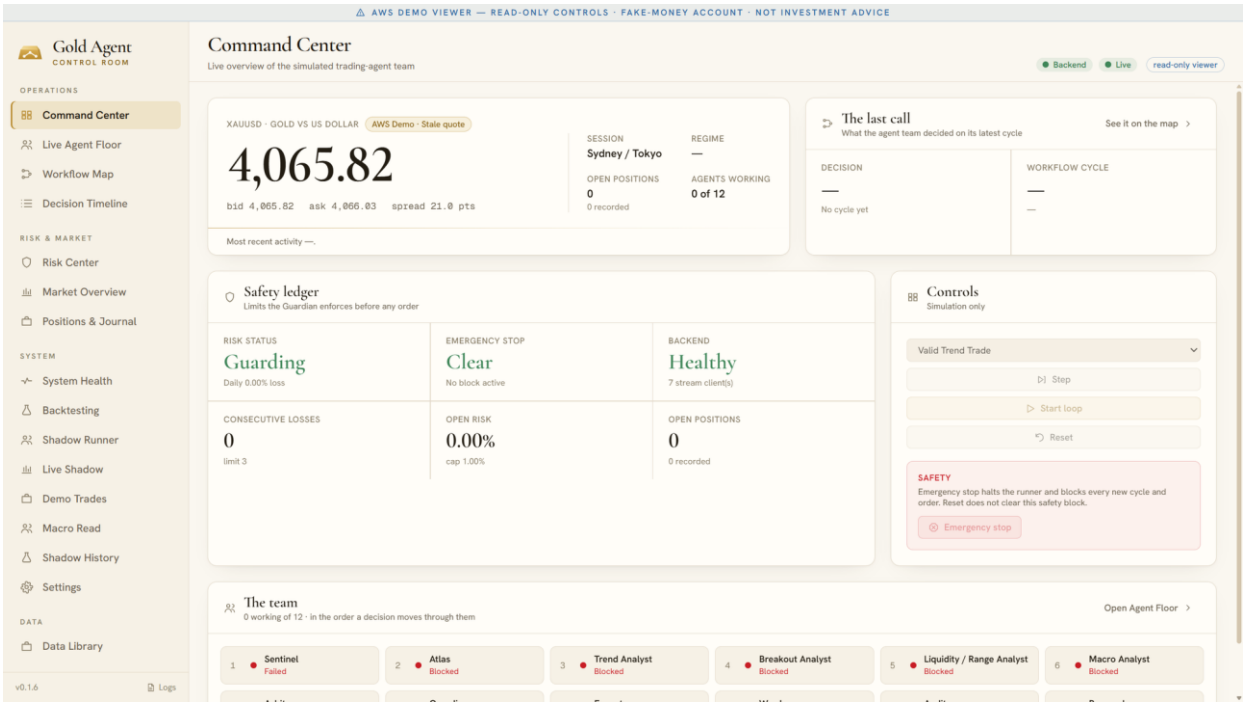


Figure 32. Command Center in the cloud viewer on a Sunday: the price and safety ledger are live, but no cycle runs while the market is closed.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only

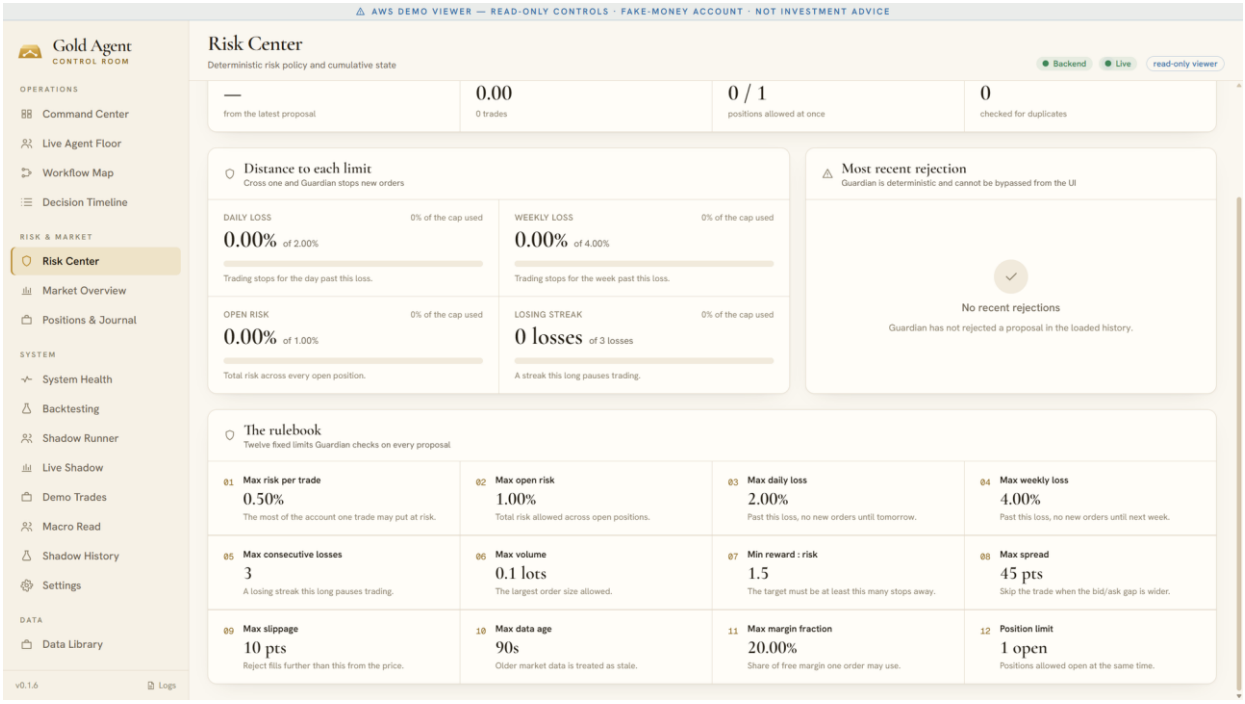


Figure 33. Risk Center rulebook in the cloud viewer: all twelve Guardian rules with their limits and explanations.

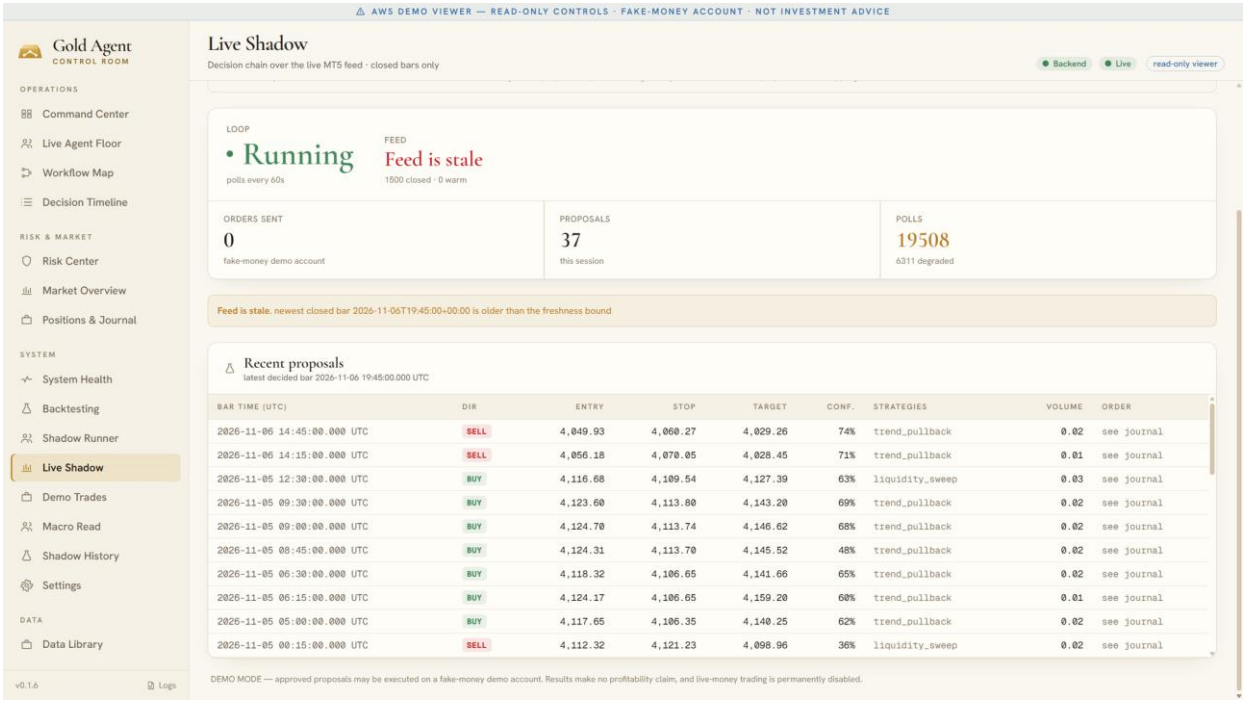


Figure 34. Live Shadow in the cloud viewer: the table of recent live proposals with entry, stop, target, confidence and strategy.

© 2026 Ilyas Daoud El Asmi · Simulation and demo results only